סיכום הרצאות - אלגוריתמים מתקדמים

2 בנובמבר 2026

גיא יער־און

סיכום זה נכתב במהלך סמס א' תשפ"ז (2027) במהלך הרצאותיהם של פרופסור אלי פורת ופרופסור משה לויינשטיין.
השימוש בסיכום באחריותכם בלבד.

תוכן עניינים

1 תכנות לינארי (LP) 2
1.1 רקע 2
1.2 צורות הצגה של LP 3
1.2.1 הצורה הכללית (Generl form) 3
1.2.2 הצורה הסטנדרטית (Standard form) 3
1.2.3 משתני עזר (Slack form) 3
1.3 Converting Unrestricted Variables (המרת משתנים חופשיים) 3
1.4 אלגוריתם simplex 4
1.4.1 דוגמה להרצת האלגוריתם 5
1.4.2 מקרי קצה 6
1.4.3 פסאודו קוד 8
1.4.4 זמן ריצה 9
1.5 אלגוריתם הEllipsoid 9
1.6 Integer Programming vs. Linear Programming 10
1.7 כיסוי קודקודים (Vertex Cover) 10
1.8 קליקה (Clique) 11
1.9 Weak Duality 11
1.9.1 דוגמה 11
1.9.2 הגדרה פורמלית 12
1.10 תרגול 1 13
1.11 Strong Duality 18
1.12 אלגוריתמי קירוב ותכנות לינארי 20
1.12.1 אלגוריתם קירוב לVC עם תכנות לינארי 20
1.12.2 Set Cover 21
1.12.3 הגרסה הדואלית של Set Cover 22
1.12.4 Greedy Set Cover 23
1.13 Max Flow Min Cut 25
1.14 תרגול 2 25
2 Succinct Data Structures: מבני נתונים תמציתיים 29
2.1 Rank Problem 30
2.1.1 אלגוריתם ראשון 30
2.1.2 אלגוריתם שני 30
2.1.3 אלגוריתם שלישי 30
2.1.4 אלגוריתם רביעי 30
2.1.5 אלגוריתם חמישי 31
2.1.6 אלגוריתם שישי 31

2.2 Select Problem 31
2.2.1 פתרון ראשון 32
2.2.2 פתרון שני 32
2.2.3 פתרון שלישי 32
2.2.4 פתרון רביעי 32
2.3 Range Minimum Query (RMQ) 32
2.3.1 פתרון ראשון 33
2.3.2 פתרון שני 33
2.3.3 פתרון שלישי 33
2.3.4 פתרון רביעי 34
3 Approximation Algorithms־ אלגוריתמי קירוב 34
3.1 חישוביות וסיבוכיות 34
3.2 אלגוריתם קירוב של VC 36
3.3 Max Cut 36
3.3.1 אלגוריתם הסתברותי 37
3.3.2 אלגוריתם חיפוש מקומי 37
3.4 Minimum Makespan Scheduling 38
3.4.1 אלגוריתם חמדן 39
3.5 סכמות קירוב 40
3.6 מרחבים מטריים (Metric space) 41
3.6.1 אלגוריתם גונזלס (Gonzalez Algorithm) לבעיית הk-center 42
3.7 Bin Packing Problem 45
3.7.1 First fit alogorithm 45
3.7.2 PTAS עבור Bin-Packing 46
3.8 Minimum Multiway Cut 50
3.9 תרגול 51

1 תכנות לינארי (LP)

1.1 רקע

בהרצאה הקרובה נעסוק בפרדיגמה של תכנות לינארי. נלמד לנסח בעיות מעשיות באמצעות מודל מתמטי לינארי לחלוטין, נלמד 3 צורות לייצוג מתמטי של תכנות לינארי ונלמד זוג אלגוריתמים:

- Simplex algorithm: אלגוריתם פרקטי ונפוץ לפתרון כללי של בעיות תכנות לינארי.
- Ellipsoid algorithm: אלגוריתם בעל סיבוכיות פולינומיאלית אשר מבטיח פתרון יעיל, תאורטית.

בסיום, נקשר כל בעיית LP לבעיה מקבילה לה ־ בעיה דואלית, אשר תספק עבורנו ליצירת חסמים וכו'.

דוגמה 1. נתבונן בדוגמה ראשונה, פשוטה למדי, לבעיית מקסימיזציה תחת מגבלת משאבים.
נניח חברה שמייצרת שולחנות וכיסאות.
שולחן (x), דורש 4 שעות נגרות ו2 שעות גימור. נניח כי הרווח מכל שולחן הינו 70$.
כיסא (y), דורש 3 שעות נגרות ושעה 1 של גימור. נניח כי הרווח מכל כיסא הוא 50$.
ברשות המפעל: 240 שעות נגרות לכל היותר, 100 שעות גימור.
השאלה שלפנינו: כמה שולחנות וכמה כסאות ־ עלינו לייצר על מנת להביא את סך הרווח למקסימום תחת מגבלת שעות העבודה הזמינות?
נקבל, אם כן, את בעיית האופטימיזציה הבאה:

$$\max\ (70x+50y)\ \text{s.t}\begin{cases}4x+3y\leq 240\\ 2x+y\leq 100\\ x,y\geq 0\end{cases}$$

1.2 צורות הצגה של LP

1.2.1 הצורה הכללית (Generl form)

למעשה, זוהי הצורה הטבעית והגמישה ביותר שבה הבעיה מוגדרת מלכתחילה.

- פונקציית המטרה יכולה להיות מסוג מקסימום או מינימום.
- האילוצים יכולים להיות כל אחד מהבאים: $=, \leq, \geq, >, <$ והמשתנים יכולים להיות חופשיים. כלומר, ללא אילוצי סימן (יכולים להיות חיוביים, שליליים או אפס)

1.2.2 הצורה הסטנדרטית (Standard form)

צורה אחידה ונוחה לניתוח מתמטי ולחלק מן האלגוריתמים שנציג בהמשך.

- פונקציית המטרה חייבת להיות מסוג MAX (כלומר, אם הבעיה המקורית הייתה מזעור - min, אזי ממירים אותה למקסום ע"י הכפלת פונקציית המטרה ב-1).
- אילוצים: כל אחד מהאילוצים מהצורה $\leq$ בלבד. וכן, כל המשתנים צריכים להיות אי שליליים.

דרך זו ניתן לייצג באופן מתמטי בשני אופנים, שקולים:

הגדרה 2. ייצוג הצורה הסטנדרטית מוגדר להיות מהצורה הבאה:

$$\max \sum_{i=1}^{n} c_i x_i \text{ s.t } \sum_{i=1}^{n} a_{1,i} x_i \leq b_1, ..., \sum_{i=1}^{n} a_{m,i} x_i \leq b_m$$

ובאופן שקול:

$$\max c^T x \text{ s.t } Ax \leq b$$

באשר, $c \in \mathbb{R}^n$ הוא וקטור המקדמים של פונקציית המטרה מגודל n, וx הוא וקטור המשתנים. $A \in \mathbb{R}^{m \times n}$ היא מטריצת האילוצים - שורה לכל אילוץ, עמודה לכל משתנה. $b \in \mathbb{R}^m$ הוא וקטור מהצורה $(b_1, ..., b_m)$. וכן, $\forall i : x_i \geq 0$.

1.2.3 משתני עזר (Slack form)

בצורה זו משתמשים במשוואת שוויון, היא הצורה הדרושה ישירות על מנת להריץ את אלגוריתם הסימפלקס.

- פונקציית המטרה מסוג MAX בלבד.
- כל האילוצים הם משוואות שוויון, מגיעים לכך ע"י הוספת משתני עזר המייצגים את ההפרש בין האגפים. כמו כן, כל המשתנים חייבים להיות אי שליליים.

אם כן, כיצד נהפוך מערכת אילוצי אי שוויון לאילוצי שוויון? על מנת להפוך אילוץ מהצורה $\sum_{i=1}^{n} a_i x_i \leq b_1$ לשוויון, מוסיפים משתנה אי שלילי s_1 שיקרא משתנה slack. משתנה זה מייצג את המקום הפנוי עד להגעה לגבול. פונקציית המטרה נותרת כשהייתה, ומקבלים:

$$\max \sum_{i=1}^{n} c_i x_i \text{ s.t } \sum_{i=1}^{n} a_{1,i} x_i + s_1 = b_1, ..., \sum_{i=1}^{n} a_{m,i} x_i + s_m = b_m \text{ s.t } \forall i \in [n], j \in [m] : x_i, s_j \geq 0$$

משפט 3. שלושת הדרכים לייצוג בעיית תכנות לינארי שקולות.

1.3 Converting Unrestricted Variables (המרת משתנים חופשיים)

טכניקה נוספת להתמודד עם בעיה חשובה: כיצד נתמודד עם משתנה שאין עליו אילוץ אי־שליליות? הרי, לפי הגישה עליה הרחבנו קודם המשתנים חייבים להיות אי־שליליים. נצטרך להעביר את המשתנה לצורה שבה $z \geq 0$. לשם כך הרעיון המרכזי הוא פשוט: כל מספר ממשי ניתן להציג כהפרש של זוג מספרים אי שליליים. לכן, בהינתן משתנה חופשי z, ניתן לפצלו באופן הבא:

$$z = z_+ - z_- \text{ s.t } z_+, z_- \geq 0$$

דוגמה 4. נתבונן בבעיית האילוצים הבאה:

$$\max\ (5x+3y)\ \text{s.t}\begin{cases} x+3y\leq 10 \\ 2x+y\leq 15 \\ x\geq 0 \end{cases}$$

על המשתנה y אין הגבלה לפיה $y\geq 0$. אך, כפי שלמדנו כל המשתנים חייבים בשיטה זו להיות אי שליליים. לכן, נוכל לבצע המרה ולקבל באופן שקול:

$$\max\ (5x+3y_+-3y_-)\ \text{s.t}\begin{cases} x+3y_+-3y_-\leq 10 \\ 2x+y_+-y_-\leq 15 \\ x\geq 0 \\ y_+\geq 0 \\ y_-\geq 0 \end{cases}$$

1.4 אלגוריתם simplex

המטרה העליונה של האלגוריתם היא למצוא את הפתרון האופטימלי אשר ממקסם את פונקציית המטרה מתוך אינסוף הפתרונות האפשריים אשר מוגדרים ע"י האילוצים. גיאומטרית, מרחב הפתרונות של אילוצי הLP יוצר צורה גאומטרית של רשת קודקודים. האלגוריתם לא סורק את כל השטח האינסופי הנ"ל, אלא "מטפס" מקודקוד לקודקוד על גבי המעטפת של האזור המותר של הפתרון, כך שבכל צעד הרווח רק גדל או נשאר זהה, עד שמגיע לקודקוד הגבוה ביותר בו נמצא המקסימום האבסולוטי.
פונקציית המטרה הינה:

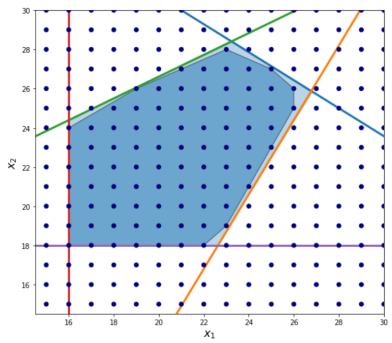
$$\max_{x=(x_1,\ldots,x_n)} c^T x$$

$$\forall i\in[m]: s_i=b_i-\sum_{j=1}^{n}a_{i,j}x_j\ \text{s.t}\ \forall i,j: x_i\geq 0, s_j\geq 0$$

באשר c מוגדר להיות וקטור המקדמים של המשתנים.

הגדרה 5. (Basic Feasible Solution). נקרא למשתנים $\{x_i\}_{i=1}^n$ משתנים לא בסיסיים (אלו של פונקציית המטרה), ויהיו $\{s_i\}_{i=1}^m$ משתני הבסיס של המערכת, שכן כל אחד מהם מבודד במשוואה משלו. מצב בו כל המשתנים הלא בסיסיים מאפסים את עצמם נקרא BFS (פתרון בסיסי פיזיבלי). אלגוריתם הSimplex מתחיל לרוץ במצב זה בו $x_1=x_2=\ldots=x_n=0$, במצב זה מתקיים כי לכל i: $s_i=b_i$ בתחילת הריצה.

הערה 6. נתבונן לצורך ההמחשה במערכת אילוצים על זוג משתנים x_1,x_2. האזור הצבוע בכחול הוא תחום ההצבה האפשרי למשתנים - כל נקודה בתוך המצולע הכחול מקיימת את האילוצים. הישרים - הם הגבולות שיוצרים את המצולע. הנקודות בתוך המצולע הכחול: אלו הנקודות של המשתנים בשלמים בתוך האזור של הפתרונות החוקיים.



1.4.1 דוגמה להרצת האלגוריתם

עוד לפני שנציג פסודו של האלגוריתם, נרצה לראות דוגמת הרצה עליו. נזכר בבעיית האופטימיזציה על השולחנות והכסאות שראינו בתחילת ההרצאה:

$$\max\ (70x+50y)\ \text{s.t}\begin{cases}4x+3y\leq 240\\2x+y\leq 100\\x,y\geq 0\end{cases}$$

נרצה להפוך אותה לבעיית אופטימיזציה מסוג שוויון, באמצעות משתני הSlack:

$$\max\ (70x+50y)\ \text{s.t}\begin{cases}4x+3y+s_1=240\\2x+y+s_2=100\\x,y,s_1,s_2\geq 0\end{cases}\implies \max\ (70x+50y)\ \text{s.t}\begin{cases}s_1=240-4x-3y\\s_2=100-2x-y\\x,y,s_1,s_2\geq 0\end{cases}$$

בשלב ראשון של האלגוריתם, מאפסים את המשתנים לפי BFS: $x=0,y=0\implies s_1=240,s_2=100$. במצב זה אנו למעשה לא מייצרים שום רהיטים, ומשתני העזר מקבלים את מלוא המשאבים הפנויים. כלומר נק' ההתחלה שנקראת BFS: Basic Feasible solution הוא פתרון בסיסי התחלתי בו כל המשתנים החופשיים הינם אפס בדיוק, ומשתני ההרפיה סופגים את כל המשאבים.

כעת, נרצה להגדיל את הרווח שלנו. שכרגע, הוא בדיוק אפס. נתבונן על המקדמים בפונקציית המטרה: המקדם של x הינו 70, גדול מהמקדם של y שהינו 50. לכן המסקנה הברורה: לx יש השפעה חיובית חזקה יותר על הרווח. לכן - הגיוני שהאלגוריתם ירצה להגדיל כמה שיותר את הערך של x. בשלב זה אנחנו נכניס "משתנה חדש למשחק" - פעולה זו נקראת Pivot. כעת נרצה לאפשר לx להפוך להיות משתנה פעיל. לפי המשוואה השניה ניתן לקבל:

$$x=50-\frac{y}{2}-\frac{s_2}{2}$$

כעת, נוכל להסתכל על הבעיה הבאה:

$$\max\ (70\cdot(50-\frac{y}{2}-\frac{s_2}{2})+50y)\ \text{s.t}\begin{cases}s_1=240-4\cdot(50-\frac{y}{2}-\frac{s_2}{2})-3y\\x=50-\frac{y}{2}-\frac{s_2}{2}\\x,y,s_1,s_2\geq 0\end{cases}$$

כלומר,

$$\max\ (3500+15y-35s_2)\ \text{s.t}\begin{cases}s_1=40-y+2s_2\\x=50-\frac{y}{2}-\frac{s_2}{2}\\x,y,s_1,s_2\geq 0\end{cases}$$

מה שינינו כאן בעצם? הרי, לכאורה ניתן לומר שסה"כ החלפנו את אותה משוואה במשוואה שקולה לה. ובכן, כמעט: אנחנו החלפנו את תפקידי המשתנים. בתחילה - x,y היו בחוץ (שווים לאפס), וs_1,s_2 היו בפנים - הם היו המשתנים שניהלו את העניינים. במצב זה, הרווח היה אפס. לאחר פעולת הPivot, קיבלנו משתנים לא בסיסיים חדשים: s_2,y.

שלב הבא: מאפסים את המשתנים הלא בסיסיים ומקבלים $s_2=y=0$. במצב זה ערך פונקציית המטרה הוא 3500. כלומר, מרווח של 0 עלינו לרווח של 3500 בצעד אחד בודד. בדומה, שוב נסתכל על המקדמים של המשתנים הלא בסיסיים. המקדם של y: 15 והמקדם של s_2: -35. כלומר, נדרשת איטרציה נוספת. כעת, נבודד את y כי המקדם שלו גבוה יותר וחיובי ממש. כלומר: יש דרך עוד לשפר אותו. לכן נבודדו ונקבל:

$$y=40+2s_2-s_1$$

ומכאן:

$$\max\ (3500+15\cdot(40+2s_2-s_1)-35s_2)\ \text{s.t}\ \begin{cases} y=40+2s_2-s_1 \\ x=50-\frac{40+2s_2-s_1}{2}-\frac{s_2}{2} \\ x,y,s_1,s_2\geq 0 \end{cases}$$

כלומר נקבל:

$$\max\ (4100-5s_2-15s_1)\ \text{s.t}\ \begin{cases} y=40+2s_2-s_1 \\ x=30+\frac{s_1}{2}+\frac{2s_2}{2} \\ x,y,s_1,s_2\geq 0 \end{cases}$$

בשלב זה, משתני ההרפיה הם x,y וכן משתני הבסיס הם s_1,s_2. כל המקדמים של משתני הבסיס שליליים, ולכן האלגוריתם מפסיק ריצתו. לפי BFS, נקבל כי $s_1=s_2=0$ ומהמשוואות נקבל כי הפתרון האופטימלי הינו:

$$x=30, y=40$$

הערה חשובה. נשים לב כי בכל שלב למשל בשלב של בידוד y היה לנו 2 משוואות שיכולנו לבודד מהם אותו. בשלב זה אנחנו מפעילים את **מבחן היחס המינימלי** אשר מטרתו להבטיח שכל משתני הבסיס ישארו אי שליליים. לדוגמה היה לנו $s_1=40-y+2s_2$. אם נציב $s_2=0$ (מדוע? השני במשתנים הלא בסיסיים) ונקבל $s_1=40-y$ לכן $y\leq 40$. כי $s_1\geq 0$. מהמשוואה השניה, אם נציב $s_2=0$ נקבל $x=50-\frac{y}{2}$. מכאן $y\leq 100$. לכן - נבחר את המשוואה הראשונה. בוחרים את החסם המחמיר יותר. אנחנו לוקחים את המחמיר מבניהם על מנת להבטיח ששאר משתני הבסיס ישארו אי שליליים.

1.4.2 מקרי קצה

עד כה הנחנו כי $b\geq 0$. כלומר, הוקטור החופשי ההתחלתי של האילוצים הוא חיובי או אפס. מה אם ההנחה הזו נכשלת ויש לנו לפחות ערך שלילי אחד בb? במצב כזה אי אפשר לאפס בכל שלב את המשתנים המקוריים $x=0$, כי אז נקבל כי $s_i=b_i<0$, כלומר משתני העזר יהפכו להיות שליליים וזה לא חוקי לפי כללי הLP. כלומר: אין לנו בכלל נקודת התחלה חוקית. כיצד פותרים את הבעיה הזו? לשם כך, נייצר בעיית עזר. על מנת למצוא קודקוד התחלתי חוקי שממנו נוכל להתחיל לרוץ עם סימפלקס, נבצע טריק מתמטי (שלב זה נקרא Phase 1).

- מכניסים משתנה חדש למערכת: x_0. מוסיפים את המשתנה המלאכותי לכל משוואות האילוצים, באגף ימין שלו עם סימן חיובי.
- משנים את פונקציית המטרה: במקום לנסות למקסם את הרווח המקורי שלנו, משנים את המטרה למטרה הבאה:

$$\max -x_0$$

כלומר, כיוון ש$x_0\geq 0$, הדרך למקסם את $-x_0$ היא לגרום לכך ש$x_0=0$.

$$\begin{array}{c} \max c^Tx \\ s_1=b_1-\sum_{i=1}^n a_{1,i}x_i \\ .. \\ s_m=b_m-\sum_{i=1}^n a_{m,i}x_i \end{array} \quad\Longrightarrow\quad \begin{array}{c} \max -x_0 \\ s_1=b_1+x_0-\sum_{i=1}^n a_{1,i}x_i \\ .. \\ s_m=b_m+x_0-\sum_{i=1}^n a_{m,i}x_i \end{array}$$

- מטרת הטריק: אם נצליח להריץ את אלגוריתם הסימפלקס ולהביא למצב שבו $x_0=0$, מצאנו פתרון בו כל האילוצים מתקיימים באופן חוקי - אנו מחזיקים בBFS חוקי שאפשר איתו לחזור לבעיה המקורית של מקסום הרווח ולרוץ עם האילוצים שלה. אם בסוף התהליך $x_0>0$ המשמעות היא שהבעיה המקורית כלל לא אפשרית ואין לה אף פתרון חוקי.
- כיצד מריצים זאת טכנית? על מנת להריץ את אלגוריתם הסימפלקס על בעיית העזר הנ"ל, אנחנו צריכים להכניס את x_0 לבסיס במקום השורה הבעייתית בה יש את הb השלילי ביותר. לכן, יהי

$$\ell=\arg\min_i(b_i)$$

כלומר, ℓ הוא האינדקס שבו הערך של b הוא הקטן ביותר ומכאן $b_\ell < 0$. לוקחים את המשוואה של אותו אילוץ בעייתי בה הוא מופיע ומסדרים אותה מחדש על מנת לבודד את x_0 באופן הבא:

$$-x_0 = b_\ell - s_\ell - \sum_{j=1}^{n} a_{\ell,j}x_j \Longrightarrow x_0 = -b_\ell + s_\ell + \sum_{j=1}^{n} a_{\ell,j}x_j$$

מציבים את הביטוי הנ"ל לשאר המשוואות ולתוך פונקציית המטרה החדשה, ומפעילים את אלגוריתם הסימפלקס הרגיל על מנת למקסם את $-x_0$. נשים לב שכיוון ש$b_\ell < 0$ אזי $-b_\ell > 0$.

באשר האלגוריתם יעצור, אם נגיע למצב שבו הערך של x_0 ירד והתאפס לחלוטין מצאנו נקודה חוקית שבה כל משתני הs חיוביים - כעת ניתן להתעלם מx_0, לחזור לפונקציית המטרה המקורית עם האילוצים החדשים שהם BFS ולפתור את הבעיה האמיתית. כלומר:

$$\begin{array}{c}
\max b_\ell - s_\ell - \sum_{i=1}^{n} a_{\ell,i}x_i \\
s_1 = b_1 + x_0 - \sum_{i=1}^{n} a_{1,i}x_i \\
.. \\
x_0 = -b_\ell + s_\ell + \sum_{j=1}^{n} a_{\ell,j}x_j \\
s_m = b_m + x_0 - \sum_{i=1}^{n} a_{m,i}x_i
\end{array}$$

כעת, אנחנו מחזיקים בבעיה שמשלימה את ההכנה של מערכת המשוואות ויש לנו פונקציית מטרה ומערכת אילוצים בה ניתן לאפס את כל המשתנים החופשיים $(x_1, ..., x_n, b_\ell)$. לאחר האיפוס שלהם, כל משתני האילוץ נוצרים חיוביים. שכן לאחר שנציב את x_0 בכל משוואות האילוצים נקבל:

$$\begin{array}{c}
\max b_\ell - s_\ell - \sum_{i=1}^{n} a_{\ell,i}x_i \\
s_1 = (b_1 - b_\ell) + s_\ell - \sum_{i=1}^{n} (a_{1,i} - a_{\ell,i})x_i \\
.. \\
x_0 = -b_\ell + s_\ell + \sum_{j=1}^{n} a_{\ell,j}x_j \\
s_m = (b_m - b_\ell) + s_\ell - \sum_{i=1}^{n} (a_{m,i} - a_{\ell,i})x_i
\end{array}$$

נשים לב כי כעת לאחר שמציבים $x_1 = ... = x_n = b_\ell = 0$ מקבלים כי לכל $i \in [m]$ מתקיים $(b_i - b_\ell) \geq 0$ ויחד עם שאר האיברים, סה"כ נקבל כי המשתנים $s_i \geq 0$. כלומר, הצלחנו למנוע את המצב שבו לאחר הצבת פתרון הBFS המשתנים הופכים לשליליים.

על המערכת הנ"ל מריצים את אלגוריתם הסימפלקס עד שפונקציית המטרה מגיעה למקסימום שלה, בעת ש$x_0 = 0$ סיימנו את השלב הראשון ונגיע למערכת משוואות מהצורה הבאה:

$$\begin{array}{c}
y_1 = b'_1 - \sum_{i=1}^{n} a'_{1,i}z_i \\
y_2 = b'_2 - \sum_{i=1}^{n} a'_{2,i}z_i \\
.. \\
y_m = b'_m - \sum_{i=1}^{n} a'_{m,i}z_i \\
\forall i : z_i \geq 0, \forall j : y_j \geq 0
\end{array}$$

הרי בסיום שלב העזר, האלגוריתם עוצר: על מנת לקרוא את הערכים של המשתנים בנק' הנוכחית שזהו הקודקוד שמצאנו, אנו מאפסים את כל המשתנים שיושבים בצד (כאן אלו $z_1, ..., z_n$). לשאר משתני המערכת - נקבל לכל i: $y_i = b'_i$. כלומר, בסיום נקבל כי $y = b'$ הוא וקטור ערכים המקיים $b' \geq 0$. לכן מצאנו קודקוד התחלתי חוקי ואמיתי ממנו ניתן להתחיל לפתור את האופטימיזציה של הבעיה האמיתית.

מסקנה 7. ניתן להסיר את x_0 ולקבל שישנו פתרון פיזיבלי למערכת אם ורק אם בסיום התהליך $x_0 = 0$. מדוע? הוא הרי לא היה משתנה אמיתי מלכתכילה של הבעיה שלנו. הוא היה שתול מלאכותי שנועד על מנת לחלץ אותנו ולמצוא לנו נק' התחלה חוקית. אם הוא לא הגיע לאפס בסיום, משמעות הדבר שאין פתרון חוקי. כלומר, אם הרצנו את הסימפלקס על המערכת המלאכותית שבנינו וגילינו בסיום כי בהכרח $x_0 > 0$ אזי אין שום שילוב בעולם של מס' אי שליליים שיכולים לספק את מערכת המשוואות. האילוצים פשוט סותרים זה את זה - למשל: אילוץ שדורש שהסכום לא יעבור את 50 ואילוץ אחר שדורש שיהיה לפחות 51. לכן - הבעיה *infeasible*.

בסיום, נקבל את בעיית האופטימיזציה הבאה:

$$\begin{gathered}\max\ (c')^T z \text{ s.t } z=(z_1,....,z_n)\\ y_1 = b'_1 - \sum_{i=1}^{n} a'_{1,i} z_i\\ y_2 = b'_2 - \sum_{i=1}^{n} a'_{2,i} z_i\\ ..\\ y_m = b'_m - \sum_{i=1}^{n} a'_{m,i} z_i\\ \forall i : z_i \geq 0, \forall j : y_j \geq 0\end{gathered}$$

וכן $c'_i = c_i + \sum_{j=1}^{m} y_j a'_{j,i}$ (כלומר אלו ערכי המקדמים החדשים בפונקציית המטרה).

1.4.3 פסאודו קוד

להלן תיאור של האלגוריתם בפסאודו קוד:

Algorithm: Simplex with Phase 1 & Phase 2

Input: Matrix A, vectors b and c
Phase 1: Feasibility

1. Find the most negative b_i (let's call it b_ℓ)
2. If $b_\ell \geq 0$: already feasible! Skip to Phase 2.
3. Else:
   (a) Add artificial variable x_0 and set objective to Max: $-x_0$
   (b) Pivot on the b_ℓ row to make x_0 basic
   (c) Run Simplex until $\max(-x_0)$ is reached
   (d) If optimal $x_0 > 0$: return "Infeasible" (no solution)
   (e) If optimal $x_0 = 0$:
      i. Remove x_0 from the system
      ii. We now have a valid initial BFS ($b' \geq 0$)

Phase 2: Optimality

1. Restore the original objective function (expressed with current non-basic variables z, yielding c')
2. While there is any non-basic variable with a positive coefficient in c':
   (a) Choose an entering variable (positive coefficient in c')
   (b) Perform the Minimum Ratio Test to choose the leaving variable
   (c) Perform Pivot
   (d) If no variable can leave (all coefficients ≤ 0 for the entering variable): return "Unbounded"
3. Return the optimal solution

האלגוריתם לעיל מתאר באופן פורמלי את התהליך שתיארנו. ביתר פירוט, בשלב ב' אנו מחזיקים בפתרון BFS ומעוניינים לשפר אותו. למעשה השיפוע הוא המקדם בc'. לכן, בוחרים מקדם חיובי c' (כלל בחירה הגיוני הוא המקסימלי בשיפוע) ומתקדמים.
שלב שבו נרצה להתעכב הוא הRatio Test: בעת שמצאנו משתנה עם שיפוע חיובי, נרצה לסדר את המערכת. נזכר בדוגמה שראינו בתחילה, שם בשלב הראשון בחרנו לסדר את המשוואה על x ולקבל:

$$x = 50 - \frac{y}{2} - \frac{s_2}{2}$$

כלומר הגדלנו אותו ל50, זהו בדיוק השלב שבו מתבצע הRatio Test. הרי, יכולנו לסדר את המשוואה בצורה אחרת ולהגדילו לפי המשוואה הראשונה (שם היינו מגדילים אותו ל60). מדוע לא עשינו זאת? תמיד נבחר במשוואה של משתנה הלא־בסיס שמגדילה כמה שבפחות. שכן, כל הפתרונות האחרים אינם חוקיים. הרי, כדי להשאר חוקי בכל האילוצים בו זמנית מוכרחים לעצור בתקרה הנמוכה ביותר מבניהן.

ביצוע הPivot הוא השלב בו x מוצב לתוך בעיית האופטמיזציה החדשה והופך למשתנה בסיס, ומקבלים בעיית אופטימיזציה חדשה.

1.4.4 זמן ריצה

נשים לב כי כל הפעולות האלגבריות בסימפלקס הינן פעולות אלמנטריות על שורות, ערך פונקציית המטרה יישאר עקבי ושקול לאחר הפעולות ומשוואת המטרה תמיד תובע במונחי המשתנים הלא בסיסיים בלבד. ברגע שבוחרים קבוצה קבועה של משתנים לא בסיסיים, מקבלים פתרון בסיסי אפשרי יחיד ומוגדר היטב.

למרות שבתאוריה, הסיבוכיות גבוהה מאוד, בפועל האלגוריתם רץ מהר מאוד על בעיות אמיתיות. כמו כן, ישנן מס' היורסטיקות שונות לבחירת Pivot כדי לשפר את מהירות זמן הריצה. חייבים להזהר ולא להכנס ללולאות אינסופיות - לכן ישנו כלל בלנד עליו לא הרחבנו בקורס.

משפט 8. זמן הריצה של אלגוריתם Simplex הוא:

$$\mathcal{O}\left(\binom{n+m}{n}\cdot nm\right)$$

באשר n הוא מס' המשתנים הלא בסיסיים, וכן m הוא מס' האילוצים. במערכת סה"כ יש $n+m$ משתנים.

- בחלק הראשון: למעשה אלגוריתם הסימפלקס כל האלגוריתם עובר מקודקוד לקודקוד בתוך פאון גאומטרי. קשה לחסום את זה - אבל ישנו חסם תאורטי מתמטי למספר הפעמים שזה יקרה. הרי מה קורה בפועל? בכל פעם בוחרים בסיס. ברגע שיש בסיס שמשתמשים בו, הוא לא ישוב שוב. לכן ניתן לומר שבחירת הבסיס האופטימלי שקולה לבחירת n משתנים מתוך $n+m$ משתנים - כך שn הנבחרים יהיו לא בסיסיים. מס' הדרכים לעשות זאת כמובן $\binom{n+m}{n}$.
- בחלק השני: כל בחירת בסיס כזה מביעה צעד Pivot. מה כולל צעד פיבוט בודד? שלושה שלבים.

1. בחירת המשתנה הנכנס - בדיקת פונקציית המטרה ולקיחת המשתנה בעל השיפוע הגדול ביותר.
2. בחירת המשתנה היוצא - בודקים את האילוצים באמצעות מבחן היחס המינימלי כדי לזהות איזה משתנה בסיס קיים יגיע לאפס ראשון ויחסום את הגדלתו של המשתנה הנכנס.
3. עדכון אלגברי של הטבלה: זה החלק שלוקח רוב הזמן. יש לעדכן את המטריצה בגודל $n\times m$ כך שאנחנו מחליפים תפקידים: כלומר פיבוט זו ההצבה שאנחנו עושים למשתנה הבסיס שנבחר על מנת שיכנס בכל המקומות כמשתנה לא בסיסי.

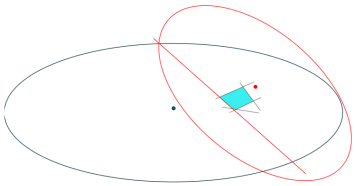
לכן סה"כ שלב זה עולה $\mathcal{O}(n\cdot m)$.

הערה 9. נשים לב שהסימפלקס לא פשוט "עובר על כל האפשרויות" בצורה נאיבית, אלא מתקדם מפתרון פיזיבלי עד שיגיע לפתרון שבו כל המקדמים אי שליליים. עם זאת, במקרה הגרוע ביותר הוא אכן יתחיל מנקודה גרועה מאוד גאומטרית ויצטרך לעבור על כל האפשרויות לנקודה חוקית עד שיגיע לנקודה נכונה. לכן החסם העליון לזמן הריצה אקספוננציאלי, למרות שמדובר באלגוריתם חמדן שרץ בזמן ריצה טוב למדי.

1.5 אלגוריתם הEllipsoid

בעוד ראינו כי אלגוריתם הסימפלקס הינו אקספוננציאלי, האלגוריתם שנציג הוא פולינומי: וזה ההבדל הגדול בניהם. עד להופעת האלגוריתם, לא היה ידוע אם תכנון לינארי היא בעיה שנמצאת בP. בעת שהגיע האלגוריתם לעולם, הספק אודות השייכות של תכנון לינארי לP נעלם.

ובכן, כיצד קשורה האליפסה? האלגוריתם מתחיל לסרוק את המרחב עם אליפסה ענקית - שבוודאות מכילה בתוכה את כל אזור הפתרונות האפשריים. בתוך האליפסה - צורה גאומטרית קטנה שמכילה את אזור הפתרונות האפשריים לבעיה. בכל איטרציה: האלגוריתם בודק האם מרכז האליפסה נמצא בתוך אזור הפתרונות. אם כן: מצאנו פתרון. אם לא: האלגוריתם מוצא אילוץ מופר שעובר דרך המרכז, ובאמצעותו בונה אליפסה חדשה וקטנה יותר (בהמחשה - זו באדום) שמקיפה רק את החצי של האליפסה הקודמת שבו עדיין עשוי להמצא אזור הפתרונות. בכל צעד האלגוריתם מכווץ את נפח האליפסה בקצב אקספוננציאלי קבוע, עד שננעל על אזור הפתרונות או שמוכיח שהוא ריק.



אלגוריתם האליפסואיד לא יותר לפתור בעיית אופטימיזציה - כלומר למצוא את המקסימום של $c^T x$. לכן מבצעים טריק מתמטי: ממירים את בעיית האופטימיזציה לבעיית החלטה בעזרת חיפוש בינארי:

- נתונה בעיית אופטימיזציה $\max\{c^T x | Ax \leq b, x \geq 0\}$.
- אנו מנחשים ערך מטרה כלשהו M ומנסים לענות על השאלה: האם קיים פתרון כך ש$c^T x \leq M$? יחד עם שאר האילוצים.
- אם האלגוריתם מוצא פתרון, סימן שניתן לשאוף גבוה יותר. לכן ננסה ערך M גבוה יותר. אם האזור ריק, ננסה ערך נמוך יותר של M. כך מריצים חיפוש בינארי על ערך האופטימום M עד שמתכנסים לאופטימום המדויק.

משפט 10. אם קיים פתרון אפשרי למערכת האילוצים $Ax \leq b$ ו$x \geq 0$ אזי תמיד קיים לפחות פתרון אחד שבו כל הערכים קטנים או שווים לחזקה אקספוננציאלית:

$$\forall i : x_i \leq n^{5n}$$

מה המשמעות של המשפט הנ"ל? כדי שאלגוריתם גאומטרי כמו אליפסואיד יעבוד, אנו חייבים לדעת כי אזור הפתרונות לא "בורח" לאנסוף או שקטן מדי כדי שנוכל לתפוס אותו. המשפט הנ"ל מבטיח כי אם קיימת נקודה אפשרית כלשהי, בהכרח קיימת לפחות נקודה אחת שנמצאת בתוך קוביה סופית וחסומה היטב בגודל n^{5n}. כלומר - אנחנו יכולים להתחיל את האלגוריתם עם אליפסה ראשונית ענקית שמכילה קוביה זו, ויודעים בוודאות כי אם יש פתרון: הוא שם בפנים. נעיר כי $x \in \mathbb{R}^n$, לכן הכוונה שכל אחד מהערכים בפתרון $x = (x_1, ..., x_n)$ מקיים את החסם.

משפט 11. המערכת המקורית $Ax \leq b, x \geq 0$ היא בעלת פתרון אם ורק אם המערכת $Ax \leq b + \varepsilon$ s.t $\varepsilon = \frac{1}{n^{5n}}$ היא בעלת פתרון.

מדוע אנו זקוקים למשפט זה עבור נכונות האלגוריתם? במחשבים, מבחינה פרקטית, לא ניתן לייצג מס' ממשיים באינסוף דיוק, לכן אי אפשר לבצע בדיקות אבסולוטיות של "האם הנקודה נמצאת על הישר בדיוק". המשפט הנ"ל מבטיח כי אם נרחיב מעט את האילוצים באופן זעיר, לא נשנה את העובדה שקיים פתרון. כלומר, אם היא פתירה היא תשאר פתירה גם עם ההרחבה הקטנה הזו.

1.6 Integer Programming vs. Linear Programming

הגדרה 12. בעיה מסוג Integer Programming היא בעיה שמנוסחת באופן דומה לLP, בתוספת הדרישה כי המשתנים הם מספרים שלמים. כלומר, לכל i מתקיים $x_i \in \mathbb{Z}$.

משפט 13. IP (Integer Programming) היא בעיה NP-קשה.

כלומר, ישנו הבדל עצום בין מציאת פתרון בשלמים למציאת פתרון כלשהו. בניגוד לפתרון כלשהו, שניתן לפתרון פולינומי באמצעות אליפסואיד, לא ידוע כיום על אלגוריתם פולינומי שפותר את IP וכנראה שלא קיים כזה.

1.7 כיסוי קודקודים (Vertex Cover)

הגדרה 14. יהי $G = (V, E)$ גרף. קבוצת קודקודים $S \subseteq V$ נקראת כיסוי קודקודים אם לכל $(u, v) \in E$ מתקיים $u \in S$ או $v \in S$.

כפי שראינו בחישוביות וסיבוכיות, מדובר בבעיה NP־שלמה. ניתן לייצג אותה באופן מתמטי כבעיית IP. בהינתן כיסוי $S \subseteq V$ לכל $v \in V$ נגדיר משתנה בינארי:

$$x_v = \begin{cases} 1 & v \in S \\ 0 & v \notin S \end{cases} \in \{0,1\}$$

נקבל את בעיית האופטימיזציה הבאה:

$$\min \sum_{v \in V} x_v \text{ s.t } \forall (u,v) \in E : x_u + x_v \geq 1$$

כלומר, IP לפחות קשה כמו VC שהיא הרי NPH. מכאן נקבל כמסקנה שאכן $IP \in NPH$.

1.8 קליקה (Clique)

הגדרה 15. יהי $G = (V,E)$ גרף. קבוצת קודקודים $S \subseteq V$ נקראת קליקה אם לכל $u,v \in S$ מתקיים $(u,v) \in E$.

תהי S תת קבוצה של קודקודים. נגדיר:

$$x_v = \begin{cases} 1 & v \in S \\ 0 & v \notin S \end{cases} \in \{0,1\}$$

נקבל את בעיית האופטימיזציה הבאה:

$$\max \sum_{v \in V} x_v \text{ s.t } \forall (u,v) \notin E : x_u + x_v \leq 1$$

מדוע האילוץ נראה ככה? לפי הגדרת קליקה, כל זו בקבוצה חייב להיות מחובר בקשת. לכן, אם נקח זוג צמתים שאין בניהם קשת בגרף אסור לשניהם להבחר בו זמנית לקליקה. אם אחד מהם יהיה 1, האחר חייב להיות אפס. נשים לב שבאמצעות אילוץ זה אנו אוכפים קליקה. מתי קליקה יכולה להיות בעייתית? אם בחרנו זוג קודקודים שאין ביניהם קשת ־ זו בהכרח לא קליקה. בכל מקרה אחר, כל הקודקודים שנבחרו לקבוצה יש בינם לבין עצמם קשת לכל הזוגות. לכן, נדרוש מכל הזוגות של הקודקודים שאין בניהם קשת, שרק אחד לכל היותר יכנס.

1.9 Weak Duality

כאשר אנחנו פותרים בעיית מקסימיזציה, לפעמים קשה למצוא את הפתרון המדויק או להוכיח מהו החסם העליון שלו. הרעיון של דואליות חלשה, הוא למצוא חסם עליון לערך המקסימלי של פונקציית המטרה ע"י שילוב לינארי של האילוצים הנתונים. לצורך כך, נפתח בדוגמה

1.9.1 דוגמה

נתבונן בבעיית האופטימיזציה הבאה:

$$\max(3x_1 + 2x_2) \text{ s.t } \begin{cases} 2x_1 - x_2 \leq 10 \\ 6x_2 - x_1 \leq 5 \\ x_1, x_2 \geq 0 \end{cases}$$

נסיון ראשון. נקח את זוג האילוצים. נראה כי:
אם נכפול ב־3 את האילוץ הראשון וב־2 את האילוץ השני נקבל:

$$3x_1 + 2x_2 \leq 4x_1 + 10x_2 = 3 \cdot (2x_1 - x_2) + 2(6x_2 - x_1) \leq 3 \cdot 10 + 2 \cdot 5 = 40$$

כלומר, ישנו חסם של 40 לערך פונקציית המטרה.
נסיון שני, חכם יותר. נכפול ב2 את האילוץ הראשון וב1 את האילוץ השני ונקבל:

$$3x_1 + 2x_2 \leq 3x_1 + 4x_2 = 2 \cdot (2x_1 - x_2) + (6x_2 - x_1) \leq 2 \cdot 10 + 5 = 25$$

כלומר, חסמנו את ערך פונקציית המטרה כעת ב25. חסם יותר טוב.
ובכן, במקום לחפש ידנית מספרים ע"י ניחוש - למה שלא נחפש את השילוב האופטימלי שיתן לנו את החסם העליון ההדוק ביותר? יהיו y_1, y_2 משתנים שיחליפו את הערכים הקבועים. נראה כי הצ"ל המשוקלל הינו:

$$y_1 \cdot (2x_1 - x_2) + y_2 \cdot (6x_2 - x_1) \leq 10y_1 + 5y_2 \implies$$

$$(2y_1 - y_2)x_1 + (-y_1 + 6y_2)x_2 \leq 10y_1 + 5y_2$$

נרצה שיתקיים

$$3x_1 + 2x_2 \leq (2y_1 - y_2)x_1 + (-y_1 + 6y_2)x_2 \leq 10y_1 + 5y_2$$

לכן נדרוש כי $2y_1 - y_2 \geq 3, -y_1 + 6y_2 \geq 2$ (זהו חסם על המקדמים). מכאן, נקבל בעיית אופטימיזציה למציאת אבי־אבות החסמים העליונים. כלומר:

$$\min(10y_1 + 5y_2) \text{ s.t} \left\{ \begin{array}{c} -y_1 + 6y_2 \geq 2 \\ 2y_1 - y_2 \geq 3 \\ y_1, y_2 \geq 0 \end{array} \right\}$$

זוהי יצירתו של הDual, מתוך הPrimal. בעיית מקסימיזציה של בחירת משאבים, הופכת לבעיית מינימום של תמחור עבור אותם משאבים. כך למעשה, נוכל למצוא חסם עליון (קטן ביותר, הדוק ביותר - לכן זו בעיית מינימיזציה) לערך פונקציית המטרה של בעיית המקסימיזציה.

1.9.2 הגדרה פורמלית

בהינתן Primal, בעיית מקסימיזציה כללית עם n משתנים וm אילוצים, נרצה למקסם את

$$\max \ c_1x_1 + ... + c_nx_n \text{ s.t} \left(\begin{array}{ccccc} a_{1,1}x_1 + & a_{1,2}x_2 & +.. & + & a_{1,n}x_n \leq b_1 \\ & & & & \\ & & & & \\ a_{m,1}x_1 + & a_{m,2}x_2 & +.. & + & a_{m,n}x_n \leq b_m \end{array} \right)$$

ובאופן שקול:

$$\max c^T x \text{ s.t } Ax \leq b$$

יהיו $y_1, ..., y_m$ משקולות. המטרה שלנו היא למזער את סכום האילוצים כפול המשקולות. כלומר:

$$\min \sum_{j=1}^{m} b_j y_j \equiv \min \ b^T y$$

הרעיון הוא כי לכל אילוץ j בוחרים משקולת $y_j \geq 0$, כופלים בה את האילוץ ומחברים את כל האילוצים. סה"כ מקבלים

$$b_1y_1 + ... + b_my_m = b^T y$$

כלומר - זה מה שיש באגף ימין: מכפלה סקלרית של b החסם ככפל במשקולות. באגף שמאל, לאחר החיבור המקדם של כל משתנה x_i הוא סכום המקדמים שלו בכל האילוצים, כל אחד כפול המשקולת של האילוץ שלו:

$$x_i = a_{1,i}y_1 + ... + a_{m,i}y_m = \sum_{j=1}^{m} a_{j,i}y_j$$

כלומר, סכימת השורה הi של A מוכפלת ב$\{y_i\}_{i=1}^m$. אנו דורשים כי המקדם של כל משתנה בצירוף יהיה לפחות המקדם בפונקציית המטרה. כלומר:

$$\forall i \in [n] : c_i \leq \sum_{j=1}^{m} a_{j,i}y_j$$

סה"כ בכתיב מטריציוני - $A^T y \geq c$.
סה"כ הבעיה הדואלית תוגדר להיות:

$$\min \sum_{j=1}^{m} b_j y_j \ \forall i \in [n] : \sum_{j=1}^{m} a_{j,i}y_j \geq c_i \ , y_1, ..., y_m \geq 0$$

או באופן שקול:

$$\min b^T y \text{ s.t } A^T y \geq c, y \geq 0$$

משפט 16. (משפט הדואליות החלשה) כל פתרון דואלי, בהכרח חוסם את פתרון הPrimal.
הוכחה: מתקיים -

$$b^T y = y^T b \geq_* y^T Ax =_{**} (A^T y)^T x \geq c^T x$$

שכן $*$ נכון כי $b \geq Ax$ וכן $**$ נכון כי מהאילוצים הדואלים דרשנו כי יתקיים $\forall i \in [n] : \sum_{j=1}^{m} a_{j,i}y_j \geq c_i$ ובאופן שקול $A^T y \geq c$.
לכן סה"כ

$$b^T y \geq c^T x$$

ולכן, כל ערך של הפתרון הדואלי חוסם את ערך הפתרון בפרימאל.

■

1.10 תרגול 1

נבדיל בין זוג הגדרות:

הגדרה 17. פתרון אפשרי (Feasible) הוא השמה של ערכים למשתנים שמקיימת את כל האילוצים, כאן פתרון הוא כל וקטור x.
פתרון אופטימלי (Optimal) הוא פתרון אפשרי, שערך פונקציית המטרה שלו טוב לפחות כמו של כל פתרון אפשרי אחר.

לכל בעיית LP נכון אחד מהשלושה הבאים:
1. infeasible: האילוצים סותרים זה את זה, ואין נקודה שמקיימת את כולם. למשל $x \leq 1, x \geq 2$.
2. Unbounded: לא חסומה. כאן, יש נקודות אפשריות אך אפשר לשפר את המטרה ללא סוף. למשל $\max x$ באשר $x \geq 0$. לכל x ניתן לקחת את $x + 1$.

3. Finite optimum: אופטימום סופי. יש נקודות אפשריות, וגם המטרה חסומה. כאן ־ יש פתרון אופטימלי.

בעיית LP תקינה היא כזו מסוג מקסימום בה כל האילוצים מסוג $\leq$ וכן המשתנים אי שליליים. ואם לא? מה נעשה?
- $\max \to \min$ למזער את $c^T x$ שקול למקסם את $-c^T x$. לכן, נהפוך את הסימן.
- $\geq \longleftrightarrow \leq$: נכפול את שני האגפים ב$-1$ וכיוון אי השוויון יתהפך.
- $=$: אם יש שוויון, ניתן להפוכו לשני אי שיוונות. למשל: $a^T x = b$ שקול ל: $a^T x \leq b$ וגם $a^T x \geq b$. אותו נמיר ונקבל $-a^T x \leq -b$ ולכן סה"כ

$$a^T x = b \iff \begin{cases} a^T x \leq b \\ -a^T x \leq -b \end{cases}$$

- אם המשתנה לא בהכרח אי שלילי. אזי, כל x ניתן לכתוב כ$x = x_+ - x_-$ כך ש$x_+, x_- \geq 0$.
- אם נתון כי $x_j \leq 0$ אזי נגדיר $z_j = -x_j \geq 0$. נחליפו בכל מקום.

הפירוש הגאומטרי נתבונן בתמונה הבאה.



בציור ישנם 5 אילוצים על זוג משתנים. האזור האפור ־ הוא אוסף כל הנקודות המקיימות את כל האילוצים, זהו האזור האפשרי. הוא מחזיק במספר קודקודים. נגדיר:

הגדרה 18. חצי מרחב (Half-space): כל אי שוויון לינארי מחלק את המישור בקו ישר ומשאיר צד אחד. למשל, $2x_1 + x_2 \leq 10$ הוא כל מה שמתחת לישר $2x_1 + x_2 = 10$.
פאון קמור (Convex polyhedron): החיתוך של כל חצאי־המרחב. קמור פירושו שאם שתי נקודות בפנים, בהכרח כל הקטע ביניהן בפנים.
קווי רמה (Level sets): כל הנקודות בהן המטרה שווה לערך מסווים α: הקו $c^T x = \alpha$. עבור ערכי α שונים מקבלים קווים מקבילים, כי לכולם אותו השיפוע c.

מסקנה חשובה: אם קיים אופטימום ־ הוא בהכרח על קודקוד. נשים לב כי כל עוד נזיז את קווי הרמה, ונהיה בתוך הפאון הקמור אנחנו בפתרון חוקי. קו ישר שזז על פני מצולע קמור יוצא ממנו דרך קודקוד בהכרח או דרך צלע שלמה. גם במקרה של צלע, הקודקודים שלה אופטימליים. זהו הבסיס לאלגוריתם הסימפלקס: שקופץ מקודקוד לקודקוד.

- כל משתנה בסיסי־ אלו שבאילוצים, יחד עם פונקציית המטרה: מבוטאים רק באמצעות המשתנים הלא בסיסיים.

הגדרה 19. מבחן היחס (Ratio Test): לכל שורה בה המקדם של המשתנה הנכנס שלילי, מחשבים את גודל המקדם חלקי הקבוע. המינימום הוא כמה שניתן להגדיל. השורה שנותנת את המינימום, קובעת את המשתנה היוצא. נשים לב כי חייבים לקחת את המינימום בשביל לעמוד בכל האילוצים.

חשוב: במבחן היחס אנחנו מקבלים שורות. רק שורות שבהן לנכנס יש מקדם שלילי מגבילות אותו. מה אם אין אף שורה כזו? במקרה זה ניתן להגדיל את הנכנס כמה שרוצים ־ כל המשתנים הבסיסיים רק גדלים או נשארים ואף אחד לא הופך לשלילי. לכן הבעיה Unbounded. לדוגמה, אם אנחנו רוצים להכניס את המשתנה x_2. מציבים ומקבלים:

$$\begin{cases} x_4 = \frac{69}{4} + \frac{3}{16} x_2 \\ x_3 = \frac{3}{2} - \frac{3}{8} x_2 \\ x_1 = \frac{33}{4} - \frac{1}{16} x_2 \end{cases}$$

אזי הראשון כאן נשים לב שהמקדם בה של x_2 הוא חיובי ־ לכן אין שום מגבלה. הגדלת x_2 רק תגדיל את x_4. אם זה היה קורה בכל השורות, למשל:

$$\begin{cases} x_4 = \frac{69}{4} + \frac{3}{16}x_2 \\ x_3 = \frac{3}{2} + \frac{3}{8}x_2 \\ x_1 = \frac{33}{4} + \frac{1}{16}x_2 \end{cases}$$

אזי כאן כל המקדמים של x_2 ־ זה שאנחנו רוצים להכניס לבסיס, הן חיוביות עם מקדם חיובי. במקרה זה אנחנו עוצרים ומכריזים: הבעיה לא חסומה.

זמן הריצה של סימפלקס הוא מתבסס על כך שמס' ערכי המטרה חסום ב$\binom{n+m}{m}$. מדוע ניתן להניח זאת? אנחנו מניחים כי בסימפלקס לא חוזרים לאותו בסיס פעמיים ־ ולכן ניתן לחסום בכלל את זמן הריצה. בכל Pivot המטרה לא יורדת: היא עולה או נשארת במקום. אם יש רצף של Pivots מנוונים האלגו' עלול לחזור לבסיס שהיה בו ויווצר מצב של Cycling: הסתובבות במעגל לנצח. לכן משתמשים ב**כלל בלנד:** תמיד בוחרים את המשתנה עם האינדקס הקטן מבין המועמדים, אם למשל מבחן היחס תמיד נותן אפס. זה ימנע מצב של Cycling.

דוגמת הרצה אם יש b_i שלילי נניח כי אנחנו מתבוננים בבעיה הבאה:

$$\begin{cases} \max 2x \\ -x \leq -1 \\ x \leq 3 \end{cases}$$

ברור כי התשובה היא $x = 3$, כלומר ערך פונקציית המטרה הוא 6. מה הבעיה? $b_1 = -1$ ובפרט שלילי ולכן להתחיל עם $x = 0$ בבסיס לא אפשרי.
לכן, נחסר x_0 מכל אילוץ. נקבל:

$$\begin{cases} \max 2x \\ -x - x_0 \leq -1 \\ x - x_0 \leq 3 \end{cases}$$

ובשפה של משתני העודף נקבל:

$$\begin{cases} \max 2x \\ s_1 = -1 + x + x_0 \\ s_2 = 3 + x_0 - x \end{cases}$$

את הבעיה ממירים למקסום של $-x_0$. נקבל:

$$\begin{cases} \max -x_0 \\ s_1 = -1 + x + x_0 \\ s_2 = 3 + x_0 - x \end{cases}$$

ובכן, הפתרון הבסיסי שלו בו $x = x_0 = 0$ מניב $s_1 = -1$ וזה לא חוקי. מה עושים? נשים לב כי השורה עם הb_i המינימלי היא שורה 1, בה $b_\ell = -1$. לכן, בשלב הראשוני אנחנו מכניסים את x_0 לבסיס ומוציאים משם את s_1. נבודד ונקבל $x_0 = s_1 + 1 - x$. נקבל:

$$\begin{cases} \max x - 1 - s_1 \\ x_0 = s_1 + 1 - x \\ s_2 = 4 + s_1 - 2x \end{cases}$$

עכשיו הפתרון הבסיסי הוא כזה שבו $x = s_1 = 0$. נשים לב כי כל הקבועים אי שליליים ולכן הBFS חוקי. כעת, ניתן להריץ על מערכת זו סימפלקס.

לx מקדם חיובי במטרה שהוא 1 ולכן הוא נכנס ללא בסיסיים. ממי נקח? נציב $s_1 = -$ והאפשרויות הן $x_0 = 1 - x$ או $s_2 = 4 - 2x$. נקבל כי השורה הראשונה נותנת את החסם $x \leq 1$ והשנייה את $x \leq 2$. עדיף את החסם הקטן ־ לכן יוצא מהראשונה. נקבל $x = s_1 + 1 - x_0$. נקבל כי $x_0 = 0$ ולכן הבעיה המקורית אפשרית ו$x = 1$ היא נק' התחלה חוקית.

MFMC בידינו הבעיה הבאה בה התעסקנו רבות באלגוריתמים 1.
נתון גרף מכוון $G = (V, E)$ עם מקור s, בור t ולכל קשת e ישנה קיבולת u_e. מטרתנו למקסם את הזרימה מs לt.

ראשית, עלינו להחליט כיצד למדל את הבעיה. עלינו לבחור משתנים, מטרה ואילוצים.
- המשתנים שלנו: יהיו כמה זורם על כל קשת. לכן, f_e יהיה הזרימה על הקשת e.
- אילוצים: אנחנו יודעים מאלגוריתמים 1 (תודה צבי) כי חייבים להתקיים החוקים הבאים:
  1. קיבולת ־ $0 \leq f_e \leq u_e$
  2. שימור זרימה. לכל צומת $v \neq s, t$ מה שנכנס שווה למה שיוצא.
- המטרה: מקסום הזרימה.

בשלב הראשון, על מנת למדל את הבעיה כבעיית LP נוסיף לגרף קשת חדשה $e_0 = (t, s)$. היא תצא מt ותכנס לs. זה בכיוון ההפוך! לזרימה הרגילה. נגדיר את הזרימה על הקשת החדשה כr, כלומר e_0 היא קשת עם זרימה r. לקשת זו, אין קיבולת ־ r לא חסום. ונגדיר סה"כ

$$E' = E \cup \{e_0\}$$

מה הרעיון? נדמיין את הזרימה הולכת מs עד לרשת t ואז חוזרת למt לs דרך e_0. ואז ־ מתחיל סיבוב חדש. כל מה שהגיע מt, חוזר דרך לe_0 ולכן:

$$r = \text{The value of the flow}$$

זרימה כזו, בה הזרימה רק מסתובבת במעגלים ללא נק' התחלה או סוף נקראת **מחזור**. במחזור, כל צומת מקיים שימור כולל s וt. s מקבל r דרך e_0 ושולח את אותו r הלאה. וכך גם t. נסמן:
$\delta^-(v; E')$: קבוצת הקשתות בE' שנכנסות לצומת v.
$\delta^+(v; E')$: קבוצת הקשתות בE' שיוצאות מצומת v.

נתחיל בהגדרת האילוצים:

$$\sum_{e \in \delta^-(v;E')} f_e - \sum_{e \in \delta^+(v;E')} f_e = 0 \ \ (\forall v \in V)$$

כלומר ־ לכל $v \in V$, ההפרש בין כמה נכנס לv לכמה יוצא לv הוא אפס. לכן: זה בדיוק השימור ־ מה שיוצא שווה למה שנכנס.
אילוץ נוסף הוא פשוט: $0 \leq f_e \leq u_e$ לכל קשת $e \in E$. זה לא על E'! לe_0 אין קיבולת. וכן $f_{e_0} = r \geq 0$. הזרימה על e_0 נקראת r והיא אי שלילית. סה"כ נקבל את הבעיה הבאה:

$$\begin{gathered} \max r \\ \textstyle\sum_{e \in \delta^-(v;E')} f_e - \sum_{e \in \delta^+(v;E')} f_e = 0 \ \ (\forall v \in V) \\ \forall e \in E : 0 \leq f_e \leq u_e, f_{e_0} = r \geq 0 \end{gathered}$$

החלק החשוב: למקסם את r הוא למקסם את הזרימה. המפתח הוא כי r איננו חופשי, האילוצים קושרים אותו לערך הזרימה. אילוץ השימור בs אומר שמה שנכנס שווה למה שיוצא, מה שנכנס לs זה רק הקשת e_0 שהקיבולת עליה r. מה שיוצא, לפי אלגו' 1 (שוב, תודה צבי) זה ערך הזרימה. לכן

$$r = \text{Flow}$$

נשים לב כי באופן תאורטי יכולנו לוותר על r, אך הניסוח הישיר הוא יותר פר גרף ופר מקרה ולא שומר על אחידות ומקרה כללי. לכן בחרנו לנסח באופן הבא.

Minimum cost flow נגדיר את בעיית האופטימיזציה הבאה:

הגדרה 20. כעת נרצה להתבונן בשאלה הבאה. כעת, לכל קשת e יש קיבולת u_e ובנוסף גם מחיר ליחידה a_e. להעביר f_e יחידות בקשת e עולה $a_e f_e$. המטרה כעת היא להעביר כמות קבועה d מ s ל t ולשלם כמה שפחות. d הוא קבוע.
נסמן לכל $v \in V$:

$$D_v(f) = \sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e$$

את ההפרש בין כמה שנכנס ל v פחות כמה שיוצא ממנו. זהו העודף של הצומת. נשים לב כי אם $D_v(f) > 0$ אזי נכנס יותר ממה שיוצא, והצומת קולט חומר. אם $D_v(f) < 0$ אזי יוצא יותר ממה שנכנס והצומת מייצר חומר. אם $D_v(f) = 0$ אזי ישנו שימור. נשים לב כי $1_{v=t}$ היא פונקצייה מציינת ששווה ל1 אם $v = t$ ו0 אחרת. נרצה שיתקיים כי: $D_t(f) = d$, והרי $1_{v=t} - 1_{v=s} = 1 - 0 = 1$ במקרה זה. לכן, המשמעות היא כי t קולט נטו בדיוק d. $D_s(f) = -d$ והרי אכן $1_{v=t} - 1_{v=s} = 0 - 1 = -1$ ולכן s שולח בדיוק d. ולכל צומת אחר מתקיים $D_v(f) = 0$. מכאן נוכל להגדיר את בעיית האופטימיזציה:

$$\begin{gathered} \min \sum_{e \in E} a_e f_e \\ D_v(f) = d\left(1_{v=t} - 1_{v=s}\right) \quad (\forall v \in V) \\ 0 \le f_e \le u_e \end{gathered}$$

משפט 21. הLP הנ"ל בעל פתרון פיזיבלי אם ורק אם הזרימה המקסימלית ברשת היא לפחות d.
הוכחה:
בכיוון ראשון, אם יש פתרון פיזיבלי אזי הפתרון הוא זרימה חוקית עם ערך d. לכן, הזרימה המקסימלית היא לפחות d.
בכיוון השני, אם הזרימה המקסימלית היא לפחות $F \ge d$ אזי נקח את הזרימה המקסימלית ונכפיל כל ערך ב$\frac{d}{F}$, זהו מס' בין 0 ל1 ולכן השימור ישמר והקיבולות ישמר וערך הזרימה החדש יהיה $F \cdot \frac{d}{F} = d$.

כעת, נוכל להכליל את הרעיון. נניח כי לכל צומת ישנו מס' משלו d_v שאומר כמה הוא צריך לקבל נטו. מכאן, $D_v(f) = d_v$. כאן נוכל להגדיר את הבעיה כך:

$$\begin{gathered} \min \sum_{e \in E} a_e f_e \\ D_v(f) = d_v \\ 0 \le f_e \le u_e \\ \sum_{v \in V} D_v(f) = 0 \end{gathered}$$

מדוע הכרחי כי $\sum_v D_v(f) = 0$? כי לכל זרימה f סכום העודפים של כל הצמתים הוא אפס. נתבונן בקשת אחת $e = (u, w)$ עם זרימה f_e. כיצד היא מופיעה בסכום $\sum_v D_v(f)$? ב$D_w(f)$ עם סימן חיובי, כי נכנס אל w. וב$D_u(f)$ עם סימן שלילי כי יוצאת מu. והיא לא נמצאת בשום D_v אחר. לכן כל קשת תורמת לסכום 0 ולכן סה"כ בהכרח התנאי הנ"ל חייב להשמר.

זרימה מרובת סחורות עד כה זרם ברשת סוג אחד של חומר, כעת נניח כי ישנם k סחורות שונות אשר חולקות אותה רשת. לסחורה i יש מקור משלה s_i ויעד משלה t_i כך ש$s_i \neq t_i$. נגדיר את אוסף המשתנים $f_e^{(i)}$ להיות כמה מסחורה i זורמת על הקשת e. ישנו משתנה נפרד לכל זוג של סחורה $\times$ קשת.
מהי המטרה? α הוא אחוז אחיד מהביקוש - אשר מספקים לכל הסחורות יחד. אם $\alpha = \frac{1}{2}$ אזי כל סחורה i מעבירה $\frac{1}{2} \cdot d_i$ - חצי ממה שהיא רוצה. $\max \alpha$ מחפש את האחוז הגדול ביותר שניתן לתת לכל הסחורות בבת אחת. אף סחורה לא מקבלת אחוז גבוה יותר על חשבון סחורה אחרת. מכאן, נותר להגדיר אילוצים על מערכת זו.
1. שימור לכל סחורה בנפרד: נרצה לדרוש כי לכל $v \in V$ ולכל $i \in [k]$ יתקיים כי:

$$D_v\left(f^{(i)}\right) = \alpha d_i \left(1_{v=t_i} - 1_{v=s_i}\right)$$

כלומר - עבור סחורה i אנחנו דורשים כי תצא נטו αd_i מs_i ותכנס נטו αd_i אל t_i ותשמר בכל צומת אחר. ישנו אילוץ לכל זוג ולכן סה"כ $k \times |V|$ אילוצים.

2. קיבולת משותפת: עלינו לדרוש כי סך הזרם בכל הסחורות לא יעבור את קיבולתה של הקשת. לכן:

$$\forall e \in E : \sum_{i=1}^{k} f_e^{(i)} \leq u_e$$

3. עלינו לדרוש כי $f_e^{(i)} \geq 0$ וכן $\alpha \geq 0$. מכאן נקבל את המערכת הבאה:

$$\begin{gathered} \max \alpha \\ \forall v \in V, i \in [k] : D_v\left(f^{(i)}\right) = \alpha d_i \left(1_{v=t_i} - 1_{v=s_i}\right) \\ \forall e \in E : \textstyle\sum_{i=1}^{k} f_e^{(i)} \leq u_e \\ \forall e \in E, i \in [k] : f_e^{(i)} \geq 0 \\ \alpha \geq 0 \end{gathered}$$

אם $\alpha = 1$ ומציבים אותו במקום למקסם, עלינו לבדוק רק אם יש פתרון. התשובה היא כן: כלומר אפשר לספק את כולם אם ורק אם הα המקסימלי הוא לפחות 1.

זרימה מקסימלית עם הפסדים כעת הנחנו כי מה שנכנס לקשת יצא ממנה בשלמותו. כעת, כל קשת מאבדת חלק ממה שעובר בה. נגדיר η_e להיות החלק שמגיע לסוף הקשת כאשר הוא ערך בין 0 ל1. אם נשלח f_e יחידות בתחילת הקשת, מגיעים לסוף שלה $\eta_e f_e$ יחידות. כמובן אם $\eta_e = 1$ הכל מגיע ואין הפסד.
כעת יש להבדיל בין שני מספרים לכל קשת: כמה נשלח f_e לעומת כמה הגיע $\eta_e f_e$.
המטרה שלנו היא למקסם את מה שבאמת הגיע ליעד ־ $\max \sum_{e \in \delta^-(t)} \eta_e f_e$.
נדרוש כי בכל צומת ביניים v יתקיים שימור. כלומר:

$$\forall v \in V \setminus \{s,t\} : \sum_{e \in \delta^-(v)} \eta_e f_e = \sum_{e \in \delta^+(v)} f_e$$

ונדרוש את הקיבולת כמובן. סה"כ נקבל את בעיית האופטימיזציה:

$$\begin{gathered} \max \textstyle\sum_{e \in \delta^-(t)} \eta_e f_e \\ \forall v \in V \setminus \{s,t\} : \textstyle\sum_{e \in \delta^-(v)} \eta_e f_e = \sum_{e \in \delta^+(v)} f_e \\ \forall e \in E : 0 \leq f_e \leq u_e \\ \forall e : \eta_e \in [0,1] \end{gathered}$$

1.11 Strong Duality

בפשטות, ההבדל אומר שתחת המצב האופטימלי, הפתרון הדואלי יניב בדיוק את הפתרון הפרימאלי. כלומר, החסם העליון הופך לשוויון בדיוק. כלומר, בפתרון האופטימלי: ערך פונקציית המטרה של הפתרון הדואלי יהיה שווה לערך פונקציית המטרה בפתרון הפרימאלי. וכן:

- אם בעיה אחת היא ברת ביצוע וחסומה, כלומר: פיזיבלית וחסומה, אז גם הבעיה השניה בהכרח ברת ביצוע וחסומה וערכי האופטימום שלהם שווים.
- אם בעיה אחת היא ברת ביצוע, אך לא חסומה ־ כלומר הערך שלה שואף לאנסוף, אזי גם הבעיה השניה איננה פיזיבלית: אין לה שום פתרון חוקי.

ובכן, תמיד מתקיים $b^T y \geq c^T x$. על מנת להוכיח דואליות חזקה, יש להראות שקיימים x, y עבורם ישנו שוויון מלא.

משפט 22. משפט הדואליות החזקה.
הוכחה:
בתחילת תהליך אלגוריתם הסימפלקס, אנו מסתכלים על הבעיה הפרימאלית בצורתה הסטנדרטית ורוצים למקסם את

$$z = \sum_{i=1}^{n} c_i x_i$$

בנוסף, יש לנו אילוצים מסוג אי שוויון. כדי להפעיל את אלגוריתם הסימפלקס, אנו הופכים אותם למשוואות ע" הוספת m משתנים $x_{n+1},....,x_{n+m}$. עד לקבלת: $x_{n+j}=b_j-\sum_{i=1}^{n}a_{j,i}x_i$ לכל $j\in[m]$.
לאחר הרצת אלגוריתם הסימפלקס, שרץ על המטריצה וכו' משוואת המטרה מקבלת צורה חדשה שכוללת קבוע מסוים יחד עם שילוב המשתנים:

$$z'=v+\sum_{i=1}^{n+m}c'_ix_i$$

הערך הקבוע v, הוא בדיוק הערך האופטימלי של פונקציית המטרה. כעת, נסתכל על הביטוי החדש של המטרה z'. נוכל לפצל אותו לשני סכומים:

$$z'=v+\sum_{i=1}^{n}c'_ix_i+\sum_{j=1}^{m}c'_{n+j}x_{n+j}$$

ובכן, אנו יודעים כי $x_{n+j}=b_j-\sum_{i=1}^{n}a_{j,i}x_i$ ולכן נוכל להציבם:

$$z'=v+\sum_{i=1}^{n}c'_ix_i+\sum_{j=1}^{m}c'_{n+j}\cdot\left(b_j-\sum_{i=1}^{n}a_{j,i}x_i\right)$$

נרצה להשוות את z' לz. לשם כך, נפתח סוגריים, נסדר את האיברים ונקבץ את הקבועים ונקבל:

$$z'=v+\sum_{j=1}^{m}c'_{n+j}b_j+\sum_{i=1}^{n}\left(c'_i-\sum_{j=1}^{m}c'_{n+j}a_{j,i}\right)x_i$$

זו פונקציה המורכבת מאיבר חופשי קבוע יחד עם סכום מקדמים כפול המשתנים המקוריים. יש לשים לב: z,z' הן אותה הפונקציה על כל הצבה חוקית של המשתנים בבעיה. לכן,

$$\forall x_1,...,x_n:z(x_1,...,x_n)=z'(x_1,....,x_n)$$

נזכר כי $z=0+\sum_{i=1}^{n}c_ix_i$ וכיוון שz,z' הן שוות לכל וקטור, האיברים החופשיים שלהם חייבים להיות שווים גם כן. ובפרט, כל מקדם בהם חייב להיות שווה. לכן:

- הקבוע שווה ולכן:

$$0=v+\sum_{j=1}^{m}c'_{n+j}b_j\Longrightarrow v=-\sum_{j=1}^{m}c'_{n+j}b_j$$

- השוואת מקדמים: לכל $1\leq i\leq n$ נקבל כי:

$$c_i=c'_i-\sum_{j=1}^{m}c'_{n+j}a_{j,i}$$

נשים לב כי בבעיה הדואלית אנו מחפשים משתנים y_j שכופלים את b_j בפונקציית המטרה, אנו מצאנו קשר כנ"ל עבור $v=-\sum_{j=1}^{m}c'_{n+j}b_j$. לכן, נגדיר את משתני הדואל להיות המקדמים הללו. כלומר:

$$y_j=-c'_{n+j}$$

נראה כי:

$$b^Ty=\sum_{j=1}^{m}b_jy_j=\sum_{j=1}^{m}b_j\cdot(-c'_{n+j})=v$$

לכן, קיבלנו כי ערך פונקציית המטרה הדואלית שווה בדיוק לv, הפתרון האופטימלי הפרימאלי. לכן קיבלנו את השוויון שרצינו. אך עוד לא סיימנו, עלינו להוכיח כי וקטור הy שהוגדר הוא פתרון פיזיבלי לבעיה הדואלית. כדי שפתרון יהיה חוקי בבעיה הדואלית עליו לקיים:

1. $A^T y \geq c$
2. $\forall j : y_j \geq 0$

קודם ראינו כי $c_i = c'_i - \sum_{j=1}^{m} c'_{n+j} a_{j,i}$, אם נסדר נקבל $c_i - c'_i = \sum_{j=1}^{m} -c'_{n+j} a_{j,i}$ כלומר:

$$c_i - c'_i = \sum_{j=1}^{m} y_j a_{j,i}$$

כיוון שהגענו לטבלה האופטימלית בהרצת הסימפלקס, כל המקדמים בשורת המטרה מקיימים כי $c'_i \leq 0$ (שיפוע שלילי). לכן בהכרח $c_i \leq c_i - c'_i$ ומכאן נקבל: $c_i \leq \sum_{j=1}^{m} y_j a_{j,i}$ ולכן $A^T y \geq c$.

אותו חוק אופטימליות של הסימפלקס פועל על משתני ההשלמה, $0 \leq c'_{n+j}$. לכן, בהכרח $y_j = -c'_{n+j} \geq 0$ לכל j.

סה"כ מצאנו פתרון אפשרי, פיזיבלי, לבעיה הדואלית שערכו הפונקציונלי שווה בדיוק לפתרון האופטימלי של הבעיה הפרימאלית. לכן, Primal=Dual.

■

כמה שאלות שעולות:

- מדוע z, z' תמיד שווים לכל הצבה וניתן להשוות בניהם? כיוון שz' איננה פונקציה חדשה שהמצאנו, אלא התקבלה במהלך אלגוריתם הסימפלקס. במהלך האלגוריתם אנו מבצעים פעולות שורה אלמנטריות על מערכת המשוואות של האילוצים שלנו. כאשר אנו מציבים את משתני ההשלמה לתוך z', אנו מבצעים מניפולציה אלגברית. מבחינה מתמטית, מדובר באותה פונקציה, כתובה באופן שונה.
- לפי הדואליות החלשה, הפתרון הדואלי מהווה חסם עליון לפרימאלי. לפי הדואליות החזקה, באופטימום, ורק בו, יתקיים שוויון בין ערכי הפרימאל והדואלי.
- למה אנו צריכים את הבעיה הדואלית? בעולם האמיתי, לעיתים בעיה פרימאלית כוללת מליוני אילוצים אך רק מעט משתנים. במעבר לבעיה הדואלית, האילוצים הופכים למשתנים והמשתנים לאילוצים. במצבים כאלו, להריץ תוכנה שתפתור את הדואלית יעיל יותר מלפתור את הבעיה המקורית.
- אם הבעיה פיזיבלית וחסומה, אלגוריתם הסימפלקס שרץ על הבעיה הדואלית יניב את הפתרון האופטימלי.

1.12 אלגוריתמי קירוב ותכנות לינארי

נחזור לדון על VC, זו בעיה NPC שאינה ניתנת לפתרון פולינומי. גם הגרסה שלה ע"י תכנות בשלמים לא ניתנת לפתרון, כיוון ש$IP \in NPH$ גם כן. אם נשנה את הגרסה, כך שלא נדרוש שהמשתנים הבינאריים יהיו אפס או אחד בלבד, נקבל:

$$\min \sum_{v \in V} x_v \ \forall (u,v) \in E : x_v + u_v \geq 1 \wedge \forall v : 0 \leq x_v \leq 1$$

האילוץ הופך את הבעיה לניתנת לפתרון ע"י תכנות לינארי. כעת, הבעיה ניתנת לפתרון יעיל באמצעות LP רגיל. כיוון שLP מאפשר יותר אפשרויות, הערך המינימלי שתמיד ימצא תמיד יהיה קטן או שווה לערך המינימלי של IP ולכן יתקיים $LP \leq IP$. זה יאפשר לנו לחשב חסם תחתון עבור VC.

נניח משולש: u, v, w. הרצת LP עליו יניב $x_v = x_u = x_w = \frac{1}{2}$. כלומר, $LP = 1.5$. אך בפועל, אי אפשר לקחת חצי צומת - לכן כדי לכסות משולש יש לבחור לפחות 2 צמתים שלמים. לכן הפתרון האופטימלי הוא $IP = OPT = 2$. כלומר, אנחנו רוצים למצוא פתרון שאיננו בהכרח אופטימלי, אך הוא טוב.

1.12.1 אלגוריתם קירוב לVC עם תכנות לינארי

האלגוריתם פשוט למדי: נריץ LP על הבעיה שאינה בשלמים, ונגדיר את הקבוצה הבאה:

$$S = \{v | x_v \geq \frac{1}{2}\}$$

משפט 23. מדובר בכיסוי חוקי, עם יחס קירוב $S \leq 2LP \leq 2OPT$

הוכחה: ראשית, מדובר בכיסוי חוקי: שכן, לכל קשת מוכרח כי מתקיים $x_u + x_v \geq 1$.

נניח בשלילה כי קיימת קשת $e=(u,v)$ שלא מכוסה ע"י הכיסוי. אזי, $x_u, x_v < \frac{1}{2}$ ולכן $x_u + x_v < 1$ בסתירה לאילוץ ולנכונות LP. לכן, בהכרח אחד מהצמתים מכוסה ע"י הכיסוי.

כעת, נוכיח את יחס הקירוב. גודל הקבוצה $|S|$ שלקחנו הוא מס' הצמתים שלקחנו. לקחנו רק צמתים שהערך שלהם הוא לפחות חצי, ולכן אם נעגל אותם לאחד, אנו לכל היותר מכפילים את התרומה שלהם לפונקציית המטרה פי 2. לכן, גודל הקבוצה S יהיה לכל היותר פעמיים הגודל של ערך הLP. שכן, מתמטית:

$$|S| = |\{x_v | x_v \geq \frac{1}{2}\}| \leq 2LP \leq 2IP = 2OPT$$

■

הערה חשובה. העולם של דואליות מגיע בשביל למצוא חסם לפתרון. נניח ואנחנו בבעית מקסימום, אזי המטרה שלנו היא למצוא חסם עליון לבעיה שלנו. כלומר ערך a שפונקציית המטרה שלנו לא יכולה לעבור. לכן אנחנו יודעים שבמקרה זה ערך פונקציית המטרה של הבעיה הדואלית חוסם את ערך פונקציית המטרה של הבעיה הפרימאלית כלומר $\text{Primal} \leq \text{Dual}$.

לעומת זאת, בבעיות מינימיזציה המטרה שלנו ליצור בעיה דואלית שחוסמת לנו מלמטה - כלומר ערך a' אשר תמיד מתקיים כי a גדול ממנו. כלומר במקרה זה של בעיות מינימיזציה אזי $\text{Dual} \leq \text{Primal}$.

Set Cover 1.12.2

הגדרה 24. בהינתן עולם U ואוסף תתי קבוצות $S_1, ..., S_\ell \subseteq U$, בעיית Set Cover מוגדרת להיות מציאת המס' המינימלי של קבוצות k מתוך האוסף כך שאיחודן יכסה את היקום. כלומר,

$$\bigcup_{i \in I} S_i = U$$

כך ש$I \subseteq \{1, ..., \ell\}$ וגם $|I| = k$.

ע"י הגדרה לפי ניסוח בתכנות לינארי, לכל קבוצה S_i נגדיר משתנה בינארי $x_i = 1$ אמ"מ S_i נלקחה לכיסוי.
נוכל להגדיר את הבעיה:

$$\min \sum_{i=1}^{\ell} x_i \text{ s.t } \forall a \in U : \sum_{i:a \in S_i} x_i \geq 1$$

כלומר, לכל איבר בעולם $a \in U$ סכום משתני ההחלטה x_i עבור הקבוצות המכילות את a חייב להיות גדול או שווה ל1. על מנת לפתור את הבעיה בתכנות לינארי, במקום $x_i \in \{0,1\}$ משנים את האילוץ ל$x_i \geq 0$.

משפט 25. קיים אלגוריתם f-קירוב לSet Cover.

כעת, לאחר שפתרנו את הLP קיבלנו ערכים שבריים. אנו צריכים להחליט אילו קבוצות באמת לקחת לכיסוי. יהי f פרמטר שמוגדר להיות המס' המקסימלי של קבוצות שאיבר כלשהו בU שייך אליהן בו זמנית. כלומר: אם האיבר הכי נפוץ נמצא ב3 קבוצות אזי $f = 3$. פורמלית:

$$f = \max_{a \in U} |\{i | a \in S_i\}|$$

נגדיר: $I = \{i | x_i \geq \frac{1}{f}\}$. נטען כי קבוצה זו בהכרח מכסה את כל האיברים. יהי איבר כלשהו $a \in U$, סכום הערכים של הקבוצות המכילות אותו הוא לפחות 1 (שכן הוא שייך לפחות לקבוצה אחת). הרי, ישנן לכל היותר f קבוצות כנ"ל, ולכן לפי עקרון שובך היונים קיימת לפחות קבוצה אחת מתוכן אשר קיבלה ערך x_i גדול-שווה מ$\frac{1}{f}$. לכן, בהכרח הקבוצה הנ"ל תבחר לאלגוריתם שלנו: ולכן a יכוסה.

כיוון שבחרנו רק קבוצות המקיימות $x_i \geq \frac{1}{f}$, אם נעגל את הערך שלהן ל1 הרי שנכפיל את הכמות הכללית לכל היותר פי f. לכן גודל הכיסוי שלנו יהיה לכל היותר $f \cdot LP$)כלומר, לכל היותר הכפלה פי f מהכמות שמקבלים בLP: הרי קיבלנו גודל ואז הניתוח מתייחס לאחר ניפוח פי f). מכאן:

$$|\text{Set-Cover}| = f \cdot LP \leq f \cdot IP \leq f \cdot OPT$$

כנדרש.

■

26. קיים אלגוריתם הסתברותי שפותר את Set-Cover בהסתברות גבוהה.

הוכחה: כעת, במקום להשתמש בx_i שהניב האלגוריתם כערך חותך, נשתמש בו כהסתברות. האלגוריתם יוגדר באופן הבא:

נטיל מטבע מוטה לכל קבוצה S_i. הקבוצה תבחר לכיסוי בהסתברות שערכה x_i (הערך שקיבלנו מהLP). נראה כי לפי לינאריות התוחלת, תוחלת גודל הקבוצה הינה:

$$\mathbb{E}[|I|] = \sum_{i=1}^{\ell} \mathbb{E}[x_i] = \sum_{i=1}^{\ell} x_i = LP$$

כלומר, אכן בממוצע נכסה את כולם אך אין לנו ודאות שאכן כל איבר יכוסה. לכן, נרצה לחשב את ההסתברות לכישלון, ולהוכיח שהיא קטנה: זה מספיק טוב עבור אלגוריתם הסתברותי. ובכן, איבר לא נבחר אמ"מ אף אחת מהקבוצות לא בחרה אותו. המאורע לבחירה של קבוצה ב"ת לקבוצות שונות ולכן נקבל כי

$$\Pr[\text{ i Failed}] = \prod_{i:a\in S_i} (1-x_i) \leq \prod_{i:a\in S_i} e^{-x_i} = e^{-\sum_{i:a\in S_i} x_i} \leq_* e^{-1}$$

שכן $*$ נכון כי $\sum_{i:a\in S_i} x_i \geq 1$. כעת, נרצה להגביר את ההסתברות. נגדיל באופן מלאכותי את ההסתברות לבחור כל קבוצה. נבחר את הקבוצה S_i בהסתברות המוגדלת $p_i = x_i \cdot (ln(n)+1)$. באשר, $n = |U|$. המחיר בכך, הוא שתוחלת כמות הקבוצות שבחרנו גדלה להיות $\mathbb{E}[|I|] = (\ln(n)+1)\cdot LP$. כעת, ההסתברות עבור איבר בודד לכישלון הופכת להיות:

$$e^{-(\ln(n)+1)} = \frac{1}{n\cdot e}$$

מכאן:

$$\Pr[\text{failed}] = \bigcup_{i=1}^{n} \Pr[\text{i failed}] \leq \sum_{i=1}^{n} \Pr[\text{i failed}] = n\cdot\frac{1}{ne} = \frac{1}{e}$$

כלומר, הסתברות הכישלון הכוללת של האלגוריתם היא e^{-1}, והאלגוריתם מניב כיסוי בגודל לכל היותר $\ln(n)$ כפול הפתרון האופטימלי.

■

1.12.3 הגרסה הדואלית של Set Cover

כעת, נרצה לעבור לגרסה הדואלית ולשם כך לכל אילוץ בבעיה הפרימאלית (כלומר לכל איבר $a \in U$) נגדיר משתנה דואלי חדש y_a. פונקציית המטרה משתנה ממינימיזציה למקסימיזציה, ומטרתה למקסם את סכום המשתנים הדואלים. וכן, לכל קבוצה $1 \leq i \leq \ell$ סכום המשתנים הדואליים עבור כל האיברים a ששייכים לקבוצה, חייב להיות קטן או שווה לאחד. סה"כ נקבל:

$$\max\left(\sum_{a=1}^{n} y_a\right) \text{ s.t } \forall i \in \{1,...,\ell\} : \sum_{a\in S_i} y_a \leq 1 \ , \forall 1 \leq a \leq n : y_a \geq 0$$

כלומר, האילוץ מוגדר עבור כל קבוצה בנפרד ־ סכום המשתנים של האיברים בקבוצה לא יכול לעבור את אחד.

משפט 27. תהי $D \subseteq U$ תת קבוצה המקיימת את התנאי $|D \cap S_i| \leq t$ לכל קבוצה S_i באוסף. אזי, גודל הכיסוי האופטימלי יקיים $Opt \geq \frac{|D|}{t}$.

הוכחה: למעשה, נראה כי קיימת הצבה חוקית למשתני הבעיה הדואלית המניבה ערך פונקציית מטרה של $\frac{|D|}{t}$. נגדיר את משתני הבעיה הדואלית y_a לכל איבר $a \in U$ באופן הבא: עבור כל איבר שלא שייך לD, נציב $y_a = 0$. לכל איבר $a \in D$ נציב $y_a = \frac{1}{t}$.

ראשית, נוכיח כי הצבה הנ"ל מהווה פתרון חוקי לבעיה הדואלית. על מנת שהפתרון יהיה חוקי, עליו לקיים את האילוץ $\sum_{a \in S_i} y_a \leq 1$ לכל קבוצה S_i. נראה כי:

$$\sum_{a \in S_i} y_a = \sum_{a \in S_i \setminus D} y_a + \sum_{a \in S_i \cap D} y_a = \sum_{a \in s_i \setminus D} 0 + \sum_{a \in S_i \cap D} \frac{1}{t} \leq 0 + t \cdot \frac{1}{t} = 1$$

לכן, ההצבה הנ"ל מהווה פתרון חוקי לבעיה. כעת, נחשב את ערך פונקציית המטרה עבור הצבה זו. נזכר כי הגדרנו ערך ששונה מאפס רק עבור איברי הקבוצה D לכן נקבל:

$$\sum_{a=1}^{n} y_a = \sum_{a \in D} y_a = \sum_{a \in D} \frac{1}{t} = \frac{|D|}{t}$$

לפי משפט הדואליות החלשה, ערך פונקציית המטרה עבור כל פתרון דואלי חוקי חסום מלמעלה ע"י ערך פונקציית המטרה של כל פתרון פרימלי חוקי. הרי כי Opt הינו העלות הפרימלית האופטימלית, והצגנו פתרון דואלי חוקי. לכן יתקיים: $Opt \geq \frac{|D|}{t}$. כנדרש.

הערה 28. מה אומר המשפט העיל למעשה? מצאנו חסם תחתון לפתרון האופטימלי של Set Cover, מבלי לפתור את הבעיה הקשה בעצמה. כיוון שקשה למצוא את הפתרון האופטימלי, ניתן להעזר בחסם לבנות קבוצה D מתאימה, לחשב את הערך $\frac{|D|}{t}$ ולקבל חסם תחתון לפתרון.

הערה 29. מדוע הבעיה הדואלית הוגדרה כך? בבעיה הפרימאלית, יש לנו משתנה x_i לכל קבוצה S_i ואנו מנסים לצמצם את סכומם. בבעיה הדואלית, המשתנים שלנו הם y_a והם מוגדרים לכל איבר ביקום ואנו מנסים למקסם את סכומם. ישנה סימטריה מלאה: איברים הופכים לאילוצים, קבוצות הופכות למשתנים.

בפרימאל, יש אילוץ לכל $a \in U$ שדורש כי הקבוצות שמכילות אותו יכסו אותו לפחות באחד: $\sum x_i \geq 1$. האילוץ הזה בפרימאל מוליד בדואל משתנה y_a.

בדואל, יש אילוץ לכל קבוצה S_i - אשר דורש כי סכום המשקולות בו לא יעלה על 1. אילוץ זה בודק את הקשר ההדדי בין קבוצה לאיברים שמרכיבים אותה.

הפרימאל, מנסה למצוא את המינימום - לשלם כמה שפחות עבור כיסוי. הדואל - שואף למצוא את המקסימום: להרוויח כמה שיותר ע"י תמחור נכון של האיברים - משפט הדואליות החלשה מבטיח כי כל פתרון חוקי של הדואלי לעולם לא יעקוף את הפרימאל.

Greedy Set Cover 1.12.4

נציג אלגוריתם חמדן שפותר את Set-Cover ביחס קירוב לוגריתמי בגודל העולם.

בתחילה, אנו מגדירים קבוצה D שמכסה את כל היקום בעת אתחול האלגוריתם. כל עוד יש איברים שלא כוסו - אנחנו בוחרים באופן חמדני את הקבוצה S_i שמכילה את המס' הגדול ביותר של איברים שעוד לא כוסו, מוסיפים לI (אוסף האינדקסים של הקבוצות שנבחרו) את אינדקס הקבוצה, ומעדכנים את D להיות הקבוצה פחות האיברים שכעת כבר כוסו. כך מתקדמים, כל עוד $D \neq \emptyset$.

Algorithm: Greedy Set Cover

Require: Universe U with $|U| = n$, collection of sets $S_1, S_2, \ldots, S_\ell$ such that $\bigcup_i S_i = U$
Ensure: A collection of indices I such that $\bigcup_{i \in I} S_i = U$

1. $D \leftarrow U$ (the elements not yet covered)
2. $I \leftarrow \emptyset$ (the chosen set indices)
3. While $D \neq \emptyset$:
   (a) $i \leftarrow \arg\max_i |D \cap S_i|$ (the set covering the most remaining elements)
   (b) $I \leftarrow I \cup \{i\}$
   (c) $t \leftarrow |D \cap S_i|$ (how many new elements this set covers)
   (d) $D \leftarrow D \setminus S_i$
4. **return** I

למען ניתוח האלגוריתם, נעזר במשפט שהוכח קודם. לכל תת קבוצה $D' \subseteq U$ בה כל קבוצה חותכת את D בלא יותר מt איברים, מתקיים $Opt \geq \frac{|D'|}{t}$. כעת, נראה כיצד האלגוריתם החמדן מנצל את המשפט הנ"ל איטרציה אחר איטרציה. נניח כי האלגוריתם בחר בסך הכל k קבוצות באיטרציות שונות. נסמן את הקבוצות שנבחרו $S_{i_1}, ..., S_{i_k}$ ואת גודל החיתוך שלהן עם האיברים שטרם כוסו באותם רגעים ב$t_1, ..., t_k$.
בתחילת הדרך, מתקיים $D = U$. הקבוצה הראשונה שנבחרה, S_{i_1} מכסה מס' איברים שנגדירו כ$t_1 = |S_{i_1}|$. כלומר האיברים שנבחרו הם $a_1^1, ..., a_{t_1}^1$. נפעיל את משפט החסם העליון על שלב זה, והרי כי מתקיים:

$$Opt \geq \frac{n}{t_1} \Longrightarrow \frac{Opt}{n} \geq \frac{1}{t_1}$$

נשים לב כי:

$$\frac{1}{t_1} + \frac{1}{t_1} + ... + \frac{1}{t_1} = 1 \leq \frac{Opt}{n} + ... + \frac{Opt}{n}$$

באשר מספר האיברים בסכימה משמאל הוא t_1. כלומר, $\frac{Opt}{n} \geq \frac{1}{t_1}$ עבור האיברים הראשונים שנוספו.
כעת, לאחר שבחרנו את הקבוצה הראשונה נותרו ביקום $n - t_1$ איברים. הקבוצה השנייה שנבחרה מקיימת $|D \cap S_{i_2}| = t_2$. האלגוריתם החמדן בוחר בכל רגע את הקבוצה שמכסה כמה שיותר מהאיברים הנותרים, לכן בהכרח גודל החיתוך של כל קבוצה בהמשך לא יכול לעבור את t_2. מכאן נקבל יחס ... $t_1 \geq t_2 \geq t_3 \geq$. מכאן על הקבוצה בגודל $n - t_1$ נקבל:

$$Opt \geq \frac{n - t_1}{t_2} \Longrightarrow \frac{Opt}{n - t_1} \geq \frac{1}{t_2}$$

באופן כללי, עבור האיטרציה הj מס' האיברים שנותרו לכיסוי הינו $|U| - \sum_{i=1}^{j-1} t_i$. לכן החסם התחתון יקיים

$$Opt \geq \frac{n - \sum_{i=1}^{j-1} t_i}{t_j}$$

נסכום את כל אי השוויונות שקיבלנו לאורך האיטרציות,הרי בכל שלב המכנה הוא מהצורה $n - \sum_{i=1}^{j-1} t_i$. מכנה זה הוא מס' האיברים שנותרו לכיסוי. סה"כ לאורך האיטרציות עוברים על כל הערכים במקרה הגרוע ביותר (בו ההתקדמות היא הכי איטית - בכל שלב אנחנו מוסיפים רק איבר אחד).
סך הקבוצות שנבחרו הוא בדיוק k, כל איבר מקבל חסם שמורכב משברים מהצורה $\frac{1}{t_j}$. מכאן נקבל:

$$k = \sum_{i=1}^{k} \sum_{a \in S_i \cap D} \frac{1}{t_i} \leq \sum_{i=1}^{k} \sum_{a \in S_i \cap D} \frac{\mathrm{Opt}}{n - \sum_{j=1}^{i-1} t_i} =\leq \mathrm{Opt} \cdot \left(\frac{1}{n} + \frac{1}{n-1} + ... + \frac{1}{3} + \frac{1}{2} + \frac{1}{1} \right) = \mathrm{Opt} \cdot \sum_{j=1}^{n} \frac{1}{j} = \ln(n) \cdot \mathrm{Opt}$$

לכן מס' הקבוצות שהאלגוריתם יניב הוא לכל היותר פקטור לוגריתמי במס' האיברים כפול הפתרון האופטימלי.

1.13 Max Flow Min Cut

הגדרה 30. רשת זרימה היא גרף מכוון בו קיים קודקוד מקור s וקודקוד יעד t, כך שקיימת פונקציית קיבולת לכל קשת בגרף. יהי x_{r_1,r_2} משתנה המייצג את כמות הזרימה שעוברת בקשת (r_1,r_2), ויהי $w(r_1,r_2)$ משתנה שמייצג את הקיבולת המקסימלית המותרת על הקשת (r_1,r_2).

נגדיר את בעיית זרימת המקסימום בצורה של תכנות לינארי:

$$\max \sum_{w\in\Gamma(s)} x_{s,w} \text{ s.t } \begin{cases} \forall w\in V\backslash\{s,t\} : \sum_{r\in\Gamma(w)} x_{w,r} - \sum_{r\in\Gamma(w)} x_{r,w} = 0 \\ \forall (r_1,r_2)\in E : x_{r_1,r_2} \le w_{r_1,r_2} \\ \forall (r_1,r_2) : x_{r_1,r_2} \ge 0 \end{cases}$$

באשר האילוצים הם חוקי הזרימה: שימור הזרימה, אילוצי הקיבולת ואי השליליות על הזרימה.

1.14 תרגול 2

בתרגול זה נשאל שאלה חדשה מהשאלה ששאלנו בתרגול הקודם. קודם, שאלנו מה זו בעיית LP וכיצד מנסחים כזו. כיום נשאל, **כיצד יודעים שפתרון הוא אופטימלי?** נניח ומישהו יביא לנו פתרון עם ערך 10. כיצד נדע שאין פתרון עם ערך 11? התשובה לכך היא דואליות. לכל LP יש Primial - עליו דיברנו בתרגול הקודם, ואחיו התואם: Dual. הפתרונות שלו נותנים חסמים על הLP המקורי.

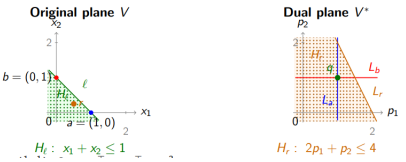
הגדרה 31. המרחב הדואלי V^* של $\mathbb{R}^2$ מורכב מפונקציונליים לינאריים. **פונקציונל לינארי** היא פונקציה שמקבלת וקטור $x\in\mathbb{R}^2$ ומחזירה מספר. בצורה לינארית, מדובר בסכום משוקלל של הקורדינטות. כל פונקציה כזו נקבעת לגמרי ע"י שני משקלים p_1,p_2. לכן ניתן לייצג פונקציונל ע"י הנקודה $p=(p_1,p_2)\in V^*$. וסה"כ המרחב הדואלי הוא:

$$\varphi_p(x) = p^T x = p_1x_1 + p_2x_2$$

לצורך דוגמה, הנקודה $p=(2,-1)$ מייצגת את הפונקציונל $\varphi_p(x)=2x_1-x_2$.
כמו כן, הביטוי p^Tx סימטרי. מה הכוונה? ניתן לקבע את p ולהזיז את x. למשל: נקבע $x=(3,4)$. נשאל - אילו נקודות דואליות מניבות את הערך 1? כלומר:

$$L_{(3,4)} : 3p_1+4p_2=1$$

זו משוואה לינארית בשני נעלמים לכן זהו **ישר במישור הדואלי.**
נתבונן בתמונה הבאה:



הישר ℓ מתאים ל$q=(1,1)$. מה אנחנו רואים בציור?

- במישור המקורי V: אנחנו רואים זוג נקודות $a=(1,0), b=(0,1)$ והישר ℓ המחבר אותן $x_1+x_2=1$. זהו הישר $q^Tx=1$ עם $q=(1,1)$. חצי המישור $H_\ell : x_1+x_2\le 1$ הוא צבוע בירוק.
- במישור הדואלי V^*: כל נקודה הפכה כבר לישר. $a=(1,0)$ הופכת לישר $L_a : p_1=1$ ו$b=(0,1)$ הפכה לישר $L_b : p_2=1$ והישר ℓ הפך לנקודה $q=(1,1)$.

כלומר: ℓ עובר דרך a וb ובמישור הדואלי q נמצאת גם על L_a וגם על L_b - בדיוק בנק' החיתוך שלהם. **ישר שעובר דרך שתי נקודות** הפך ל**נקודה שנמצאת על שני ישרים.** כי - נקודה הפכה לישר, וישר הפך לנקודה.

נגדיר נקודה נוספת $r=\left(\frac{1}{2},\frac{1}{4}\right)$. אזי $H_r : \frac{1}{2}p_1+\frac{1}{4}p_2\le 1$, לאחר כפל ב4 נקבל $H_r : 2p_1+p_2\le 4$.

נשים לב כי $q^T r = \frac{3}{4} \leq 1$ ולכן

$$r \in H_\ell \iff q \in H_r$$

מדוע? שני הצדדים הם אותו אי השוויון בדיוק, כי $r \in H_\ell$ משמעות הדבר כי $q^T r \leq 1$, כלומר $q \in H_r$.

מטריצה. תהי A מטריצה ממשית, ונניח כי כל שורה של A מחשבת פונקציונל לינארי על x. השורה הi נותנת את המספר $(Ax)_i$. נקודה $p \in \mathbb{R}^m$ בוחרת משקל לכל שורה. נראה כי:

$$p^T (Ax) = \left(A^T p\right)^T x$$

ובמילים אחרות - וקטור המקדמים המשולב, הופך להיות $A^T p$.
מה משוואה זו אומרת? מצד שמאל, אנחנו מחשבים כל שורה על x ואז לוקחים סכום משוקלל של התוצאות. מצד ימין - קודם מחברים את השורות עצמן עם המשקלים, מקבלים בעצם וקטור מקדמים חדש ואז מחשבים אותה על x. המתמטיקה אומרת שזה שקול.

דוגמה 32. נתבונן ב$A = \begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix}$. אזי נגדיר $p = (2,1)$. שקלול השורות מניב:

$$2 \cdot (1,2) + 1 \cdot (3,1) = (5,5) = A^T p$$

בדואליות של LP, משקלים אלו יהפכו למשתנים הדואליים.

הגדרה 33. ננית כי $p^T x = 1$. נבהיר כי $x = (x_1, x_2)$ היא נקודה במישור המקורי. במישור הדואלי $p = (p_1, p_2)$ זו נקודה. לכל זוג: נק' מהמישור המקורי ונק' מהמישור הדואלי ניתן לחשב את $p^T x$. נאמר כי הם מתאימים וננית כי $p^T x = 1$. זה כלל שנבחר על מנת להתאים זוג אובייקטים בשני המישורים. כך למשל אם $r = (3,4)$ נקודה במקורי אזי $3p_1 + 4p_2 = 1$ בדואלי זהו ישר. בדיוק כמו כלל 1. ובאופן כללי יותר:
אזי:

1. במישור הראשוני V, נקודה או וקטור $r \neq 0 \iff$ במישור הדואלי V^* זהו ישר $L_r = r^T p = 1$.
2. במישור הראשוני V, ישר $\ell : q^T x = 1 \iff$ במישור הדואלי V^* זה נקודה או וקטור q.
3. במישור הראשוני, $H_\ell : q^T x \leq 1$ חצי מישור $\iff$ במישור הדואלי V^* זו משפחה של ישרים דואלים: כל $r \neq 0$ בH_ℓ נותן L_r עם צד נבחר H_r שמכיל את q. כלומר,

$$r \in H_\ell \iff q \in H_r$$

דואליות של LP. תהי A מטריצת המשקולות וb וקטור החסמים. אזי $Ax \leq b$. נבחר מכפילים אי שליליים $y \geq 0$, אחד לכל אילות. אזי: נשים לב כי

$$y^T Ax \leq y^T b$$

שכן $Ax \leq b$. הכפל בערך $y_i \geq 0$ לא הופל את הכיוון.
כמו כן, אם $A^T y \geq c$ אזי:

$$c^T x \leq \left(A^T y\right)^T x = y^T Ax \leq b^T y$$

ובעברית: כל $y \geq 0$ עם $A^T y \geq c$ מניב חסם עליון $b^T y$ על ערך כל פתרון אפשרי x.
יתכנו הרבה y כנ"ל, וכל אחד נותן חסם. על מנת לחסום מלמעלה, תחת ההנחה כי הPrimial מסוג max, אזי נרצה את החסם הכי הדוק ולכן:

$$\min\ b^T y \text{ s.t } A^T y \geq c, y \geq 0$$

הוא הLP הדואלי.

כיצד בונים דואלי?
- לכל אילוץ ראשוני בפרימאל, מגדירים **משתנה דואלי אחד**. זהו המכפיל y_i של האילוץ i.
- לכל משתנה ראשוני בפרימאל, מגדירים **אילוץ דואלי אחד**. זוהי הדרישה $\left(A^T y\right)_j \geq c_j$, על המקדם של x_j.
- b, c מחליפים תפקידים וA משוחלפת.

זו טבלת ההחלפה:

	ראשוני	דואלי
כיוון	max	min
מטרה	$c^T x$	$b^T y$
אילוצים	$Ax \leq b$	$A^T y \geq c$
סימן משתנים	$x \geq 0$	$y \geq 0$
גודל	n משתנים, m אילוצים	m משתנים, n אילוצים

משפט הדואליות החלשה טוען כי הפתרון הדואלי חוסם את הפרימאלי.

משפט 34. אם ישנו פתרון x של הפרימאלי ופתרון y של הדואלי ושניהם אפשריים וכן ערך המטרה שלהם שווה, אזי שניהם אופטימליים.
הוכחה: אם ערך המטרה שלהם שווה אזי $c^T x = b^T y$. מכאן:
לכל פרימאלי אחר x' יתקיים:

$$c^T x' \leq b^T y = c^T x$$

באשר אי השוויון נכון מדואליות חלשה. ולכן, x הוא האופטימלי.
וכן לכל דואלי אחר y' יתקיים

$$b^T y' \geq c^T x = b^T y$$

ולכן y הוא המינימלי, ואופטימלי.

משפט הדואליות החזקה לעומת זאת אומר, כי אם הפרימאל אפשרי ובעל אופטימום סופי אזי גם לדואלי יש אופטימום סופי והערכים האופטימליים שלהם שווים.

דוגמה 35. נניח הבעיה $\begin{cases} \max\ 3x_1 + 2x_2 \\ x_1 + x_2 \leq 4 \\ x_1 \leq 2 \\ x_2 \leq 3 \\ x_1, x_2 \geq 0 \end{cases}$. אזי, ניצור לכל אילוץ משתנה $y_1, y_2, y_3 \geq 0$. כעת, חוסמים את הערך y_1 כפול אילוץ ראשון - 4, וכו ומקבלים $\min\ 4y_1 + 2y_2 + 3y_3$. מכאן, האילוצים: x_1 מופיע באילוץ הראשון והשני עם מקדם 1 ולכן $y_1 + y_2 \geq 3$ - גדולים מהערך שלו בפונקציית המטרה המקורית. וכן x_2 מופיע באילוץ הראשון והשלישי ולכן $y_1 + y_3 \geq 2$. סה"כ נקבל $\begin{cases} \min\ 4y_1 + 2y_2 + 3y_3 \\ y_1 + y_2 \geq 3 \\ y_1 + y_3 \geq 2 \\ y_1, y_2, y_3 \geq 0 \end{cases}$.

משפט 36. מתקיים:
1. אם הפרימאל אפשרי בעל אופטימום סופי, הדואל אפשרי עם **אותו** אופטימום סופי.
2. אם הפרימאל אפשרי ולא חסום, אזי הדואל שמנסה לחסום - לא אפשרי.
3. אם הפרימאל לא אפשרי, אזי הדואל לא חסום **או** לא אפשרי.

כעת נתאר כללי המרה לכל צורה של LP.
1. אם הפרימיאל $a_i^T x_i \leq b_i$, אזי המשתנה הדואלי שמתאים לאילוץ יהיה $y_i \geq 0$.
2. אם הפרימיאל עם האלוץ $a_i^T x_i = b_i$, אזי y_i לא מוגבל.
3. אם $a_i^T x_i \geqq b_i$ אזי $y_i \leq 0$
4. אם המשתנה הראשוני $x_i \geq 0$ אזי האילוץ התואם $\left(A^T y\right)_i \geq c_i$.
5. אם x_i איננו מוגבל, נכריח $\left(A^T y\right)_i = c_i$
6. אם $x_i \leq 0$ אזי האילוץ התואם $\left(A^T y\right)_i \leq c_i$.

מדוע זה ככה? אנחנו רוצים כי יתקיים $y_i \cdot \left(a_i^T x_i\right) \leq y_i b_i$. אזי, ־ כדי שהחיבור יתן חסם עליון
1. אם $a_i^T x_i \leq b_i$ אזי צריך להכפיל במס' $y_i \geq 0$.
2. אם $a_i^T x_i \geq b_i$ יש להכפיל במס' שלילי.
3. אחרת זה שוויון, ואין מגבלה על y_i.

ומדוע תנאים $4-6$? כי רוצים $c_i x_i \leq \left(A^T y\right)_i x_i$. לכן:
4. אם $x_i \geq 0$ מספיק לדרוש כי $\left(A^T y\right)_i \geq c_i$
5. אם x_i לא מוגבל, נדרוש שוויון.
6. אחרת $x_i \leq 0$ לכן נדרוש במקרה זה כי $\left(A^T y\right)_i \leq c_i$ ואז יתהפך אי השוויון לאחר מכפלה בx_i משני הצדדים.

משפט 37. הלמה של Farkas: יהיו $A \in \mathbb{R}^{m \times n}$ ו$b \in \mathbb{R}^m$. בדיוק אחת מהאפשרויות הבאות מתקיימת:
1. קיים $x \in \mathbb{R}^n$ כך ש$Ax = b$ וכן $x \geq 0$.
2. קיים $y \in \mathbb{R}^m$ עם $A^T y \geq 0$ וכן $b^T y < 0$.

מה זה אומר? אפשרות ראשונה היא שיש פתרון למערכת. האפשרות השניה היא **תעודה שאין פתרון.** מה הכוונה? נניח שיש גם x כזה. אזי:

$$0 \leq \left(A^T y\right)^T x = y^T A x = y^T b$$

אבל $y^T b < 0$ בסתירה. לכן y כנ"ל מוכיח שאין x אפשרי.

חתך מינימלי כLP. יהי חתך S עם $s \in S$ ו$t \notin S$. נסמן p_v משתנה אינדיקטור באופן הבא: $p_v = \begin{cases} 1 & v \in S \\ 0 & v \notin S \end{cases}$. כמו כן נסמן y_{uv} לכל קשת $e = (u,v) \in E$ כמשתנה אינדיקטור שמוגדר: $y_{uv} = \begin{cases} 1 & (u,v) \in E, u \in S, v \in T \\ 0 & o.w \end{cases}$ כלומר להיות 1 אם הקשת היא קשת חתך.
נרצה להגדיר LP של בעיית חתך מינימלי. המטרה היא למזער את סכום הקיבולות של הקשתות שיוצאות מS. קשתות שנכנסות לS לא נספרות. המטרה היא $\sum_{(u,v) \in E} u_{uv} y_{uv}$. באשר u_{uv} היא הקיבולת של הקשת (u,v).
הקשת צריכה להספר רק כאשר $p_u = 1$ וגם $p_v = 0$. כך $u \in S, v \notin S$. כיצד נעשה זאת? נאכוף זה ע"י

$$y_{uv} \geq p_u - p_v$$

נסביר. $p_u - p_v = 1$ בדיוק כאשר $u \in S, v \notin S$. המקרה השני שבו $u, s \in S$ יביא $p_u - p_v = 0$ אבל אז $y_{uv} = 0$ ולכן זה בסדר.
כמו כן, נדרוש כי $p_s - p_t \geq 1$. נשים לב כי אם הם אכן בצדדים שונים אזי $p_s - p_t = 1$. בכל מקרה אחר הם קטנים מ1. וכן לא נגביל את p_v. זו רלקסציה. סה"כ נקבל את המערכת

$$\begin{gathered} \min \sum_{(u,v) \in E} u_{uv} y_{uv} \\ y_{uv} \geq p_u - p_v \; \forall (u,v) \in E \\ p_s - p_t \geq 1 \\ y_{uv} \geq 0, \forall (u,v) \in E \end{gathered}$$

נשים לב כי כל חתך הוא פתרון אפשרי. לכל חתך S אם נציב

$$p_v = \mathbf{1}_{v \in S}, y_{uv} = \mathbf{1}_{u \in S, v \notin S}$$

כי האילוצים מתקיימים, וערך המטרה הוא בדיוק קיבולת החתך.
לכן, האופטימום של LP הוא חסום בקיבולת החתך המינימלי. הLP יכול לבחור כל חתך ויתכן גם בגלל העיגול שיבחר משהו טוב יותר.

נזכר בפתרון הLP לבעיית הזרימה במחזור מהתרגול הקודם.

$$\begin{gathered}\max r = f_{ts} \\ f_e \leq u_e \ \forall e \in E \\ \textstyle\sum_{\text{in}(v)} f - \sum_{\text{out}(v)} f = 0 \ \forall v \in V \\ f \geq 0\end{gathered}$$

אזי נקבע משתנה דואלי p_v לכל שוויון שימור כלומר $\sum_{\text{in}(v)} f - \sum_{\text{out}(v)} f = 0$. כיוון שהוא שווה לאפס, אזי p_v לא מוגבל.
כעת נקבע גם משתנה $y_e \geq 0$ לכל אילוץ מהצורה $f_e \leq u_e$. נשים לב כי האילוץ מחייב $y_e \geq 0$.
כעת לאילוצים הדואליים.
המשתנה הראשון בפרימיאלי הוא $f_{uv} \geq 0$, נשים לב כי הוא נמצא בשלוש שורות. בראשונה בקיבולת של e עם מקדם 1, כמו כן בשימור של v עם מקדם 1 וכן בשימור של u עם מקדם מינוס 1. לכן סה"כ $y_{uv} + p_v - p_u \geq 0$. מכאן נקבל $y_{uv} \geq p_u - p_v$. כמו כן לf_{ts} אין חסם קיבולת, הוא נכנס לs ויוצא מt. והמקדם שלו בפונקציית המטרה הוא 1. הוא נכנס לs ולכן מופיע בקידומת 1, יוצא מt ולכן קידומת -1. הוא צריך להיות גדול מערכו בפונקציית המטרה ולכן $p_s - p_t \geq 1$.
קיבלנו **בדיוק** את המערכת של חתך מינימלי.
מסקנה: הדואלי של LP הזרימה הוא בדיוק הLP של החתך המינימלי. האם זה במקרה?

משפט 38. Max Flow = Min Cut
הוכחה: ראינו את ההוכחה באלגוריתמים 1, וכעת נראה הוכחה נוספת.
למעשה, זה בדיוק דואליות חזקה. לפי דואלית חזרה, הזרימה המקסימלית OPT של LP שווה לOPT של הדואלי, שהוא Min Cut.

עיגול ורלקסציה. תכנות לינארי בשלמים, IP דורשת כי חלק מהמשתנים או כולם יהיו שלמים. השמטת דרישות אלו נותנת רלקסציית LP.

- עבור בעיית מקסימום, $\text{OPT}_{\text{LP}} \geq \text{OPT}_{\text{IP}}$, כיוון שכל פתרון שלם הוא גם פתרון של הLP, שמקיים דרישה נוספת. לכן יתכנו יותר מרחב תמרון עבור הLP.
- עבור בעיית מינימום, $\text{OPT}_{\text{LP}} \leq \text{OPT}_{\text{IP}}$. שוב, מסיבות דומות ־ הוא יכול למזער יותר.

קבוצה בלתי תלויה מקסימלית. יהי גרף $G = (V, E)$. נרצה קבוצה ב"ת מקסימלית. אם כן נציג את המערכת הבאה

$$\begin{gathered}\max \textstyle\sum_{v \in V} x_v \\ x_u + x_v \leq 1 \ \forall (u, v) \in E \\ x_v \in \{0, 1\} \ \ \forall v \in V\end{gathered}$$

מדוע זה ככה? כי בקבוצה בלתי תלויה אין שכנים, לכן לכל זוג שכנים ־ כל קשת, רק אחד מהם לכל היותר יכול להכנס.

2 Succinct Data Structures: מבני נתונים תמציתיים

באופן כללי, כאשר נדבר על מבני נתונים תמציתיים המטרה שלנו היא במקום לבזבז במבנה נתונים זכרון רגיל ־ לייצג את הנתונים במינימום מקום האפשרי לכך.

Rank Problem 2.1

הבעיה מוגדרת באופן הבא:

הגדרה 39. יהי מערך $A=(a_1,...,a_n)$ כך שלכל $i\in[n]$ מתקיים $a_i\in\{0,1\}$. בהינתן אינדקס i, עלינו להחזיר כמה אחדות ישנם במערך מתחילתו ועד לאינדקס i. כלומר:

$$\mathrm{Rank}\left(i\right)=\sum_{j=0}^{i}A\left[j\right]$$

על מנת לפתור את הבעיה, נוכל להציג כמה פתרונות. חלקם, נאיביים.

2.1.1 אלגוריתם ראשון

נוכל לבצע את הלולאה לסכימה בעלות לכל היותר $\mathcal{O}\left(n\right)$, ללא זיכרון נוסף וללא עיבוד מקדים.

2.1.2 אלגוריתם שני

לחשב את $\mathrm{Rank}(i)=\mathrm{Rank}\left(i-1\right)+A\left[i\right]$. נבצע זאת כך באמצעות עיבוד מקדים ע"י תכנות דינמי בעלות $\mathcal{O}\left(n\right)$ וכעת זמן השאילתא יהיה $\mathcal{O}\left(1\right)$. אך, נשים לב כי סיבוכיות הזיכרון במקרה זה היא $\mathcal{O}\left(nlogn\right)$. מדוע? עלינו לשמור מערך Rank. לכל i מתקיים $\mathrm{Rank}\left(i\right)\leq n$ ולכן צריך $\log_2\left(n\right)$ ביטים לייצוג. לכן מס' הביטים לייצוג הוא לכל היותר

$$n\log_2\left(n\right)$$

2.1.3 אלגוריתם שלישי

במקום לשמור תשובה לכל אינדקס ואינדקס, שזה יקר מאוד, או לא לשמור כלום כמו באפשרות הנאיבית - נבצע חלוקה לבלוקים. נחלק את המערך הגדול לבלוקים באורך t, ונשמור תשובה מראש רק עבור תחילת כל בלוק. ישנם סה"כ $\frac{n}{t}$ בלוקים ולכן נפח הזיכרון שנצטרך לשמור הוא $\frac{n}{t}\log_2\left(n\right)$ ביטים.

כיצד מתבצעת שאילתא בפתרון שכזה?

- קפיצה לבלוק הרלוונטי בזמן $\mathcal{O}\left(1\right)$
- בתוך הבלוק עצמו, ספירה ידנית של האחדות בזמן $\mathcal{O}\left(t\right)$.

נשים לב כי אם נקח את $t=c\log_2\left(n\right)$ באשר $c\geq1$ קבוע, נקבל כי זמן שאילתא הוא $\mathcal{O}\left(\log\left(n\right)\right)$ ואילו סיבוכיות הזיכרון הינה $\frac{n}{c\log_2(n)}\cdot\log_2\left(n\right)=\frac{n}{c}=\mathcal{O}\left(n\right)$. כלומר, הגענו לסיבוכיות זיכרון לינארית.

2.1.4 אלגוריתם רביעי

בפתרון הקודם, היינו צריכים לשלם בזמן השאילתא $\mathcal{O}\left(\log\left(n\right)\right)$ כיוון שהיינו צריכים לספור ידנית בתוך בלוק בגודל t. הרעיון כעת

- נפעיל את אותו הטריק פעם נוספת בתוך כל בלוק.

כיצד המבנה כעת יהיה בנוי?

- בלוקים גדולים, אשר מחלקים את המערך לבלוקים בגודל t. לכל בלוק שכזה שומרים את מס' האחדות המצטבר עד לתחילתו, ישנם $\frac{n}{t}$ בלוקים כאלו ולכן סיבוכיות הזיכרון בשלב זה הינה $\frac{n}{t}\log_2\left(n\right)$ ביטים.
- תת בלוקים: בכל בלוק גדול בגודל t, מחלקים אותו לחתיכות קטנות יותר. נשמור את מס' האחדות יחסית לתחילת הבלוק הגדול - על מנת שהערכים יהיו קטנים ולא ידרשו הרבה ביטים. הערך המקסימלי שנצטרך לשמור הוא t, לכן כל ערך כזה דורש לכל היותר $\log_2\left(t\right)$ ביטים.

סה"כ יש n איברים מחולקים לתתי בלוקים, ולכן תוספת הזיכרון לחלק זה היא $n\log_2\left(t\right)$.

כעת:

- זמן טרום העיבוד: $\mathcal{O}\left(n\right)$ עבור מעבר על המערך ובניית הטבלאות של חישובי הבלוקיסג.
- שאילתא: $\mathcal{O}\left(1\right)$. מדוע?

על מנת לחשב $\mathrm{Rank}\left(i\right)$ מבצעים:

1. לוקחים את הערך של הבלוק הגדול בו נמצא האינדקס i, בעלות $\mathcal{O}\left(1\right)$.

2. מוסיפים לו את הערך של תת הבלוק התואם לi בבלוק הגדול, בעלות $\mathcal{O}(1)$.
- סה"כ סיבוכיות הזיכרון הינה:

$$\frac{n}{t}\log_2(n)+n\log_2(t)$$

אם נקח $t=c\log_2(n)$ עבור $c\geq 1$ קבוע, נקבל כי סיבוכיות הזיכרון הינה:

$$\frac{n}{c}+n\log_2(c\log_2(n))=\mathcal{O}(n\log(\log(n)))$$

כלומר, קיבלנו מבנה נתונים הנותן שאילתות בזמן $\mathcal{O}(1)$ עם עיבוד מקדים בעלות לינארית, ותוספת הזיכרון קטנה מאוד ביחס למינימום התאורטי.

2.1.5 אלגוריתם חמישי

אם האלגוריתם הקודם היה כל כך טוב, למה לעצור בשתי קריאות? נוכל להמשיך ולחלק את הבלוקים שוב ושוב, בתוך עצמם. נבצע רמות של בלוקים באופן רקורסיבי.

נקח את הבלוק, אותו נחלק לתתי בלוקים, וכל אחד מתתי הבלוקים מחולק שוב לתתי בלוקים. כך עד שמגיעים לגודל מינימלי.

כעת, זמן טרום העיבוד נותר $\mathcal{O}(n)$. זמן השאילתא יהיה כעת $\mathcal{O}(\log^*(n))$, כמס' הפעמים שיש להפעיל לוגריתם עד שמגיעים לאחד: שכן עלינו לחבר את כל התוצאות הללו מהמבנים השונים. תוספת הזיכרון תרד בצורה משמעותית - כעת סיבוכיות הזיכרון תהיה $\mathcal{O}(n\log^*(n))$ בלבד.

2.1.6 אלגוריתם שישי

אם כן, במקום לבצע רקורסיה או לחלק לתתי בלוקים עמוקים. נסתכל על גודל בלוק קטן מספיק, נקרא לו t. כמה אפשרויות שונות יכולות להיות לבלוק בינארי בגודל t? כיוון שהבלוק מורכב מאפסים ואחדות, יש סה"כ 2^t אפשרויות. אם t קטן מספיק, 2^t הוא קטן מספיק על מנת שנוכל להכין טבלה מראש לכל האפשרויות של בלוקים בגודל t. כיצד זה עובד בפועל?

- הבלוקים הגדולים: עדיין נשמור מערך חיצוני שיתן את מס' האחדות עד תחילת כל בלוק בעלות זיכרון $\frac{n}{t}\log_2(n)$.
- הטבלה האוניברסלית: לכל מחרוזת בינארית אפשרית באורך t, נחשב מראש טבלה שתניב את הRank הפנימי לכל מיקום בתוך הבלוק. כל תשובה בתוך הטבלה דורשת $\log_2(t+1)$ ביטים לכל היותר. לכן גודל הטבלה הכולל הוא $2^t\cdot\log_2(t+1)$. עבור t הבלוקים נקבל שחלק זה צורך זיכרון של $2^t\cdot t\cdot\log_2(t+1)$.

סיבוכיות המבנה היא:

$$2^t\cdot t\cdot\log_2(t+1)+\frac{n}{t}\log_2(n)$$

עבור $t=\frac{1}{2}\log_2(n)$ נקבל:

$$2^{\frac{1}{2}\log_2(n)}\cdot\frac{1}{2}\log_2(n)\log_2\left(\frac{1}{2}\log_2(n)+1\right)+\frac{2n}{\log_2(n)}\log_2(n)=2n+o(n)$$

כלומר, סיבוכיות הזיכרון היא לינארית וסיבוכיות השאילתא במקרה זה היא בדיוק $\mathcal{O}(1)$.

2.2 Select Problem

הבעיה מוגדרת באופן הבא:

הגדרה 40. נתון מערך בינארי $A\in\{0,1\}^n$. באשר הוא מעובד מראש. בהינתן $i\in[n]$ עליך למצוא x כך ש:

$$\sum_{j=1}^{x}A[j]=i$$

נשים לב כי נרצה את הx הקטן ביותר שמקיים זאת. הסכום נשאר הרי i גם אם יש אפסים לאחר מכן.

2.2.1 פתרון ראשון

סריקה: נבצע סריקה של המערך ובאופן נאיבי נפתור את הבעיה. זמן השאילתא $\mathcal{O}(n)$.

2.2.2 פתרון שני

בעיבוד מקדים עוברים פעם אחת על המערך בזמן $\mathcal{O}(n)$ ושומרים מערך P כך ש$P[i]$ הוא המיקום הראשון של x הנ"ל שיש לחפש. זמן הסריקה הוא לינארי כמובן.

לאחר מכן, השאילתא היא קריאה אחת מהמערך - זמן $\mathcal{O}(1)$

המחיר הוא במקום - יתכנו עד n אחדים ולכן כל מספר ידרוש $\mathcal{O}(\log_2(n))$ ביטים - לכן סה"כ $\mathcal{O}(n\log(n))$ ביטים. כעת, נרצה להוריד את המקום אבל גם לשמור על זמן שאילתא מהיר יחסית.

2.2.3 פתרון שלישי

כעת, במקום לשמור את המיקום של כל אחד שומרים רק את המיקום של האחד שהסכום עד אליו $t, 2t, 3t$ וכו'.

כעת יש לכל היותר $\frac{n}{t}$ דגימות: כל אחת בעלות $\mathcal{O}(\log_2(n))$ ביטים ולכן סה"כ המיקום הוא $\mathcal{O}\left(\frac{n}{t}\cdot\log_2(n)\right)$.

שאילתא: על מנת למצוא את הi, נחשב $k=\frac{i}{t}$ בערך שלם תחתון. נקפוץ לדגימה הk, ונמשיך לספור משם עוד $i \mod t$ אחדות. נשים לב כי יתכן שהסכימה תשאר רק בתחום בין הדגימה הk לדגימה ה$k+1$. זהו קטע שאורכו d_i. לכן, זמן השאילתא $\mathcal{O}(d_i)$. מה הבעיה? אין לנו חסם טוב על d_i.

לכן, נשתמש בשיטת **קל כבד.**

נבחר סף c ונטפל בכל קטע לפי האורך שלו:

- קטע קצר ("קל") הוא קטע אשר $d_i < c$. לא עושים במקרה זה כלום - סורקים בתוכו בעלות $\mathcal{O}(c)$.
- קטע ארוך ("כבד") הוא קטע אשר $d_i \geq c$. בקטע כזה אנחנו נשמור במפורש את כל t המיקומים של האחדות בתוכו. השאילתא, היא בעלות $\mathcal{O}(1)$ כי אין כאן עבודה - שולפים מהעבודה המוקדמת.

נשים לב שיש לכל היותר $\frac{n}{c}$ קטעים מאורך c ומעלה. לכל אחד שומרים t מיקומים של $\log_2(n)$ ביטים. לכן החסם כאן לזיכרון הוא $\frac{n}{c}\cdot t\log_2(n)$.

כעת, נגדיר $t=\log_2(n)$ וכן $c=\log_2^2(n)$. אזי סיבוכיות המקום הינה

$$\mathcal{O}\left(\frac{n}{t}\cdot\log_2(n)\right)+\mathcal{O}\left(\frac{n}{c}\cdot t\log_2(n)\right)=\mathcal{O}(n+n)=\mathcal{O}(n)$$

וכן - עלות שאילתא היא סה"כ $\mathcal{O}\left(\log_2^2(n)\right)$. העיבוד המקדים לינארי.

2.2.4 פתרון רביעי

מה נעשה כרגע? הקטעים הארוכים - מטופלים בדיוק כמו קודם. ההבדל הוא בקטעים הקצרים. במקום לסרוק אותם, מה שיעלה $\mathcal{O}\left(\log^2(n)\right)$ נשים לב כי כל קטע קצר הוא בעצמו מערך ביטים קטן. באורך פחות מ$c=\log_2^2(n)$. לכן אפשר לבנות את אותו מבנה עליו בדיוק.

התחכום שיגרום לזה לחסוך מקום - בתוך קטע קצר באורך m מספיק לשמור מיקומים יחסיים לתחילת הקטע. כל מיקום כזה דורש $\log(m)$ ביטים במקום $\log(n)$. מכאן יבוא השיפור.

$\log^*(n)$ הוא מס' הפעמים שניתן להפעיל $\log$ על n עד שנגיע ל1.

אם כן, נסתכל על תת בעיה באורך m ונבחר בה $t=\log(m)$ ו$c=\log^2(m)$. סה"כ נקבל לפי אותו חישוב $2m$ ביטים למיקום. תתי הבעיות באותה רמה זרות, לכן כל רמה עולה $\mathcal{O}(n)$ ביטים לכל היותר.

מספר הרמות: נשים לב שהגדלים יורדים - $n \to \log^2(n) \to \log^4(n) \to \ldots$ ולכן יש סה"כ $\mathcal{O}(\log^*(n))$ רמות. לכן סה"כ סיבוכיות המקום $\mathcal{O}(n\log^*(n))$ ביטים.

העיבוד המקדים - כל רמה היא מעבר לינארי על קטעיה, לכן סה"כ $\mathcal{O}(n\log^*(n))$

שאילתא - בכל רמה $\mathcal{O}(1)$ עבודה לשליפת הערך, ואז התאמה בחישוב פשוט. יש $\mathcal{O}(\log^*(n))$ רמות ולכן זמן השאילתא $\mathcal{O}(\log^*(n))$.

2.3 Range Minimum Query (RMQ)

נגדיר את הבעיה הבאה:

הגדרה 41. נתון מערך בינארי $A \in \mathbb{R}^n$. בהינתן $i,j \in [n]$ יש למצוא את:

$$x = \arg\min_{i \le k \le j} A[k]$$

המערך מועבד מראש. כל שאילתא מקבלת שני אינדקסים $i \le j$ ומבקשת את המיקום של הערך הקטן ביותר בטווח $A[i...j]$.

2.3.1 פתרון ראשון

בשאילתא פשוט נסרוק את האיברים בתחום, נשמור את המיקום של הקטן ביותר עד כה. אם הגיע קטן יותר - נחליף. סיבוכיות השאילתא היא

$$\mathcal{O}(j-i) = \mathcal{O}(n)$$

אין סיבוכיות מקום.

2.3.2 פתרון שני

ניצור טבלה של כל הזוגות. נשמור את התשובה לכל זוג (i,j). יש כ$\mathcal{O}\left(n^2\right)$ זוגות וכל תשובה היא אינדקס של $\mathcal{O}(\log_2(n))$ ביטים. לכן סה"כ סיבוכיות המקום היא $\mathcal{O}\left(n^2\log_2(n)\right)$. השאילתא - קריאה מהטבלה בעלות $\mathcal{O}(1)$. את הטבלה נמלא בתכנות דינמי פשוט:

$$M[i,j] = \begin{cases} i & i = j \\ j & A[j] \le A[M[i,j-1]] \\ M[i,j-1] & o.w \end{cases}$$

2.3.3 פתרון שלישי

כעת, לכל מיקום i במערך נשמור את המיקום של המינימום בקטעים שמתחילים בi ואורכם חזקה של 2: $1,2,4,8,...$ ימינה מi, ואותו דבר שמאלה מi. ישנם $\log_2(n)$ קטעים כאלו לכל היותר. כלומר אם נסביר ברור יותר - אנחנו שומרים את מיקום המינימום רק עבור אורכים בחזקה של 2. לכל מיקום i במערך שומרים:

- את המינימום בקטע שמתחיל בi והולך ימינה באורך $2,4,8,16,..$
- את המינימום בקטע שמתחיל בi והולך שמאלה באורך $2,4,8,16..$

כמה קטעים כאלו יש? יש $\log_2(n)$ קטעים לכל כיוון. לכן מהו הזיכרון סה"כ?
עושים זאת לכל מיקום i, ויש n כאלו לכן:

$$n \cdot 2\log_2(n) \cdot \log_2(n) = \mathcal{O}\left(n\log_2^2(n)\right)$$

נשים לב כי הלוג האחרון כאן הוא משמירה של ערך שיתכן ומגיע עד לn. מיוצג ע"י $\log_2(n)$ ביטים.

מבצעים עיבוד מקדים: קטע באורך 2^k שמתחיל בi הוא בדיוק שני קטעים צמודים באורך 2^{k-1}: אחד מתחיל בi ואחד ב$i+2^{k-1}$. את שניכם חישבנו קודם, לכן :

$$M[i][k] = \min\left\{M[i][k-1], M\left[i+2^{k-1}, k-1\right]\right\}$$

הסבר: כל תא מחושב ב$\mathcal{O}(1)$ ויש $\mathcal{O}(n\log n)$ תאים לכן זמן עיבוד המקדים הוא $\mathcal{O}(n\log n)$.

מה עם זמן שאילתא? מקבלים i,j. אורך הקטע $L = j-i+1$.
1. מוצאים את החזקה הגדולה ביותר של 2 שלא עולה על L: $2^k \le L \le 2^{k+1}$.
2. לוקחים את הקטע באורך 2^k שמתחיל בi והולך ימינה.

3. לוקחים את הקטע באורך 2^k שמתחיל בj והולך שמאלה.
4. כל קטע כזה הוא באורך לפחות $\frac{L}{2}$ ולכן שניהם יחד מכסים את כל $[i,j]$. הם חופפים באמצע - וזה לא בעייתי.
5. מחזירים את הקטן מבין התשובות השמורות.

למה חפיפה לא מפריעה? כי מינימום לא נפגע מספירה כפולה.

דוגמה: אם $j=12, i=3$ אזי $L=10$ ולכן $k=3$ כי $2^3=8$.
הקטע מ3 באורך 8 ימינה הוא $[3,....,10]$. הקטע מ12 באורך 8 שמאלה הוא $[5,....,12]$
יחד הם מכסים את:

$$[3,...,10] \cup [5,...,12] = [3,....,12] = A\,[i,...,j]$$

סה"כ זמן השאילתא הוא $\mathcal{O}(1)$. נעיר כי דורש חישוב $K=\log(L)$ אבל נניח שזה נעשה בפקודת מעבד פשוטה.

2.3.4 פתרון רביעי

הרעיון כזה - נחלק את המערך לבלוקים באורך t. מבנה החזקות של פתרון השלישי נשמר עבור כל בלוק. כלומר במיקומים $t, 2t, 3t, ..$ נקרא לנקודות אלו נקודות שמורות. נשים לב כי יש לנו כעת $\frac{n}{t}$ מיקומים. לכן גודל הזיכרון שנתפס:

$$\frac{n}{t} \cdot 2\log(n) \cdot \log(n) = \frac{2n}{t}\log^2(n)$$

שאילתה בסיסית: i וj לא יושבים לרוב על נקודות שמורות. לכן מפצלים את הקטע לשלושה קטעים:
1. מi עד לנקודה השמורה הבאה. פחות מt איברים.
2. מהנקודה השמורה הראשונה ועד האחרונה באותו קטע.
3. מהנקודה האחרונה באותו קטע עד לj. פחות מt איברים.

לכן סה"כ זמן השאילתא $\mathcal{O}(t)$ - שמירה בקצוות.
עיבוד מקדים: מחשבים את המינימום של כל בלוק בזמן לינארי, סה"כ $\mathcal{O}(n)$. ואז בונים את טבלת החזקות על $\frac{n}{t}$ הנקודות ב$\mathcal{O}\left(\frac{n}{t}\log(n)\right)$.
נבחר $t=\sqrt{n}$ ואז שאילתא $\mathcal{O}(\sqrt{n})$, הזיכרון $2\sqrt{n}\log^2(n)$ ועיבוד מקדים בזמן $\mathcal{O}(n)$.

3 Approximation Algorithms- אלגוריתמי קירוב

3.1 חישוביות וסיבוכיות

הגדרה 42. תהי L שפה. $L \in P$ אם ורק אם קיימת מכונת טיורינג דטרמיניסטית M כך שמתקיים:
- לכל $x \in L$: $M(x)=1$.
- לכל $x \notin L$: $M(x)=0$.

הגדרה 43. תהי L שפה. $L \in NP$ אם ורק אם קיימת מכונת טיורינג לא דטרמיניסטית M כך שמתקיים:
- אם $x \in L$, קיימת ריצה כך ש$M(x)=1$.
- אם $x \notin L$, לכל ריצה $M(x)=0$.

משפט 44. הבעיה Integer programming מוגדרת להיות: בהינתן מטריצה A, האם קיים וקטור $x \in \mathbb{N}^n_{\geq 0}$ כך ש$Ax \geq b$.
אזי, $IP \in NP$.

הגדרה 45. L שפה נקראת NP־קשה, אם לכל שפה $L' \in NP$ קיימת רדוקציית קארפ בזמן פולינומיאלי לL.

הגדרה 46. בעיית אופטימיזציה מוגדרת באמצעות רביעייה: $O = (I, Sol, m, type)$ באשר:
I היא קבוצת המופעים.
$Sol(i)$ - קבוצת הפתרונות האפשריים עבור המופע הi.
$m(i,s)$ היא פונקציית המדידה של הפתרון, עבור $i \in I, s \in Sol(i)$.
$type \in \{\min, \max\}$.
הערך האופטימלי מוגדר להיות:

$$opt(i) = \text{Type}_{s \in Sol(i)} m(i,s)$$

דוגמה 47. לדוגמה, נניח ונרצה למצוא את המסלול הקצר ביותר מs לt בגרף G. אזי זו בעיה מטיפוס מינימום וכן:

$$I = \{((G = (V,E), s, t) | s, t \in V\}$$

$$Sol((G = (V,E), s, t) = \{P | \text{P is an s-t path in G}\}$$

$$m((G = (V,E), s, t), P) = |P|$$

הגדרה 48. Decision version (גרסת ההכרעה) של בעיית אופטימיזציה: בהינתן פרמטר $k \in \mathbb{R}$
- אם $type = \max$ צריך להכריע האם קיים פתרון אפשרי $s \in Sol(i)$ כך שמתקיים $m(i,s) \geq k$
- אם $type = \min$ צריך להכריע האם קיים פתרון אפשרי $s \in Sol(i)$ כך שמתקיים $m(i,s) \leq k$

הגדרה 49. NPO היא מחלקת בעיות האופטימיזציה שגרסאות ההכרעה שלהן שייכות למחלקה NP. כלומר, בהינתן בעיית אופטימיזציה $O = (I, Sol, m, type)$ אזי בהינתן מופע i ופתרון מועמד s אנו יכולים:
- לקבוע האם $s \in Sol(i)$.
- לחשב ולהעריך את $m(i,s)$ - מדד ועלות הפתרון.

דוגמה 50. לדוגמה, בעיית הסוכן הנוסע (TSP) שמוגדרת להיות מציאת המסלול הקצר ביותר שמבקר בכל הקודקודים, היא בעיית אופטימיזציה שגרסת ההכרעה שלה הרי שייכת לNP, ולכן היא בNPO.

הגדרה 51. שפה או בעיית אופטימיזציה $O \in NPO$ נמצאת במחלקה PO אם קיים אלגוריתם שמוצא בזמן פולינומיאלי את הערך האופטימלי:

$$\text{opt}(i) = type_{s \in Sol(i)} m(i,s)$$

הגדרה 52. תהי $O \in NPO$ בעיית אופטימיזציה. אזי, הבעיה היא NP־שלמה אם גרסת גרסת ההכרעה המתאימה שלה היא NP־שלמה.

3.2 אלגוריתם קירוב של VC

נתבונן באלגוריתם פשוט וטריוואלי שמצליח לקרב בצורה די טובה את VC, למעשה: עוברים על כל הקשתות לפי סדר כלשהו. אם הקשת מכוסה כבר ע"י אחד הקודקודים - מתקדמים לאיטרציה הבאה, ואחרת מוסיפים את זוג הקודקודים שלה לכיסוי. נתבונן בתיאור האלגוריתם:

Algorithm: 2-Approximation for Vertex Cover

1. $A \leftarrow \emptyset$
2. While there is an uncovered edge $\{u, v\}$: add both u, v to A
3. **return** A

משפט 53. האלגוריתם הנ"ל הוא $2-\text{approximation}$. כלומר, A היא כיסוי קודקודים ומתקיים $|A| \leq 2 \cdot |Opt|$

הוכחה: יהי M קבוצת הקשתות שנבחרו, קבוצה זו מהווה שידוך - כלומר: אין שתי קשתות בM שחולקות קודקוד משותף. נניח בשלילה שקיימות זוג קשתות $e_1 = (x, y), e_2 = (y, z)$ החולקות קודקוד משותף. בה"כ e_1 נבחרה ראשונה לכיסוי. אזי, בעת בחירת הקשת e_2 היא נבחרה כיוון שהיא לא הייתה מכוסה - כלומר y, z לא היו בכיסוי בעת כניסתה. אבל, $e_1 = (x, y)$ וקודקודיה כבר נכנסו לכיסוי. סתירה.

מכאן, הפתרון האופטימלי חייב להכיל לפחות קודקוד אחד מכל קשת שנמצאת בM, אחרת הוא לא כיסוי קודקודים. לכן מתקיים

$$|A| = 2 \cdot |M| \leq 2 \cdot |Opt|$$

שכן, A הוא מס' הקודקודים. וכן, הוא מהווה שידוך שגודלו $|M|$ - שיוך נמדד בגודלו בקשתות, בכל צעד של בחירת קשת מוסיפים 2 קודקודים זרים לA, ואין בM קשתות חופפות, בהכרח A הוא מגודל פעמיים כפול מס' הקשתות בM. כנדרש.

■

הערה 54. הניתוח הדוק. כלומר, ישנן דוגמאות של גרפים בהם האלגוריתם יניב כיסוי בגודל בדיוק פעמיים מגודל הכיסוי האופטימלי.

הגדרה 55. APX היא משפחת בעיות הקירוב $O = (I, Sol, m, type)$ מתוך מחלקת NPO כך שעבור קבוע אבסולוטי כלשהו $c \geq 0$ קיים אלגוריתם בזמן פולינומי כך שלכל מופע $i \in I$ ניתן למצוא פתרון $s' \in Sol$ כך שמתקיים:
- אם $type = \min$, אזי $m(i, s') \leq c \cdot opt(i)$ עבור $c \geq 1$.
- אם $type = \max$ אזי $m(i, s') \geq c \cdot opt(i)$ עבור $c \leq 1$.

מסקנה 56. $VC \in APX$ עם $c = 2$.

נשאלת השאלה ההגיונית, האם אפשר יותר טוב מזה? ובכן, התשובה המפתיעה היא שלא יותר מדי.

משפט 57. (Dinur, Safra 2005). תחת ההנחה כי $P \neq NP$, לא ניתן לקרב את בעיית כיסוי קודקודים ביחס טוב יותר מ1.3606.

משפט 58. (Khot, Regev 2008). תחת הנחת משחקים ייחודיים, לא ניתן לקרב את בעיית כיסוי קודקודים ביחס טוב יותר מ2.

3.3 Max Cut

הגדרה 59. בהינתן גרף לא מכוון $G=(V,E)$, בעיית חתך מקסימלי מוגדרת להיות מציאת קבוצת קודקודים $S \subseteq V$ אשר ממקסמת את מס' הקשתות שנמצאות בחתך. כלומר, קשתות שמחברות בין קודקוד שנמצא בS לבין קודקוד שנמצא ב$V \backslash S$:

$$E(S, V \backslash S) = \{\{u,v\} \in E | u \in S \text{ and } v \in V \backslash S\}$$

בעיית Max Cut היא NP־קשה.

3.3.1 אלגוריתם הסתברותי

כיוון שמדובר בבעיה קשה, אין לנו דרך לפתור אותה בזמן פולינומי. במקום זאת, נסתכל על אלגוריתם הסתברותי שפותר אותה בהסתברות די טובה:

Algorithm: Random Cut (Probabilistic Approximation for Max-Cut)

1. Add each vertex $v \in V$ to S independently with probability $\frac{1}{2}$

נרצה לנתח את תוחלת מספר הקשתות שבחתך. תהי קשת כלשהי $\{u,v\}$. על מנת שהקשת תשתייך לחתך, בדיוק אחד מהקודקודים שלה צריך להיות בתוך הקבוצה S והשני צריך להיות מחוץ לה - $V \backslash S$. לכן:

$$\Pr[\{u,v\} \in E[S, V \backslash S] = \frac{1}{2} \cdot \frac{1}{2} + \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{2}$$

שכן, ההסתברות שווה להסתברות בה"כ כי u בS וv לא בS, ועוד המקרה ההפוך. מלינאריות התוחלת, נקבל כי תוחלת גודל החתך:

$$\mathbb{E}[E(S, V \backslash S)] = \sum_{e \in E} \Pr[e \in E(S, V \backslash S)] = \sum_{e \in E} \frac{1}{2} = \frac{|E|}{2}$$

כיוון שכל קשת בפתרון האופטימלי היא קשת בגרף, בהכרח מתקיים $|E| \geq opt$. אנחנו קיבלנו כי בתוחלת מס' הקשתות הוא חצי, לכן $\frac{1}{2}|E| \geq \frac{1}{2}opt$. כלומר, האלגוריתם הפשוט הנ"ל מצליח בתוחלת לספק בממוצע כחצי מהפתרון האופטימלי. לכן מדובר באלגוריתם 0.5-approximation.

3.3.2 אלגוריתם חיפוש מקומי

כעת נציג אלגוריתם דטרמיניסטי פולינומי שמקרב את Max-cut. האלגוריתם פועל בגישה חמדנית - הוא מתחיל מחלוקה שרירותית של הקודקודים, וכל עוד ניתן לשפר את החתך ע"י העברת קודקוד יחיד מS ל$V \backslash S$ או להפך, הוא מבצע את ההעברה. כלומר, כל עוד קיים קודקוד שלא בS שהוספתו לS תגדיל את גודל החתך: הכנס אותו לS. אחרת, אם קיים קודקוד שהוצאתו מS תגדיל את גודל החתך, תוציא אותו מS. אחרת - לא ניתן לשפר: החזר את S.

Algorithm: Local Search for Max-Cut

1. $S \leftarrow$ arbitrary subset
2. While (true):
   (a) If $\exists v \notin S$ such that $|E(S \cup \{v\}, V \setminus (S \cup \{v\}))| > |E(S, V \setminus S)|$:
      i. $S \leftarrow S \cup \{v\}$
   (b) Else if $\exists v \in S$ such that $|E(S \setminus \{v\}, (V \setminus S) \cup \{v\})| > |E(S, V \setminus S)|$:
      i. $S \leftarrow S \setminus \{v\}$
   (c) Else: **return** S

ניתוח סיבוכיות זמן הריצה: בכל איטרציה של הלולאה, גודל החתך גדל באחד לפחות. מס' הקשתות המקסימלי בגרף לא מכוון עם n קודקודים חסום ע"י $|E| \leq \binom{n}{2}$. לכן, האלגוריתם יבצע לכל היותר $\binom{n}{2}$ איטרציות, כל אחת מהן בעלות פולינומית. לכן זמן הריצה פולינומי.

משפט 60. האלגוריתם הוא אלגוריתם 0.5-approximation לMax Cut.
הוכחה: נתבונן בתנאי העצירה באופטימום המקומי S. לכל קודקוד $v \in S$ מס' הקשתות הנכנסות לחתך לתוך v כלומר $|E(\{v\}, V\backslash S)|$ חייב להיות לפחות מחצית מדרגתו, שכן אחרת הוצאתו מS הייתה מגדילה את החתך בסתירה לכך שהאלגוריתם עצר. נסביר לעומק: בהינתן קודקוד כנ"ל, סך הקשתות שיוצאות ממנו הן $deg(v)$. אלו מתחלקות לשני סוגים -
1. קשתות שחוצות את החתך, מספרן $|E(\{v\}, V\backslash S)|$.
2. קשתות פנימיות בתוך S, מספרן הוא $|E(\{v\}, S)|$.
נניח ונעביר את v מהקבוצה S אל הקבוצה $V\backslash S$. במצב זה, הקשתות החוצות שהיו לו יפסיקו לחצות, והפנימיות יתחילו לחצות את החתך. השינוי בגודל החתך הרי יהיה:

$$\Delta = |E(\{v\}, S)| - |E(\{v\}, V\backslash S)|$$

האלגוריתם נעצר באופטימום מקומי, לכן מעבר שכזה בהכרח לא מגדיל את גודל החתך. לכן $\Delta \leq 0$. כלומר: $|E(\{v\}, S)| \leq |E(\{v\}, V\backslash S)|$. לכן נקבל: $|E(\{v\}, V\backslash S)| \geq \frac{\deg(v)}{2}$.
באופן זהה, לכל קודקוד $v \notin S$ מס' הקשתות ממנו אל S מקיים אף הוא $|E(\{v\}, S)| \geq \frac{deg(v)}{2}$.
אם נסכום את תרומת הקודקודים לחתך מעבר לכל v, כל קשת בחתך תספר בדיוק פעמיים (פעם לכל קודקוד בה). לכן גודל החתך הינו:

$$|E(S, V\backslash S)| = \frac{1}{2}\sum_{v\in V}|E(\{v\}, V\backslash S)| \geq \frac{1}{2}\sum_{v\in V}\frac{deg(v)}{2} = \frac{|E|}{2} \geq \frac{\text{opt}}{2}$$

לכן, גודל החתך בסיום הוא לפחות חצי מהפתרון האופטימלי. לכן Max-Cut $\in APX$.

הערה 61. החסם של האלגוריתם הדוק.

משפט 62. קיימים אלגוריתמי קירוב טובים יותר לחתך מקסימלי:
1. 1994: אלגוריתם קירוב שמשיג יחס קירוב של

$$\frac{\pi}{2} \cdot \min_{\theta\in[0,\pi]} \frac{\theta}{1-\cos(\theta)} \approx 0.87$$

2. משפט Hasted: תחת ההנחה כי $NP \neq P$, לא קיים אלגוריתם פולינומי שישיג יחס קירוב טוב יותר מ$\frac{16}{17}$ לבעיה.

3.4 Minimum Makespan Scheduling

הגדרה 63. בעיית תזמון למזעור זמן הסיום מוגדרת באופן הבא: יהיו n משימות - $j_1, ..., j_n$ כך שלכל משימה j_i ישנו זמן עיבוד נדרש להשלמתה - j_i מייצג את הזמן שנדרש לבצע את המשימה הi. ויהי $m \in \mathbb{N}$ מס' המכונות אשר פועלות במקביל, ובקצב אחיד, עליהן ניתן להריץ את המשימות. המטרה: למצוא חלוקה/שיבוץ של n המשימות בין m המכונות כך שזמן הסיום הכולל יהיה מינימלי. זמן הסיום (makespan) מוגדר להיות זמן העבודה של המכונה העמוסה ביותר:

$$\text{Makespan} = \max_{1\leq k\leq m}\left(\sum_{j_i\in M_k} j_i\right)$$

משפט 64. בעיית תזמון למזעור זמן הסיום היא בעיה NP-קשה. גם כאשר $m = 2$, זו בעיית Partition, שגם היא NPC.

3.4.1 אלגוריתם חמדן

נציג אלגוריתם חמדן לקירוב הבעיה.

Greedy Algorithm for Job Scheduling

1. Order the jobs arbitrarily
2. Schedule jobs on machines in this order, scheduling the next job on the machine that has been assigned the **least amount** of work so far

האלגוריתם עובד באופן הבא: קובעים סדר שרירותי כלשהו עבור n המשימות, ואז עוברים על המשימות אחת אחת לפי הסדר שנקבע, ומשבצים את המשימה הנוכחית למכונה בעלת העומס המצטבר (!) הקטן ביותר באותו הרגע.

משפט 65. האלגוריתם מספק יחס קירוב של $2 - \frac{1}{m}$

הוכחה: ראשית, נשים לב לזוג אבחנות שחייבות להתקיים בכל פתרון אופטימלי. מתקיים בהכרח $opt \geq \frac{\sum_{i=1}^{n} j_i}{m}$ שכן סך העבודה היא סכום זמני המשימות, אם נחלק את העבודה הזו באופן מושלם בין m המכונות נקבל את ממוצע העומס. המכונה העמוסה ביותר לא יכולה לסיים לפני הממוצע הזה. לכן בהכרח, הפתרון האופטימלי הוא לפחות ממוצע העומס. שנית, בהכרח מתקיים $opt \geq \max_i j_i$. שכן, מכונה שמקבלת את המשימה הארוכה ביותר חייבת לעבוד לפחות משך הזמן שלוקח לבצע אותה. כעת, נרצה להוכיח את יחס הקירוב.

תהי q המכונה שעבדה הכי הרבה זמן באלגוריתם שלנו, ויהי t_{max} זמן הסיום שלה. תהי j_s המשימה האחרונה ששובצה למכונה q. ויהי t_s זמן העבודה של המכונה q לפני ששובצה אליה המשימה האחרונה. מכאן זמן הריצה של האלגוריתם בהכרח:

$$\text{alg time} = t_s + j_s$$

נבחין: ברגע הזמן t_s בו המשימה j_s שובצה למכונה q, האלגוריתם החמדני בחר בה כי היא הייתה המכונה הפנויה ביותר. לכן, באותו הרגע כל m המכונות האחרות היו עמוסות לפחות באותה מידה ולכן כולן עבדו לפחות בזמן t_s. אם נסכום את עבודת המכונות עד לאותו הרגע $(m \cdot t_s)$, ונוסיף את המשימה האחרונה, נראה כי סכום זה חסום מלמטה ע"י סך כל המשימות שהרי לפי האבחנה שלנו חסום ב$m \cdot opt$. נקבל:

$$m \cdot t_s + j_s \leq \sum_{i=1}^{n} j_i \leq m \cdot opt$$

נחלק את אי השוויון בm ונקבל:

$$t_s + \frac{j_s}{m} \leq opt \Longrightarrow t_s \leq opt - \frac{j_s}{m}$$

נציב זאת בחזרה לביטוי זמן הסיום של האלגוריתם:

$$\text{alg time} = t_s + j_s \leq \left(opt - \frac{j_s}{m}\right) + j_s = opt + \left(1 - \frac{1}{m}\right) j_s$$

נשים לב שמהאבחנה שלנו מתקיים $j_s \leq opt$ בהכרח , לכן:

$$\text{alg time} \leq opt + \left(1 - \frac{1}{m}\right) opt = \left(2 - \frac{1}{m}\right) opt$$

כנדרש.

הערה 66. מדובר בחסם הדוק.

משפט 67. נניח אלגוריתם תזמון שבו ממיינים מראש את המשימות בסדר יורד לפי משך הביצוע:

$$j_1 \geq j_2 \geq ..... \geq j_n$$

אזי, האלגוריתם מבטיח makespan שאינו גרוע יותר מ$\frac{3}{2}$ ביחס לפתרון האופטימלי.
הוכחה: יהי opt זמן הסיום המקסימלי בפתרון האופטימלי. נגדיר את הקבוצה

$$J_L = j_1, ..., j_k | \forall i : j_i > \frac{opt}{2}$$

כלומר, קבוצת כל המשימות שמשכן הוא לפחות חצי מזמן הסיום האופטימלי.
ראשית, נטען כי $k \leq m$, כלומר: מס' המשימות בקבוצה זו לא יכול לעלות על מספר המכונות. נניח בשלילה שזהו לא המצב, אזי $m < k$. לפי עקרון שובך היונים, אם יש יותר מm משימות גדולות ורק m מכונות, קיימת לפחות מכונה אחת שתקבל זוג מכונות גדולות. אך כל משימה בקבוצה זו גדולה ממש מ$\frac{opt}{2}$, לכן זמן העבודה של המכונה גדול ממש מopt בסתירה.
כעת, תהי q המכונה העמוסה ביותר שהגיעה לזמן הסיום $alg = t_{max}$. תהי j_s המשימה האחרונה ששובצה למכונה q, ושובצה אליה ברגע הזמן t_s. מכאן זמן סיום האלגוריתם הינו $alg = t_s + j_s$.
כעת, נחלק לשני מקרים ביחס למיקום האינדקס s:
- $s \leq k$: כלומר, המשימה האחרונה היא משימה "גדולה". כיוון שהמשימות ממוינות בסדר יורד, המשימה j_s בקידומת לפני או בדיוק k, אם מכונה q הייתה מקבלת משימה נוספת פרט לj_s גם אותה משימה הייתה שייכת לJ_L בעקבות המיון היורד, מה שהיה גורר לפחות 2 משימות גדולות על אותה מכונה, בסתירה כפי שראינו. לכן מכונה q מכילה אך ורק את המשימה j_s ומתקיים

$$\text{alg} = j_s \leq \frac{opt}{2} \leq \frac{3}{2} opt$$

- $s > k$. כלומר, המשימה האחרונה איננה משימה גדולה. כיוון ש$s > k$ המשמעות היא שj_s לא משימה גדולה ולכן $j_s \leq \frac{opt}{2}$. ברגע הזמן j_s בו שובצה המשימה j_s למכונה q, כל m המכונות היו פעילות ותפוסות - אחרת האלגוריתם החמדני היה בוחר במכונה פנויה. לכן, סך העבודה שבוצעה עד לזמן t_s חסום מלמטה ע"י העבודה של כל המכונות עד לאותו רגע, וכפי שראינו בהוכחה קודמת, זה חסום ע"י $m \cdot opt$. לכן:

$$t_s \cdot m \leq \sum_{i=1}^{n} j_i \leq m \cdot opt \Longrightarrow t_s \leq opt$$

לכן, נראה כי:

$$\text{alg} = t_s + j_s \leq \frac{1}{2} opt + opt = \frac{3}{2} opt$$

כפי שרצינו. כלומר, במקרה זה זמן העבודה עד להגעת המשימה האחרונה במכונה הכבדה ביותר הוא לכל היותר opt, והמשימה האחרונה חסומה ב$\frac{1}{2} opt$ בזמנה. כנדרש.

■

משפט 68. (Graham,1969). קיים אלגוריתם מתקדם יותר של תזמון למציאת MakeSpan מינימלי, בו ממיינים את המשימות בסדר יורד לפי משך הביצוע שלהם, ומקבלים יחס קירוב של $\frac{4}{3}$ ביחס לפתרון האופטימלי. חסם זה הוא הדוק.

משפט 69. בעיית תזמון למציאת Makespan מינימלי מקיימת סכמת קירוב בזמן פולינומי $(PTAS)$. המשמעות, לכל $\varepsilon > 0$ ניתן למצוא קירוב של $1 + \varepsilon$ בזמן ריצה פולינומי.

3.5 סכמות קירוב

הגדרה 70. PTAS (Polynomial-Time Approximation Scheme): בעיית אופטימיזציה מסוג מזעור מקיימת $PTAS$ אם לכל $\varepsilon > 0$ קבוע, קיים אלגוריתם שמשיג קירוב של $1+\varepsilon$ בזמן פולינומי.

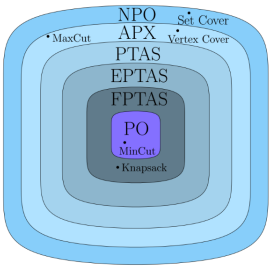
לדוגמה, זמן ריצה הגיוני לאלגוריתם שכזה הוא $n^{\mathcal{O}(\frac{1}{\varepsilon})}$ או $n^{2\cdot\mathcal{O}(\frac{1}{\varepsilon})}$. למעשה זה פולינומי אך עדיין תלוי בדיוק בחזקה ויכול להיות איטי מאוד.

הגדרה 71. EPTAS (Efficient Polynomial-Time Approximation Scheme): בעיית אופטימיזציה מסוג מזעור מקיימת $EPTAS$ אם לכל $\varepsilon > 0$ קבוע, קיים אלגוריתם שמשיג קירוב של $1+\varepsilon$ בזמן ריצה של $\mathrm{Poly}(n) \cdot f\left(\frac{1}{\varepsilon}\right)$, באשר f יכולה להיות כל פונקציה שהיא.

למשל, זמן ריצה הגיוני לאלגוריתם שכזה הוא $\mathrm{Poly}(n) \cdot 2^{\mathcal{O}(\frac{1}{\varepsilon})}$. כאן, זמן הריצה יותר יעיל.

הגדרה 72. FPTAS (Fully Polynomial-Time Approximation Scheme). בעיית אופטימיזציה מסוג מזעור מקיימת $FPTAS$ אם כל $\varepsilon > 0$ קבוע, קיים אלגוריתם שמשיג קירוב של $1+\varepsilon$ בזמן ריצה של $\mathrm{Poly}\left(n+\frac{1}{\varepsilon}\right)$.

הערה 73. תחת ההנחה כי $P \neq NP$, כך נראית תמונת העולם של המחלקות:



כפי שראינו, MinCut לפי משפט $MFMC$ שקול לזרימה מקסימלית בגרף. זו בעיית אופטימיזציה לה קיים אלגוריתם פולינומי יעיל (Dinic). המחלקות מקיימות הירכיה של הכלות, כאשר למשל Set Cover $\in NPO$, שכן ניתן לאמת פתרון חוקי. וכן, APX זו מחלקת הבעיות שקיים עבורן אלגוריתם שמבטיח יחס קבוע - למשל VC,MaxCut.

משפט 74. תחת ההנחה כי $P = NP$, כל המבנה קורס ומתקיים $NPO = PO$. כלומר, כל מחלקות הסיבוכיות באופטימיזציה מתמוטט וקורס למחלקה PO.

3.6 מרחבים מטריים (Metric space)

מרחב מטרי הוא מבנה מתמטי אשר מגדיר פורמלית את המושג מרחק בין איברים בקבוצה כלשהי.

הגדרה 75. מרחב מטרי הוא זוג סדור (X, d_X) באשר X היא קבוצה כלשהי וכן $d_X : X \times X \to \mathbb{R}_{\geq 0}$ פונקציית מרחק המקיימת שלוש אקסיומות:
1. זהות: לכל $x, y \in X$ מתקיים $d_X(x,y) = 0$ אם ורק אם $x = y$.
2. סימטריה: לכל $x, y \in X$ מתקיים $d_X(x,y) = d_X(y,x)$.
3. אי שוויון המשולש: לכל $x, y, z \in X$ מתקיים $d_X(x,y) \leq d_X(x,z) + d_X(z,y)$.

דוגמה 76. מס' דוגמאות למרחבים מטריים:

$$\ell_2 \in \mathbb{R}^d : d_{\ell_2}(x,y) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$$

$$\ell_1 \in \mathbb{R}^d : d_{\ell_1}(x,y) = \sum_{i=1}^{d} |x_i - y_i|$$

$$d_G \text{ s.t } G=(V,E) \text{ is a graph} : d_G(u,v) = \min\{|P| \text{ P is a path from u to v}\}$$

הגדרה 77. יהי (X, d_X) מרחב מטרי, כדור סגור ברדיוס $r > 0$ סביב נקודה $x \in X$ מוגדר כקבוצת כל הנקודות במרחב שמרחקן מx הוא לכל היותר r. כלומר:

$$B_X(x,r) = \{y \in X | d_X(x,y) \leq r\}$$

הגדרה 78. בעיית מרכז הk. בהינתן מרחב מטרי (X, d_X) ופרמטר שלם $k \geq 1$, המטרה היא למצוא תת קבוצה $K \subseteq X$ המקיימת $|K| = k$ (כלומר, בחירת k נקודות מרכזיות), אשר תמזער את הביטוי:

$$\max_{x \in X} d_X(x,K) = \max_{x \in X} \min_{y \in K} d_X(x,y)$$

לצורך הבנת הרעיון, נדמיין שאנו צריכים להקים k תחנות כיבוי אשר בעיר. אנו רוצים למקם אותם כך שהתושב הכי רחוק מתחנה כלשהי לא יסבול מרחק עצום. כלומר, לוקחים את המרחק של כל נקודה למרחק הקרוב אליה ביותר (min), כלומר, זהו המרכז הקרוב y, ומחפשים את המרכז שסובל מהלקוח הרחוק ביותר ־ מנסים לבחור את המיקומים, כך שהמקסימום הזה יהיה מינימלי. כלומר: מזעור המקרה הגרוע ביותר עבור האיש הרחוק ביותר. באופן שקול נוכל להגדיר את הבעיה:
מציאת הרדיוס המינימלי r וקבוצת מרכזים K כך שאיחוד הכדורים ברדיוס r סביב כל נקודות המרכז מכסה את כל המרחב X:

$$X = \bigcup_{y \in K} B_X(y,r)$$

כלומר, כל המרחב X מכוסה לחלוטין ע"י איחוד הכדורים ברדיוס r אשר המרכז שלהם נמצא בK.

3.6.1 אלגוריתם גונזלס (Gonzalez Algorithm) לבעיית הk-center

האלגוריתם הוא אלגוריתם חמדן פשוט ויעיל שנבנה עבור הבעיה. בתחילה, בוחרים מרכז ראשון ־ t_1: באופן שרירותי, אותו מקבעים כמרכז הראשון. בכל איטרציה, בוחרים קודקוד t_{i+1} את הקודקוד שהוא הכי רחוק מבין כל המרכזים שנבחרו עד כה: $\{t_1, ..., t_i\}$ והופכים אותו למרכז הבא. כך ממשיכים עד שבוחרים k מרכזים. בסיום $K = \{t_1, ..., t_k\}$.

Algorithm: k-Center Selection (Farthest-First Traversal)

1. **Choose the first center (t_1):** Pick an arbitrary vertex in the space and set it as the first center
2. **Iterative selection (t_{i+1}):** At each step, choose the vertex that is **farthest** from all centers chosen so far ($\{t_1, \ldots, t_i\}$), and make it the next center
3. Continue until exactly k centers have been chosen: the final set of centers is $K = \{t_1, \ldots, t_k\}$

נגדיר את הפרמטר

$$\Delta = d_X(t_{k+1}, \{t_1, ..., t_k\})$$

זהו המרחק של הנקודה ה$k+1$ (הנקודה הבאה שהאלגוריתם היה בוחר אילו היינו ממשיכים עוד צעד אחד), כלומר ־ זהו המרחק של נקודה בודדת t_{k+1} שמוגדר להיות המרחק מהמרכז הקרוב ביותר אליה מבין המרכזים בקבוצה, אך מהגדרת האלגוריתם זו הנקודה הרחוקה ביותר מבין כל נקודות המרחב ביחס לקבוצה K ולכן

$$\Delta = \max_{x \in X} \min_{y \in K} d_X(x,y)$$

משפט 79. מתקיים $X=\bigcup_{y\in K}B_X\left(y,\Delta\right)$. כלומר, המרחב כולו מכוסה ע"י איחוד הכדורים ברדיוס Δ סביב איברי K.
הוכחה: תהי נקודה $x\notin K$, לפי אופן פעולת האלגוריתם, t_{k+1} נבחרה בתור הנקודה שהמרחק שלה מK הוא מקסימלי מבין כל הנקודות במרחב, לכן המרחק של כל נקודה כלשהי מK חייב להיות קטן או שווה למרחק של t_{k+1} מK כלומר

$$d_X\left(x,K\right)=d_X\left(x,\{t_1,.....,t_k\}\right)\leq d_X\left(t_{k+1},\{t_1,....,t_k\}\right)=\Delta$$

מכיוון שמרחקה של כל נקודה מx אל המרכז הקרוב ביותר אליה בK לכל היותר Δ נובע כי x נמצאת בתוך כדור ברדיוס Δ סביב אחד המרכזים, לכן כל המרחב מכוסה. כנדרש.

■

משפט 80. יהיו $i\neq j$ אינדקסים של זוג נקודות t_i,t_j מתוך $k+1$ הנקודות הראשונות שהאלגוריתם בחר. אזי,

$$d_X\left(t_i,t_j\right)\geq\Delta$$

הוכחה: נניח בה"כ כי $i<j$. אזי, t_i נבחרה לפני t_j. כיוון שt_j עצמה נבחרה להיות הרחוקה ביותר מהקבוצה הקודמת $\{t_1,....,t_{j-1}\}$ והנקודה t_i הרי נמצאת בקבוצה זו, המרחק של t_j מקבוצה זו חסום מלמטה ע"י המרחק הספציפי שלה מt_i לכן $*$. מכאן:

$$d_X\left(t_i,t_j\right)\geq_* d_X\left(t_j,\{t_1,.....,t_{j-1}\}\right)\geq_{**} d_X\left(t_{k+1},\{t_1,...,t_{j-1}\}\right)\geq_{***} d_X\left(t_{k+1},\{t_1,...,t_k\}\right)=\Delta$$

באשר $**$ נכון כי האלגוריתם בחר את t_j בתור הנקודה הרחוקה ביותר במרחב כולו מהקבוצה $\{t_1,...,t_{j-1}\}$. בפרט, כיוון שt_{k+1} היא גם נקודה במרחב, המרחק שלה מהקבוצה היה קטן־שווה.
וכן $***$ נכון כי הוספת נקודות לקבוצה יכולות רק להקטין את המרחק של t_{k+1} מהקבוצה.

הערה 81. הנקודה ה$k+1$ לא נמצאת מעולם ע"י האלגוריתם, אלא משמשת לניתוח מתמטי בלבד של החסם.

משפט 82. מתקיים $opt \geq \frac{\Delta}{2}$ ולכן האלגוריתם של גונזלס הוא 2-approximation לבעיית מרכז הk.
הוכחה: נסתכל על פתרון אופטימלי כלשהו לבעיה, נסמנו K_{opt}: הוא מכיל k מרכזים. נגדיר פונקציה f אשר משייכת כל אחת מ$k+1$ הנקודות שבחרנו $(\{t_1, ..., t_{k+1}\})$ אל המרכז הקרוב ביותר אליה מתוך הפתרון האופטימלי. לפי עקרון שובך היונים, ישנם $k+1$ נקודות מקור ורק k יעדים, לכן חייבת להיות לפחות זוג נקודות בסדרה שלנו, נסמנם t_i, t_j שמשויכות לאותו מרכז אופטימלי בדיוק. נסמנו y. כלומר:

$$f(t_i) = f(t_j) = y$$

כעת, לפי אי שוויון המשולש עבור הנקודות הללו מתקיים

$$d_X(t_i, t_j) \leq d_X(t_i, y) + d_X(y, t_j)$$

אך, אנו יודעים כי המרחק בין זוג הנקודות הוא לפחות Δ לפי משפט קודם. לכן:

$$\Delta \leq d_X(t_i, y) + d_X(y, t_j)$$

כיוון שy הוא מרכז בפתרון האופטימלי, והמרחק של כל נקודה למרכז הקרוב ביותר אליה מוגבל ע"י רדיוס האופטימום, מתקיים $d_X(t_i, y), d_X(y, t_j) \leq opt$. נקבל:

$$\Delta \leq 2opt \Longrightarrow \frac{\Delta}{2} \leq opt$$

אם כן, האלגוריתם של גונזלס מייצר פתרון בו הרדיוס המקסימלי הוא בדיוק Δ, לכן $\Delta \leq 2opt$ כפי שראינו, כלומר האלגוריתם מחזיר רדיוס Δ וקבוצת מרכזים שהרדיוס הוא לכל היותר 2 מהפתרון האופטימלי ולכן מדובר ב2-approximation.
■

משפט 83. תחת ההנחה כי $P \neq NP$, לכל $\varepsilon > 0$ לא קיים אלגוריתם קירוב בזמן פולינומי ביחס של $2 - \varepsilon$ לבעיית ה$k-$center.
הוכחה: ההוכחה מתבצעת ברדוקצייה מDS. נזכר כי DS מוגדרת להיות: בהינתן גרף $G = (V, E)$ קבוצה $A \subseteq V$ נקראת קבוצה שולטת אם לכל $v \in V$ מתקיים $v \in A$ or $\exists w \in \Gamma(v) : w \in A$. בעיית ההחלטה המתאימה, היא האם בהינתן $k \in \mathbb{N}$ קיימת קבוצה שולטת בגרף בגודל לכל היותר k. נגדיר את הרדוקציה באופן הבא:
בהינתן גרף $G = (V, E)$ וערך k טבעי, נגדיר מרחב מטרי על קודקודי הגרף באמצעות מטריקת המסלולים הקצרים ביותר בין קודקודי הגרף. כלומר - (V, d_G).
כעת, נניח בשלילה שקיים אלגוריתם קירוב בזמן פולינומי ביחס $t = 2 - \varepsilon$ עבור $\varepsilon > 0$ לבעיית הK-center. נפעיל את האלגוריתם על המרחב (V, d_G) כדי לקבל פתרון $K \subseteq V$ בעל רדיוס α. כלל ההחלטה יוגדר להיות: אם $\alpha < 2$ החזר כן, אחרת אם $\alpha \geq 2$ החזר לא. נוכיח נכונות:
מקרה ראשון - קיימת קבוצה שולטת בגודל לכל היותר k: משמעות הדבר כי לכל קודקוד בגרף יש שכן בקבוצה זו, או שהוא בעצמו בה. לכן, מרחב הקודקודים מכוסה לחלוטין ע"י איחוד כדורים ברדיוס 1 סביב איברי K כלומר $V = \bigcup_{u \in K} B_G(u, 1)$, לכן קיים פתרון מדויק אופטימלי לבעיית הK-center. כיוון שהנחנו שאלגוריתם הקירוב בעל יחס של $t = 2 - \varepsilon$, הרדיוס α שהאלגוריתם יחזיר הינו לכל היותר עם יחס השגיאה

$$\alpha \leq 1 \cdot t = 2 - \varepsilon < 2$$

ולכן נשיב כן במקרה זה.
מקרה שני - כל קבוצה שולטת בגרף היא מגודל $k+1$ ומעלה. במקרה זה לא קיים כיסוי בגודל k, לכן כל פתרון לבעיית הK-center חייב להיות בעל רדיוס לפחות 2. כלומר $opt \geq 2$. כיוון שהאופטימום הוא לפחות 2, ואלגוריתם הקירוב מבטיח תוצאה שאינה גרועה פי יותר מt ביחס לאופטימום המינימלי, הערך α שהאלגוריתם ימצא יקיים $\alpha \geq 2$ בהכרח. לכן, האלגוריתם ישיב לא: בצדק.
סה"כ, אם היה קיים אלגוריתם פולינומי ביחס של $2 - \varepsilon$ לבעיית K-center, היינו יכולים להכריע בזמן פולינומי את DS, אך תחת ההנחה $P \neq NP$, לא יתכן הדבר. כנדרש.
■

3.7 Bin Packing Problem

הגדרה 84. בהינתן קבוצה סופית של פריטים $U=\{u_1,....,u_n\}$ עם גדלים רציונליים, כלומר $\forall i: s(u_i)\in[0,1]$. הפלט הוא חלוקה של U לתתי קבוצות־ מכלים, $U_1,....,U_k$ כך שלכל $j\in\{1,...,k\}$ סכום הגדלים של הפריטים בתוכו לא עולה על 1, כלומר:

$$\forall j\in\{1,...,k\}:\sum_{u_i\in U_j}s(u_i)\leq 1$$

כאשר המטרה היא מזעור $k\in\mathbb{N}$. מדובר בבעיה שהיא NPC.

מדובר בבעיה שקשה לקרב, תחת ההנחה $P\neq NP$ כמובן.

משפט 85. אם $P\neq NP$, לכל $\varepsilon>0$ לא קיים אלגוריתם קירוב ביחס של $\frac{3}{2}-\varepsilon$ לבעיית הBin-Packing.
הוכחה: בדומה להוכחה הקודמת שראינו, ההוכחה תתבצע ברדוקציה מPartition ־ נזכר כי בעיה זו מוגדרת בהינתן קבוצה $X=\{x_1,....,x_n\}$ האם ניתן לחלק את X לשתי קבוצות שסכומן שווה.
נניח בשלילה שקיים אלגוריתם קירוב ביחס של $\frac{3}{2}-\varepsilon$ לבעיית הBin-Packing. בהינתן קלט $S=\{x_1,.....,x_n\}$ לPartition נסמן $\mu=\sum_{x_i\in S}x_i$ ונבנה קלט עבור Bin-Packing באופן הבא:

$$S'=\left\{\frac{2}{\mu}x_1,\frac{2}{\mu}x_2,.....,\frac{2}{\mu}x_n\right\}$$

נבחין כי:

$$\sum_{x_i\in S'}\frac{2}{\mu}x_i=\frac{2}{\mu}\sum_{x_i\in S}x_i=\frac{2}{\mu}\cdot\mu=2$$

כלומר, סכום האיברים בקבוצה שבנינו הוא בדיוק 2. נפעיל את אלגוריתם הקירוב בהנחה שהוא פועל על הקלט החדש S', האלגוריתם יחזיר פתרון אריזה הכולל מס' כולל של k מכלים. אם מס' הכלים שהתקבל הוא בדיוק $k=2$ נחזיר כן, אחרת כלומר $k\geq 3$ נחזיר לא.
כעת נרצה להוכיח את נכונות הרדוקציה:
בכיוון ראשון, נניח שקיימת חלוקה חוקית בבעיית החלוקה המקורית. אזי לכל קבוצה בחלוקה ישנו סכום של $\frac{\mu}{2}$, משמעות הדבר במרחב של S' כי ניתן לארוז את כל הפריטים בדיוק בזוג מכלים. כלומר, $opt=2$. האלגוריתם מבטיח יחס קירוב של $\frac{3}{2}-\varepsilon$ ולכן מס' הכלים שהאלגוריתם יחזיר יקיים את החסם:

$$k<\left(\frac{3}{2}-\varepsilon\right)opt=3-2\varepsilon$$

מס' המכלים חייב להיות מס' שלם, ולכן בהכרח נקבל $k\leq 2$ ולכן האלגוריתם ישיב כן.
בכיוון שני, נניח שלא קיימת חלוקה בבעיית הPartition המקורית. משמעות הדבר כי שום חלוקה של המספרים לזוג קבוצות לא תניב סכומים ששווים בדיוק ל$\frac{\mu}{2}$. כלומר, לא אפשרי לארוז את הפריטים של S' (שסכומם הכולל הוא 2) בשני מכלים בלבד, כך שכל מיכל יכיל לכל היותר 1 כי אם היו זוג מכלים כאלו, הם בהכרח היו חייבים להיות מלאים עד בדיוק 1, ואז היינו מקבלים זוג מכלים מלאים בסתירה לכך שאין פתרון לבעיית החלוקה. לכן בהכרח $\mathrm{Opt}\geq 3$, האופטימום הוא לפחות 3 ולכן אלגוריתם הקירוב לא יוכל להחזיר 2 שהרי יחזיר ערך גדול או שווה מ3, ולכן נחזיר לא.
לכן סה"כ הרדוקציה חוקית ונכונה, אם היה קיים אלגוריתם כנ"ל בזמן פולינומי היינו מכריעים את Partition בסתירה לכך ש$P\neq NP$.

■

3.7.1 First fit alogorithm

כעת נציג אלגו' פשוט שכן יקרב את Bin-Packing, פולינומי, שעובד באופן הבא: לכל פריט μ_i לפי סדר הופעתו, הכנס אותו אל המיכל בעל האינדקס הקטן ביותר שיכול להכיל אותו.

Algorithm: First Fit (Bin Packing)

The First Fit approximation algorithm for the bin packing problem works as follows:

1. We start with n empty bins: $B_1, B_2, \ldots, B_n$
2. For each item μ_i, in order of appearance ($i \in \{1, \ldots, n\}$), we insert it into the bin with the **smallest index** that it can still fit into (i.e. the sum of weights in that bin does not exceed 1 after the insertion)

משפט 86. מספר המכלים שאלגוריתם ה-First-Fit מחזיר עבור קלט I, יסומן ב-$FF(I)$ ויקיים:

$$FF(I) \leq 2 \cdot opt + 1$$

ולכן, מדובר באלגוריתם 3-approximation.
הוכחה: נשים לב כי המשקל הכולל של כל הפריטים מוכל תחת הפתרון האופטימלי, כיוון שכל מיכל בפתרון האופטימלי מכיל משקל כולל של לכל היותר 1 ולכן מתקיים

$$\text{Opt} \geq \sum_{\mu_i \in U_j} s(\mu_i)$$

כלומר, המשקל הכולל חסם תחתון לפתרון האופטימלי. כעת, יהיו $B_1, ..., B_m$ המכלים שהוחזרו ע"י האלגוריתם. נטען: יש לכל היותר מיכל יחיד B_j שמשקלו קטן או שווה מחצי. שכן, אילו היו שני מיכלים שונים שמשקלם קטן מחצי, האלגוריתם היה מאחד אותם. כיוון שיש לפחות $m-1$ מכלים שמשקל כל אחד מהם גדול ממש מ-$\frac{1}{2}$, סכום המשקלים הכולל של כל הפריטים מקיים

$$\text{Opt} \geq \sum_{\mu_i \in U_j} s(\mu_i) \geq \frac{m-1}{2} \Longrightarrow \text{Opt} \geq \frac{m-1}{2} \Longleftrightarrow 2 \cdot Opt \geq m-1 \Longrightarrow m \leq 2 \cdot Opt + 1$$

נשים לב כי $opt, \geq 1$, לכן נוכל לומר כי $m \leq 2 \cdot opt + 1 \leq 2 \cdot opt + opt = 3 \cdot opt$, ולכן האלגוריתם הינו 3-approximation.
■

משפט 87. לכל קלט I, אלגוריתם ה-First-Fit מקיים את החסם:

$$\text{FF}(I) \leq \frac{17}{10} \cdot opt(I) + 2$$

משפט 88. אלגוריתם ה-FFD מוגדר באופן דומה ל-FF, כאשר הוא מתחיל במיון הפריטים בסדר יורד - מהגדול לקטן. עבור אלגוריתם זה, מתקיים לכל קלט I:

$$\text{FFD}(I) \leq \frac{11}{9} \cdot opt(I) + 4$$

כלומר, הוא קרוב מאוד (≈ 1.22) לפתרון האופטימלי במקרה הגרוע ביותר, ולכן הוא מאלגוריתמי הקירוב היעילים והפופולריים ביותר לבעיה.

3.7.2 PTAS עבור Bin-Packing

משפט 89. לכל $\varepsilon > 0$ קבוע, קיים אלגוריתם בזמן פולינומי שמוצא פתרון ל-Bin-Packing המשתמש בלכל היותר

$$(1+\varepsilon)\,\text{Opt} + 1 \text{ bins}$$

הוכחת המשפט, והאלגוריתם עצמו, מורכבים. לכן, נחלק את הבעיה למס' מקרים פרטיים, ובסיום נאחד אותם לאלגוריתם אחד כולל.

משפט 90. נניח כי כל הפריטים הם בגודל לכל היותר δ, אזי מס' המכלים של אלגוריתם FF מקיים

$$FF\left(I\right)\leq\left(1+2\delta\right)\cdot opt+1$$

הוכחה: אם $\delta\geq\frac{1}{2}$, נקבל כי $1+2\delta\geq2$ ולכן זה בדיוק הטענה שראינו קודם. אחרת, נניח $\delta<\frac{1}{2}$.
נשים לב: כל המכלים פרט לאחרון $B_1,....,B_m$ מכילים משקל של לפחות $1-\delta$. נניח בשלילה שקיים מיכל כלשהו B_j כך ש$j<m$ ־ מיכל שאיננו האחרון, בו סכום המשקלים קטן ממש מ$1-\delta$. אם סכום המשקלים בB_j קטן ממש מ$1-\delta$, אזי, המקום הפנוי במיכל הוא לפחות: $1-\left(1-\delta\right)=\delta$, אך כל הפריטים הם בגודל לכל היותר δ: זו סתירה, שכן אחרת היה נכנס למיכל זה פריט כלשהו קודם.
לכן, ישנם $m-1$ מיכלים, בכל אחד מהם יש לפחות $\delta-1$ משקל, מכאן שסכום המשקלים שנמצאים במכלים הללו הוא לפחות

$$\left(\mathrm{FF}\left(I\right)-1\right)\cdot\left(1-\delta\right)\leq opt$$

שכן ברור שסכום המשקלים חייב להיות חסם תחתון לפתרון האופטימלי, שהוא מס' הסלים, נקבל כי

$$\mathrm{FF}\left(I\right)-1\leq\frac{\mathrm{OPT}}{1-\delta}\Longrightarrow\mathrm{FF}\left(I\right)\leq\frac{\mathrm{OPT}}{1-\delta}+1$$

אך, כיוון ש$\delta<\frac{1}{2}$, אזי $\frac{1}{1-\delta}\leq2\delta+1$ וקיבלנו את הדרוש.

משפט 91. כעת נניח כי ישנה קבוצה של k ערכים אפשריים בלבד. נסמן $A=\{\alpha_1,.....,\alpha_k\}$. במקרה זה, ניתן למצוא את הפתרון האופטימלי בזמן פולינומי של $\mathcal{O}\left(n^{2k+1}\right)$ באמצעות תכנון דינמי.
הוכחה: על מנת להשתמש בתכנון דינמי, נצטרך להגדיר מהו מצב. בבעיה שלנו, מצב מתאר אילו פריטים נמצאים באריזה שלנו.
כיוון שיש רק k סוגי גדלים שונים, אנו לא חייבים לרשום את כל הרשימה של הפריטים. אלא, מספיק שנרשום כמה פריטים יש מכל סוג. לכן כל מצב ייוצג ע"י וקטור בן k איברים

$$\mathcal{S}=\left\{\left(\beta_1,.....,\beta_k\right)\in\mathbb{N}^k|\sum_{i=1}^{k}\beta_i\leq n\right\}$$

באשר β_i הם מס' הפריטים בגודל α_i. נשים לב כי גודל מרחב המצבים הוא $\mathcal{O}\left(n^k\right)$. שכן, מדובר בבעיה קומבינטורית של חלוקת n כדורים לk תאים ופתרונה נתון ע"י

$$\left|\mathcal{S}\right|=\binom{n+k}{k}=\mathcal{O}\left(n^k\right)$$

כיוון שk קבוע, מדובר בגודל מרחב מצבים פולינומי. האלגוריתם עובד כאלגוריתם תכנון דינמי, בשלב הבסיס S_1 נמצא את כל תתי הקבוצות שניתן לארוז במיכל יחיד באופן חוקי , ניתן לחשב זאת ע"י מעבר על כל האפשרויות.
בצעד אינדוקטיבי: נניח שבידינו $\mathcal{S}_{i-1}$, כלומר אוסף כל המולטי־סטים שניתן לארוז ב$i-1$ מכלים, כדי למצוא את $\mathcal{S}_i$ נעזר באבחנה

$$\mathcal{S}_i\subseteq\{B+C|B\in\mathcal{S}_{i-1},C\in\mathcal{S}_1\text{ and }|B|+|C|\leq n\}$$

כלומר, לקיחת מולטי סט שארזנו ב$i-1$ מכלים והוספת מיכל יחיד מלא.
זמן הריצה: כדי לחשב את $\mathcal{S}_i$ לכל שלב i, אנו עוברים על כל הזוגיות האפשריים במרחב המצבים בעלות: $\mathcal{O}\left(n^k\right)\cdot\mathcal{O}\left(n^k\right)=\mathcal{O}\left(n^{2k}\right)$. יש סה"כ n מכלים אפשריים שהרי $1\leq i\leq n$ ולכן זמן הריצה הכולל הינו

$$\mathcal{O}\left(n\right)\cdot\mathcal{O}\left(n^{2k}\right)=\mathcal{O}\left(n^{2k+1}\right)$$

משפט 92. נניח כעת כי כל הפריטים בקלט הם בגודל של לפחות δ. כלומר לכל פריט μ_i מתקיים $s(\mu_i) \geq \delta$. אזי, קיים אלגוריתם שמייצר פתרון לבעיית אריזת המכלים המקיים:

$$\mathrm{Alg}(U) \leq (1+\delta) \cdot \mathrm{Opt}(U) + 1$$

וזמן הריצה של האלגוריתם הינו $\mathcal{O}\left(n^{\frac{2}{\delta^2}+1}\right)$.

הוכחה: יהי k כלשהו, אשר ערכו יקבע בהמשך הניתוח. נגדיר את האלגוריתם הבא:

- ניקח את קבוצת הפריטים U ונמיין אותם לפי משקלם.
- נחלק את הפרטים הממויינים ל$\left[\frac{n}{k}\right]$ קבוצות ונסמנן $G_1, ..., G_{\left[\frac{n}{k}\right]}$.
- כל קבוצה G_i מכילה בדיוק k פריטים - פרט לאחרונה, שיכולה להכיל פחות פריטים.
- הקבוצות מסודרות לפי סדר גדלים, הקבוצה G_1 מכילה את הפרטים הגדולים ביותר וכו'.

על מנת לפתור את הבעיה ביעילות, נגדיר מופע קטן יותר U' באופן הבא:

1. נתעלם לחלוטין מהקבוצה הראשונה G_1.
2. לכל שאר הקבוצות $2 \leq i \leq \left[\frac{n}{k}\right]$ נשנה את המשקל של כל הפריטים בקבוצה כך שיהיו שווים למשקל הפריט הכבד ביותר באותה הקבוצה.

נטען כמה טענות. ראשית, מתקיים $\mathrm{Opt}(U') \leq \mathrm{Opt}(U)$: שכן, קיימת פונקציה חד חד ערכית $f : U' \to U$ כך שכל אלמנט במופע החדש U' ממופה לאלמנט במופע המקורי שלו במשקל שווה או גדול יותר, כיוון שכל פריט בU' הוחלף למשקל המקסימלי של קבוצתו, ובמופע המקורי U היו שם פריטים מקוריים, כל אריזה חוקית בU יכולה לשמש בסיס לאריזה בU', לכן סה"כ האופטימום של U' לא יכול לעלות על האופטימום של U.

שנית, ניתן לחשב את $\mathrm{Opt}(U')$ בזמן $\mathcal{O}\left(n^{2\cdot\frac{n}{k}+1}\right)$. שכן: נשים לב כי במופע החדש U' כיוון שזרקנו את G_1 ישנן $\left[\frac{n}{k}\right]-1$ קבוצות, בכל קבוצה כזו הפריטים קיבלו ערך אחיד ולכן ישנם לכל היותר $\frac{n}{k} \leq \left[\frac{n}{k}\right]-1$ משקלים שונים סה"כ בתוך U'. כפי שראינו, נקבל בדיוק את הטענה הקודמת בשימוש בתכנות דינמי.

כעת, לאחר שהצלחנו לפתור את U' נרצה לפתור את המופע המקורי U. האלגוריתם יורכב משלושת השלבים הבאים:

1. לוקחים כל פריט ופריט בקבוצה G_1 ושמים אותו במיכל נפרד משל עצמו. סה"כ, ישנם k מכלים.
2. מוצאים את הפתרון האופטימלי למופע הקטן U'.
3. משתמשים במבנה האריזה האופטימלי של U' על מנת לארוז חזרה את יתרת הקבוצות $G_2, ..., G_{\left[\frac{n}{k}\right]}$ במופע המקורי U. נשים לב כי אריזה זו חוקית גם אצלנו שכן כל משקל בבעיה U' גדול או שווה למשקל בU.

כעת, מס' המכלים הכולל של האלגוריתם: השתמשנו בk מכלים עבור G_1 ובעוד $\mathrm{Opt}(U')$ בשלב האחרון. לכן סה"כ

$$\mathrm{Alg}(U) = \mathrm{Opt}(U') + k$$

תחת האבחנה $\mathrm{Opt}(U') \leq \mathrm{Opt}(U)$ נקבל $\mathrm{Alg}(U) \leq \mathrm{Opt}(U) + k$. כעת, נגדיר את k להיות:

$$k = \left[\delta \cdot \sum_{\mu_i \in U} s(\mu_i)\right]$$

נשים לב כי $\sum_{\mu_i \in U} s(\mu_i) \leq \mathrm{Opt}(U)$. לכן:

$$k \leq [\delta \cdot \mathrm{Opt}(U)] \leq \delta \cdot \mathrm{Opt}(U) + 1$$

ומכאן:

$$\mathrm{Alg}(U) \leq \mathrm{Opt}(U) + k \leq \mathrm{Opt}(U) + \delta \mathrm{Opt}(U) + 1 = (1+\delta)\,\mathrm{Opt}(U) + 1$$

כפי שרצינו, כלומר יחס הקירוב כפי שרצינו. כעת, באשר לזמן הריצה: נזכר כי זמן הריצה הינו $\mathcal{O}\left(n^{2\cdot\frac{n}{k}+1}\right)$. נציב את הערך של k ונעזר באבחנה שכל הפריטים בגודל לפחות δ ונקבל

$$\frac{n}{k} = \frac{n}{[\delta \cdot \sum s(\mu_i)]} \leq \frac{n}{\delta \cdot n \cdot \delta} = \frac{1}{\delta^2}$$

ולכן זמן הריצה הינו:

$$\mathcal{O}\left(n^{\frac{2}{\delta^2}+1}\right)$$

כנדרש.

■

כעת, לאחר הוכחת כל הלמות: נתפנה להצגת האלגוריתם הכללי.

Algorithm: General Case Bin Packing

1. Let U_L be all the items in U of weight $\geq \frac{\varepsilon}{2}$, and $U_S = U \setminus U_L$
2. Run the algorithm of Lemma 2 on U_L
3. Use FF to pack U_S into remaining spaces of large item bins. When all such space is used, open new bins using FF

האלגוריתם משלב את זוג הלמות שכבר הוכחנו, יחד עם אלגוריתם הFF. מגדירים סף של $\delta = \frac{\varepsilon}{2}$, ומחלקים את קבוצת הפריטים לזוג קבוצות. אלו שגדולים או שווים לסף, ואלו שמתחתיו. לקבוצת הפריטים הגדולים, משתמשים באלגוריתם שהוכחנו בלמה לעיל. עבור הפריטים הקטנים, מנסים לארוז אותם באמצעות אלגוריתם FF אל המקום הפנוי שנותר במיכלים הקיימים של הפרטים הגדולים. אם נגמר המקום במיכלים הקיימים, פותחים מכלים חדשים בהתאם. כעת נרצה להוכיח שהאלגוריתם עושה את הדרוש, כלומר מניב יחס קירוב של $1+\varepsilon$ לכל $\varepsilon > 0$.

מקרה ראשון: לא נפתחו מיכלים חדשים במהלך הFF. אם כן, אם כל הפריטים הקטנים בU_S נכנסנו לרווחים הפנויים במיכלים הקיימים, אזי בהכרח

$$\mathrm{Alg}\left(U\right) = \mathrm{Alg}\left(U_L\right)$$

נפעיל את החסם של הלמה לעיל עבור קבוצת הפריטים הגדולים וכן $\delta = \frac{\varepsilon}{2}$ ונקבל:

$$\mathrm{Alg}\left(U_L\right) \leq \left(1+\frac{\varepsilon}{2}\right) \cdot \mathrm{Opt}\left(U_L\right) + 1$$

הרי בהכרח האופטימום של U_L חוסם מלמטה את האופטימום של U ונקבל:

$$\mathrm{Alg}\left(U_L\right) \leq \left(1+\frac{\varepsilon}{2}\right) \cdot \mathrm{Opt}\left(U\right) + 1 \leq (1+\varepsilon) \cdot \mathrm{Opt}\left(U\right) + 1$$

מקרה שני: כן נפתחו מיכלים חדשים במהלך הFF. נניח כי האלגוריתם פתח סה"כ m מיכלים חדשים עבור תהליך השיבוץ הסופי. נסתכל על $m-1$ המיכלים הראשונים $(B_1, ..., B_{m-1})$ ועל המיכל האחרון B_m. כיוון שאלגוריתם FF פועל בצורה חמדנית, כל המיכלים פרט למיכל האחרון חייבים להיות מלאים מעבר לסף מסוים. כיוון שהפרטים הקטנים בגודל קטן מ$\frac{\varepsilon}{2}$, המשקל הכולל בכל אחד מהכלים הללו פרט לאחרון חייב להיות לפחות $1-\frac{\varepsilon}{2}$, אחרת פריט היה נכנס לשם. לכן סכום המשקלים של כל הפריטים שנארזו ב$m-1$ המכלים הראשונים מקיים

$$(FF-1) \cdot \left(1-\frac{\varepsilon}{2}\right) < \text{Total Weight} \leq opt$$

$$\implies FF < \frac{\mathrm{OPT}}{1-\frac{\varepsilon}{2}} + 1 \leq (1+\varepsilon) \cdot \mathrm{OPT} + 1$$

וסיימנו.

אם כן, ננסה לחשוב מדוע האלגוריתם המורכב הזה עובד - קצת לאינטואיציה. האלגוריתם הנ"ל מבצע חלוקת עבודה. הפריטים הגדולים הם אלו שלרוב קובעים את מבנה הבעיה והאופטימום, אך יש מהם מעט יחסית. לכן, ניתן להשקיע זמן בלפתור אותם בצורה מדויקת - אך יקרה מבלי לפוצץ את סיבוכיות הזמן הכוללת. הפריטים הקטנים, אמנם רבים אך בגלל שהם קטנים מאוד יחסית, הם מתנהגים כמו חומר מילוי והם גמישים - אפשר פשוט לרוב לזרוק אותם לחורים בצורה חמדנית (FF).

3.8 Minimum Multiway Cut

הגדרה 93. בהינתן גרף $G=(V,E)$ ממושקל עם פונקציית משקל w, וקבוצה של צמתים מיוחדים הנקראים טרמינלים ומסומנת ב$K=\{v_1,...,v_k\}\subseteq V$.
חיתוך רב כיווני הוא תת קבוצה של צלעות $F\subseteq E$ כך שכאשר מסירים אותן ונוצר הגרף $G'=(V,E\backslash F,w)$, כל טרמינל $v_i\in K$ שייך לרכיב קשירות שונה.
המטרה: מציאת חיתוך רב כיווני F כך שמשקל הצלעות בחיתוך F מינימלי.
באופן חלופי, נוכל להגדיר את הבעיה כיצירת חלוקה של הצמתים V לקבוצות $S_1,....,S_k$ כך שלכל $v_i\in S_i$ i והמטרה היא מזעור

$$\sum_{i<j} w\left(E\left(S_i,S_j\right)\right)$$

לכל $k\geq 3$, מדובר בבעיה NP־קשה.

נציג את האלגוריתם הבא:

Algorithm: Multiway Cut

1. For every $v_i\in K$, let E_i be the minimum $(v_i,K\setminus\{v_i\})$ cut
2. Let E_j be the cut of maximum weight among $E_1,\ldots,E_k$
3. **return** $\bigcup_{i\in[1,k]\setminus\{j\}} E_i$

האלגוריתם פועל באופן הבא ־ לכל טרמינל $v_i\in K$, האלגוריתם מוצא באמצעות רדוקציה לזרימה מקסימלית בזמן פולינומי את החיתוך המינימלי $(v_i,K\backslash\{v_i\})$ ומסמנו בE_i. בהמשך, האלגוריתם מוצא את החיתוך E_j בעל המשקל המקסימלי מבין החיתוכים $E_1,....,E_k$ ומחזיר את איחוד כל החיתוכים, פרט לאותו E_j. נשים לב שהאלגוריתם בהכרח מחזיר פתרון חוקי, כלומר לכל זוג טרמינלים הם נמצאים ברכיב קשירות אחר. שכן, בשלב הראשון האלגוריתם מוצא לכל טרמינל v_i חיתוך E_i שמנתק אותו משאר הטרמינלים בגרף ־ בפועל, האלגוריתם מחזיר את איחוד כל החיתוכים פרט לאחד. נבחין שזה מבטיח שכל זוג טרמינלים מנותק. שכן: יהיו v_i,v_q זוג טרמינלים. כיוון שהשמטנו מהאיחוד רק את E_j, לפחות אחד מהשניים ־ E_i או E_q נמצא בוודאות בתוך קבוצת הצלעות שהשבנו. בה"כ E_i נכלל בתוצאה F: לפי הגדרה E_i הוא חיתוך שמנתק את v_i משאר הטרמינלים, בפרט הוא מנתק אותו מv_q שנמצא בקבוצת האחרים ־ לכן אין מסלול מv_i לv_q. הערה ־ נשים לב כי F זו קבוצה של צלעות שנמחקת מהגרף. כיוון שמדובר באיחוד של חתכים, בהכרח כל טרמינל יהיה מנותק מאחרים.

משפט 94. האלגוריתם מספק יחס קירוב של $2-\frac{2}{k}$.
הוכחה: יהי פתרון אופטימלי כלשהו לבעיה, E^*. לכל טרמינל v_i, נגדיר את S_i להיות רכיב הקשירות של v_i בגרף שנוצר לאחר הסרת צלעות הפתרון האופטימלי. כלומר, בגרף $G'=(V,E\backslash E^*,w)$.
נגדיר את $E_i^*=E\left(S_i,V\backslash S_i\right)$ להיות קבוצת הצלעות היוצאות החוצה מהרכיב של v_i בפתרון האופטימלי. כיוון שכל הטרמינלים האחרים אינם נמצאים ברכיב הקשירות S_i, קבוצת הצלעות E_i^* מהווה חיתוך חוקי מהסוג $(v_i,K\backslash\{v_i\})$. כיוון שE_i מראש הוגדר כחיתוך המינימלי האפשרי המנתק את v_i משאר הטרמינלים, נובע כי בהכרח משקלו קטן או שווה למשקל החיצוני בפתרון האופטימלי:

$$w\left(E_i\right)\leq w\left(E_i^*\right)$$

עבור כל הטרמינלים, נקבל כי:

$$\sum_{i=1}^{k}w\left(E_i\right)\leq\sum_{i=1}^{k}w\left(E_i^*\right)=_*2w\left(E^*\right)$$

באשר $*$ נכון כי כל צלע שנמצאת בפתרון האופטימלי E^* בהכרח מחברת בין שני רכיבי קשירות שונים S_i,S_j ולכן צלע זו תופיע פעמיים בסכום הE_i^*. מכאן נקבל:

$$\sum_{i=1}^{k}w\left(E_i\right)\leq2w\left(E^*\right)$$

האלגוריתם מייצר את הפתרון הסופי F ע"י איחוד הקבוצות פרט לחיתוך הכבד ביותר E_j

$$w\left(F\right)\leq\sum_{i=1}^{k}w\left(E_i\right)-w\left(E_j\right)$$

נראה כי כיוון שE_j נבחר להיות החיתוך בעל המשקל המקסימלי מבין כל הE_i, משקלו גדול או שווה לממוצע כל החיתוכים: $w\left(E_j\right)\geq\frac{1}{k}\sum_{i=1}^{k}w\left(E_i\right)$. מכאן נקבל עם קצת אלגברה:

$$w\left(F\right)\leq\sum_{i=1}^{k}w\left(E_i\right)-w\left(E_j\right)\leq\sum_{i=1}^{k}w\left(E_i\right)-\frac{1}{k}\sum_{i=1}^{k}w\left(E_i\right)=\left(1-\frac{1}{k}\right)\sum_{i=1}^{k}w\left(E_i\right)\leq\left(1-\frac{1}{k}\right)\cdot2w\left(E^*\right)$$

לכן קיבלנו סה"כ

$$w\left(F\right)\leq\left(2-\frac{2}{k}\right)w\left(E^*\right)$$

כנדרש.

■

הערה 95. החסם הדוק. כמו כן,
בשנת 2014 הציגו אלגוריתם המציג יחס קירוב של 1.2965 לבעיה.
בשנת 1994 הוכיחו כי עבור $k\geq3$ הבעיה לא ניתנת לקירוב תוך פקטור של $1+\delta$ עבור $\delta>0$, כלומר לבעיה אין $PTAS$.
בשנת 2019 הוכיחו כי תחת השערת המשחקים הייחודיים, הבעיה לא ניתנת לקירוב בפקטור שטוב מ1.20.

3.9 תרגול

בתרגול נמשיך להתעסק באלגוריתמים קירוביים. לפני שבונים אלגוריתמים, עלינו להגדיר מה בדיוק אנחנו מבטיחים.
נגדיר ρ־קירוב. יהי $\rho\geq1$ קבוע. אלגוריתם ρ־קירוב מבטיח כי:
1. אם מדובר בבעית אופטימיזציה מסוג מינימיזציה אזי $\mathrm{OPT}\leq\mathrm{ALG}\leq\rho\cdot\mathrm{OPT}$

2. אם מדובר בבעית מקסימיזציה אזי $\frac{\mathrm{OPT}}{\rho} \leq \mathrm{ALG} \leq \mathrm{OPT}$

באשר ALG זהו הערך של הפתרון שהאלגוריתם מחזיר וOPT זה ערך הפתרון הטוב ביותר.
במינימיזציה, האלגוריתם שנביא לא יוכל להיות יותר טוב מהאופטימום, לא נוכל להביא יותר טוב ממנו: נביא קצת יותר ממנו אולי, לא יותר מערך מסויים ρ כפול הOPT־ימום. במקסימיזציה, אנחנו מחזירים ערך של פתרון שהוא לכל היותר האופטימלי.

מה שחשוב ־ ה**ערבות** הנ"ל חייבת להתקיים לכל קלט, גם אם האלגוריתם לא יודע מהו OPT.

הגדרה 96. יהי עולם U עם $|U| = n$ איברים ותהי $\mathcal{F}$ משפחה של m תתי קבוצות כך ש

$$\bigcup_{S \in \mathcal{F}} S = U$$

המטרה היא לבחור כמה שפחות קבוצות כך שאיחודן יהיה U:

$$\min \left\{ |C| : C \subseteq \mathcal{F} \text{ s.t } \bigcup_{S \in C} S = U \right\}$$

בעיה זו נקראת Set-Cover. כל קבוצה היא עלות של 1 מבחינתנו.

מדובר בבעיה NP־קשה. לכן נתפנה לקרב אותה. בתרגול קודם, ובהרצאה ראינו אלגוריתם f־קירוב באשר:

$$f = \max_{e \in U} |\{S \in \mathcal{F} : e \in S\}|$$

כלומר, מס' הקבוצות המקסימלי שאיבר יכול להיות בו. הבעיה? f יכול להיות יחסית גדול ־ ולכן נרצה להציג אלגוריתם שלא תלוי בf, אלא אולי בערך קבוע כלשהו.
נתבונן באלגוריתם הבא:

Algorithm: Greedy Set Cover

1. $R \leftarrow U, \quad C \leftarrow \emptyset$
2. While $R \neq \emptyset$:
   (a) Choose $S \in \arg\max_{T \in F} |T \cap R|$
   (b) $C \leftarrow C \cup \{S\}, \quad R \leftarrow R \setminus S$
3. **return** C

מה האלגוריתם עושה? בתחילה הוא מאתחל את R להיות U, כל העולם. בכל שלב R הוא אוסף האיברים שעדיין לא מכוסים. הוא בוחר בכל שלב קבוצה S להיות הקבוצה עם הרווח הגדול ביותר, כלומר זו שמכסה כמה שיותר איברים ממי שנשארו. הוא מוסיף אותה לC: כיסוי הקבוצות, ומוריד את איבריה מR. נשים לב כי אלגוריתם זה הוא **חמדן** שכן בכל שלב הוא עושה את הצעד שנראה הכי טוב כרגע, מבלי לחשוב קדימה. הוא לא תמיד אופטימלי. אבל הגיונית ־ הוא מצליח לקרב לא רע.

זמן הריצה: נשים לב כי מס' האיטרציות הוא n ־ שכן במקרה הגרוע, בכל איטרציה אנחנו מכסים איבר אחד בודד. בכל איטרציה עלינו לעבור על m קבוצות: ולכל אחת לחשב את חיתוכה עם R בזמן $\mathcal{O}(n)$ ולמצוא את החיתוך הגדול ביותר. לכן איטרציה בודדת עלותה $\mathcal{O}(mn)$ וסה"כ כפול n איטרציות זמן הריצה $\mathcal{O}(mn^2)$.

זהו לא אלגוריתם אופטימלי. למשל עולם $U = \{1,2,3,4,5,6\}$ ושלושת הקבוצות:

$$A = \{1,2,3,4\}, B = \{1,2,5\}, C = \{3,4,6\}$$

האלגוריתם האופטימלי יניב את $B \cup C$ להיות הכיסוי, בעלות 2 קבוצות. האלגוריתם שלנו יביא את $A \cup B \cup C$ בעלות של 3 קבוצות. לכן יחס החמדן הוא $\frac{\mathrm{ALG}}{\mathrm{OPT}} = \frac{3}{2}$ במקרה זה.

משפט 97. קיים אלגוריתם קירוב לוגריתמי לבעיית Set-Cover. בפרט, $\mathrm{ALG} \leq \mathrm{OPT} \cdot \ln(n) + 1$.

הוכחה: נניח כי $\mathrm{OPT} = k$, כלומר לכיסוי האופטימלי יש k קבוצות. נניח כי נשארו u איברים לא מכוסים בשלב כלשהו. נבהיר: אנחנו עוצרים את החמדן באמצע הריצה, בכל רגע שנרצה וסופרים כמה איברים לא מכוסים. אזי, אם ברגע זה נותרו u איברים לא מכוסים אזי בסיבוב הבא החמדן יכסה לפחות $\frac{u}{k}$ איברים מהם. מדוע? k הקבוצות האופטימליות מכסות את כל U. בפרט את u האיברים שנותרו. אם כל אחת מהן הייתה מכילה פחות מ$\frac{u}{k}$ מהן, ביחד הן היו מכסות פחות מu $k \cdot \frac{u}{k} = u$ בסתירה, כי הן מכסות את כולם. לכן ישנה קבוצה בהכרח עם רווח $\frac{u}{k}$ לפחות. והחמדן, יקבל לפחות ערך זה.

נסביר ברור יותר. בOPT הכיסוי מכסה את כל האיברים, בפרט את u איברים הנ"ל. יש בו רק k קבוצות לכן בהכרח קבוצה אחת תקבל לפחות $\frac{u}{k}$ מהאיברים שלא מכוסים בשלב זה **באופטימלי**. כעת, החמדן לא מכיר את OPT, וגם לא אכפת לו ממנו. הוא עובר על כל הקבוצות ובוחר את זו עם הכי הרבה איברים לא מכוסים. אחת הקבוצות שהוא בודק היא הקבוצה עם $\frac{u}{k}$ האיברים שלא מכוסים בשלב זה. לכן, הוא מכסה **לפחות** כמוה.

יהיו u_t מס' האיברים שנותרו לאחר t בחירות חמדניות. אזי,

$$u_{t+1} \leq \left(1 - \frac{1}{k}\right) u_t$$

מדוע? בסיבוב ה$t+1$ החמדן מכסה לפחות $\frac{u_t}{k}$ מהאיברים. אזי נותרו לכל היותר $u_t - \frac{u_t}{k} = u_t\left(1 - \frac{1}{k}\right)$ איברים. סה"כ נוכל להפעיל שוב ושוב ולקבל:

$$u_t \leq \left(1 - \frac{1}{k}\right)^t \cdot u_0 = \left(1 - \frac{1}{k}\right)^t n \leq n \cdot \exp\left(-\frac{t}{k}\right)$$

שכן בתחילה מס' האיברים שלא מכוסים הוא $u_0 = n$. נקח $t = \lfloor k \cdot \ln(n) \rfloor + 1$ ואז נקבל כי

$$u_t < n \cdot \exp(-\ln(n)) = n \cdot n^{-1} = 1$$

אם כן u_t הוא מס' שלם, וכן $u_t < 1$ ולכן $u_t = 0$. סה"כ החמדן מצליח לכסות את כולם ומשתמש לכל היותר ב$k \cdot \ln(n) + 1$ קבוצות. כלומר, לאחר $t = \lfloor k \cdot \ln(n) \rfloor + 1$ איטרציות.

נרצה לנתח את האלגוריתם בצורה שונה.

הגדרה 98. חלוקת עלות (Charging). נניח כי כאשר קבוצה שנבחרה מכסה g איברים חדשים, כל אחד מהם מכוסה ב$\frac{1}{g}$. כל איבר מחויב רק כאשר הוא מכוסה בפעם הראשונה. ונסמן:

$$\sum_{x \in U} \mathrm{charge}(x) = \mathrm{ALG}$$

כאן, במקום לספור את הסיבובים נחלק את העלות של כל קבוצה שהחמדן בחר בין האיברים שהיא כיסתה. העלות הכוללת לא משתנה אלא מתפזרת. מה הרעיון? כל קבוצה שהחמדן בוחר עולה 1. את אותו 1, מחלקים בשווה בין האיברים החדשים שהיא כיסתה כך שהעלות הכוללת של האלגוריתם מפוזרת בין האיברים. הסכום הוא ALG כי כל קבוצה שנבחרת מחלקת בדיוק 1 וכל איבר מחויב בדיוק פעם אחת. המטרה שלנו: הכיסוי האופטימלי C^* מכסה כל איבר. אזי, אם כל קבוצה סופגת לכל היותר H חיוביים סך החיוביים הוא לכל היותר $H \cdot \mathrm{OPT}$ $H \cdot |C^*| \leq H \cdot \mathrm{OPT}$.

תהי $S \in \mathcal{F}$ קבוצה כלשהי עם $|S| = s$. היא לא בהכרח קבוצה שהחמדן בחר. נסדר איבריה $x_1, ..., x_s$ לפי הזמן בו כל אחד כוסה לראשונה.

נשים לב כי רגע לפני שx_j כלשהו כוסה, האיברים $x_j, x_{j+1}, ..., x_s$ עדיין לא מכוסים. אלו לפחות $s - j + 1$ איברים של S שעדיין זמינים. כלומר, באותו הרגע הקבוצה S עצמה הייתה מכסה לפחות $s - j + 1$ חדשים. החמדן בוחר את הקבוצה עם הרווח הגדול ביותר, ולכן הקבוצה שהוא בחר מכסה לפחות $s - j + 1$ חדשים. לכן

$$\mathrm{charge}(x_j) \leq \frac{1}{s - j + 1}$$

דרך אחרת להסתכל על זה. בתחילה: x_1 לפני שמכוסה כל s האיברים של S לא מכוסים, לכן אם החמדן היה בוחר את S באותו הרגע הוא היה מכסה s חדשים והעלות $\frac{1}{s}$. עבור האיבר השני, x_1 מכוסה אך $x_2, ..., x_s$ עוד לא. אלו $s-1$ איברים, החמדן מכסה לפחות $s-1$ חדשים ולכן x_2 משלם לכל היותר $\frac{1}{s-1}$.
מכאן: נסמן $H_s = \sum_{r=1}^{s} \frac{1}{r}$ ואז:

$$\sum_{x \in S} \text{charge}\,(x) \leq \sum_{j=1}^{s} \frac{1}{s-j+1} = H_s$$

כעת, יהי $d = \max_{S \in \mathcal{F}} |S|$ גודל הקבוצה המקסימלי. ויהי C^* כיסוי אופטימלי. אזי:

$$\text{ALG} = \sum_{x \in U} \text{charge}\,(x) \leq \sum_{S \in C^*} \sum_{x \in S} \text{charge}\,(x) \leq \sum_{S \in C^*} H_{|S|} = |C^*| \cdot H_d \leq H_d \cdot \text{OPT}$$

הסבר למעברים: משמאל לימין - לפי הגדרה, ואז ה$\leq$ הראשון משמאל הוא כיוון שC^*מכסה כל איבר. לכן כל $S \in C^*$ ניתן לעבור עליה ואז על איבריה בחיובים. ה$\leq$ השני נכון כי ראינו קודם $\sum_{x \in S} \text{charge}\,(x) \leq H_s$. ה$\leq$ השלישי נכון כי $|S| \leq d$ לכל $S \in \mathcal{F}$.
אנחנו יודעים כי $H_s \approx \ln(s) + 1$ ולכן אצלנו: $H_d \approx 1 + \ln(d)$. סה"כ:

משפט 99. האלגוריתם שהצגנו מקיים יחס קירוב של $\text{ALG} \leq \text{OPT} \cdot (1 + \ln(d))$.

אלגוריתם קירוב זה נחשב לטוב יותר, קודם ראינו אלגוריתמי קירוב שתלויים בn. $\ln(n)$ וכו'.
כעת רואים בd, שיתכן שהוא משמעותית קטן מn. הוא בכל מקרה לא עולה עליו.

TSP ו־TSP מטרי

נגדיר את הבעיה הבאה:

הגדרה 100. בעיית TSP מוגדרת להיות:
קלט: גרף שלם $G = (V, E)$ לא מכוון עם $n \geq 3$ קודקודים. וכן פונקציית עלות $c : E \to \mathbb{R}^+$ אי שלילית על הקשתות.
פלט: מעגל המילטוני בעלות מינימלית. כלומר, מעגל המבקר בכל קודקוד פעם אחת וחוזר להתחלה במשקל המינימלי.

נעיר כי מדובר בגרף שלם, כלומר ישנה קשת בין כל שני קודקודים. ישנן 2 גרסאות של בעיה זו:
- בעיה ראשונה: General TSP - עלויות הקשתות שרירותיות.
- בעיה שנייה: Metric TSP - העלויות של הקשתות סימטריות ומקיימות את אי שוויון המשולש.

שתי הגרסאות של הבעיה הן NPC, אך לבעיה הכללית אין קירוב ביחס קבוע ולמטרי יש קירוב שנדון בהמשך.

משפט 101. לTSP הכללי אין אף אלגוריתם ρ־קירוב לכל $\rho \geq 1$, אלא אם $NP = P$.
הוכחה: על מנת להוכיח טענה זו, נבצע רדוקציה. נניח בשלילה כי היה קיים אלגוריתם כזה, והיינו יוצרים אלגו' פולינומי שמקרב את בעיית המעגל ההמילטוני שהיא NPC בסתירה.
בהינתן קלט של HC, גרף $G = (V, E)$ נשלים אותו לגרף שלם ונקבע את פונקציית המשקל להיות:

$$c(u,v) = \begin{cases} 1 & \{u,v\} \in E \\ M & \{u,v\} \notin E \end{cases} \qquad \text{s.t } M = \lceil \rho n \rceil + 1$$

כלומר הקשתות המקוריות זולות, ואלו החדשות יקרות מאוד. מדוע בחרנו את M באופן הזה? על מנת שגם קשת חדשה **אחת** תהפוך את הסיור ליותר יקר מ$\rho \cdot n$ ־ זה סיור שתמיד אפשרי.
כעת כיצד האלגו' יפעל? מפעילים את אלגוריתם הקירוב. הוא מחזיר ערך שלא יכול לעלות על $\rho \cdot |OPT|$. וכן $|OPT| = n$ ולכן הוא לא יכול לעלות על $\rho \cdot n$: שכן נשים לב כי האופטימלי הוא מעגל המילטון n שתמיד קיים בגרף השלם.
לכן, אם אלגו' הקירוב החזיר ערך שקטן מρn אזי בG יש מעגל המילטוני, ונשיב 1 ואחרת נשיב שאין (0) מעגל המילטוני בG.
אם בG יש מעגל המילטוני, האלגוריתם מזהה אותו. אם אין מעגל המילטוני, כל סיור ישתמש בקשת אחת חדשה לפחות ולכן **כל** פלט אפשרי יהיה לפחות $M > \rho n$.
סה"כ בנינו אלגוריתם פולינומי לHC, בסתירה לכך שהיא בעיה NPC. לכן, לא קיים אף אלגוריתם קירוב לבעיה זו.

אם כן, אין לנו יכולת לקרב את TSP הכללית ולכן נתמקד במקרה המטרי. כעת נניח כי לכל שלושה קודקודים u, v, z מתקיים:

$$c(u,v) \leq c(u,z) + c(z,v) \quad \wedge c(u,v) = c(v,u)$$

משפט 102. לכל $k \geq 1$ ולכל מסלול $(v_0, ..., v_k)$ מתקיים:

$$c(v_0, v_k) \leq \sum_{i=1}^{k} c(v_{i-1}, v_i)$$

הוכחה: ע"י אינדוקציה.
בסיס: $k = 1$. אזי, מדובר בשוויון בין שני הצדדים.
צעד: נניח נכונות הטענה עבור k. אזי:

$$c(v_0, v_{k+1}) \leq c(v_0, v_k) + c(v_k, v_{k+1}) \leq \sum_{i=1}^{k} c(v_{i-1}, v_i) + c(v_k, v_{k+1}) = \sum_{i=1}^{k+1} c(v_{i-1}, v_i)$$

נתקדם. יהי T עץ פורש מינימלי (מה הקשר?), מיד נגלה מה הקשר.
יהי C^* סיור TSP אופטימלי בגרף G.
אזי, הסרה של קשת כלשהי e מהסיור האופטימלי C^* משאירה **מסלול** שעובר בין כל הקודקודים והוא בפרט עץ פורש: מדוע עץ פורש? קשיר, וכן מס' הקשתות בו $n - 1$. נשים לב כי כיוון שהעליות אי שליליות מתקיים:

$$\text{cost}(T) \leq \text{cost}(C^* \setminus \{e\}) \leq \text{cost}(C^*) = \text{OPT}$$

באשר $\leq$ ראשון נכון כי $C^* \setminus \{e\}$ הוא **עץ פורש**, וכן T הוא MST ובפרט עץ הפורש בעל המשקל הקטן ביותר. ה$\leq$ השני נכון כי הורדנו קשת, לכן העלות רק קטנה יותר או שווה ביחס לC^*.

מה קיבלנו? את OPT איננו יודעים לחשב. אבל את $\text{cost}(T)$ כן. הרי, MST הוא בזמן פולינומי.
כלומר ראינו כי $\text{cost}(T) \leq \text{OPT}$. כלומר, מהסיור האופטימלי ניתן למחוק צלע אחת ולקבל מסלול שעובר בכל הקודקודים והרי הוא עץ פורש. אבל, עץ הוא לא סיור: כי אין בו מעגל שחוזר להתחלה. על מנת להפוך אותו לסיור, אנחנו נראה אלגוריתם שמקיים $\text{ALG} \leq 2\text{cost}(T)$, ומאי השוויון שראינו נקבל $\text{ALG} \leq 2\text{OPT}$ ונקבל סה"כ אלגוריתם קירוב ביחס 2 לבעיית TSP המטרי. שוב, אלגוריתם שמחזיר לנו סיור.

ALGORITHM ():
1. מחשבים MST, ע"י הרצת פרים או קרוסקל ונסמנו T.
2. יוצרים שני עותקים של כל קשת בעץ. זהו מולטיגרף $2T$.
3. מוצאים מעגל אוילר W במולטיגרף שהתקבל. נסמנו $W=(w_0,...,w_m)$ וכן $w_0=w_m$.
4. שומרים את הביקור הראשון בכל קודקוד, מדלגים על ביקורים חוזרים וחוזרים להתחלה. כלומר בונים את הסיור H: עוברים על W משמאל לימין, שומרים כל קודקוד רק במופע הראשון שלו ־ מקבלים $u_1,...,u_n$ ומחזירים $H=(u_1,...,u_n,u_1)$

נסביר מה קורה כאן, כי זה יכול לבלבל: מולטי גרף הוא גרף שבו בין זוג קודקודים מותרות כמה קשתות שונות. כאן, כל קשת של העץ מופיעה פעמיים. **מעגל אוילר** הוא מעגל שעובר בכל קשת בדיוק פעם אחת, בניגוד למעגל המילטוני שעובר בכל קודקוד פעם אחת. בכל קודקוד ניתן לבקר כמה פעמים.

מדוע קיים מעגל אוילר בגרף? הכפלת כל קשת, יוצרת בעצם מצב שבגרף החדש שנוצר דרגת כל קודקוד היא זוגית. בנוסף, הגרף קשיר וכן לפי משפט ממבנים בדידים: קיים מעגל אוילר בגרף.

בסעיף 4, אנחנו מקבלים מעגל אוילר שהוא רשימת קודקודים W. יש בה חזרות, ובTSP דורשים לבקר בכל קודקוד פעם אחת בדיוק. לכן עוברים על הרשימה משמאל לימין: קודקוד המופיע בפעם הראשונה נשאר וקודקוד שכבר ראינו נמחק. בסוף מוסיפים חזרה לקודקוד ההתחלה על מנת ליצור מעגל.

אלגוריתם למציאת מעגל אוילר בגרף: פשוט ־ בוחרים קודקוד שרירותי והולכים רק על קשתות שעוד לא השתמשנו בהם, כל קשת שעוברים עליה מוחקים, וממשיכים עד שנתקעים. מתי נתקעים? בקודקוד ההתחלה: כל הדרגות זוגיות ולכן כל פעם שנכנסים לקודקוד אחר נשארת קשת שאפשר לצאת בה, והתוצאה היא מעגל סגור. אם נותרו קשתות לא משומשות, ישנו קודקוד על המעגל שיש לו קשת כזו לא משומשת: אזי מתחילים ממנו מעגל נוסף באותה השיטה, ומשחילים אותו לתוך המעגל הראשי מאותה נקודה. חוזרים על שלב 2 עד שכל הקשתות נוצרו. זמן הריצה הוא $\mathcal{O}(|E|)$.

משפט 103. האלגוריתם מחזיר מעגל המילטוני.

הוכחה: ראשית ראינו כי אכן ב$2T$ ישנו מעגל אוילר, ולכן שלב שלוש של האלגוריתם מוגדר היטב. כמו כן, נטען:

למה. כל קודקוד של G מופיע בW לפחות פעם אחת. **הוכחה:** T הוא עץ פורש עם $n\geq 2$ קודקודים. לכן כל קודקוד v נוגע בלפחות קשת אחת של T ובפרט של $2T$. מעגל אוילר עובר בכל הקשתות, לכן עובר גם בהכרח בקשת שנוגעת בv. לכן v נמצא בW.

כעת נוכיח כי H המוחזר הוא מעגל המילטוני:

ראשית, כל קודקוד מופיע ב$u_1,...,u_n$ לפחות פעם אחת. נשים לב כי כל $v\in V$ מופיע בW בהכרח במעגל האוילר, כלומר יש לו מופע ראשון ואנחנו שומרים על מופע ראשון בH.

כמו כן, אף קודקוד לא מופיע פעמיים.

נשים לב כי זהו הגרף השלם G, לכן $u_1,...,u_n$ המוחזרים הם n קודקודים שכל אחד מופיע בדיוק פעם אחת. כל הקשתות קיימות כי זהו הגרף השלם, ולכן H הוא מעגל העובר בכל קודקוד בדיוק פעם אחת. כנדרש.

משפט 104. מדובר באלגוריתם 2־קירוב. כלומר, $c(H)=\mathrm{ALG}\leq 2\cdot\mathrm{OPT}$

הוכחה: ראשית, ראינו כי עבור T שהוא עץ פורש מינימלי מתקיים כי $c(T)\leq\mathrm{OPT}$. כיוון ש:

$$\mathrm{cost}(T)\leq\mathrm{cost}(C^*\setminus\{e\})\leq\mathrm{cost}(C^*)=\mathrm{OPT}$$

כעת, נטען כי $c(W)=2c(T)$. שכן, מעגל אוילר עובר בכל קשת של $2T$ בדיוק פעם אחת. לכן:

$$c(W)=c(2T)=2c(T)$$

כעת, ראינו קודם כי $c(v_0,v_k)\leq\sum_{i=1}^{k}c(v_{i-1},v_i)$. נטען כי $c(H)\leq c(W)$. כלומר, עלות מעגל ההמילטון המוחזר לא עולה על זה האוילרי.

יהי i_j המקום בW שבו u_j מופיע לראשונה. אזי $u_1=w_0$ ולכן $i_1=0$. וכן $u_1,...,u_n$ נאספו לפי סדר ההופעה הראשונה בW ולכן

$$0=i_1<.....<i_n\leq m$$

האינדקסים הללו מחלקים את W לn קטעים רצופים שאינם חולקים רשתות ומכסים יחד את כל W:

$$[i_1,i_2],....,[i_{n-1},i_n],[i_n,m]$$

לכל קשת של H נתאים קטע:
- קשת (u_j, u_{j+1}) עבור $j < n$. הקטע $\left(w_{i_j}, ..., w_{i_{j+1}}\right)$ הוא הליכה מu_j לu_{j+1} ולפי למה שראינו קודם אזי

$$c\left(u_j, u_{j+1}\right) \leq \sum_{t=i_j+1}^{i_{j+1}} c\left(w_{t-1}, w_t\right)$$

- קשת הסגירה (u_n, u_1). הקטע $w_{i_n}, ..., w_m$ הוא הליכה מu_n ל$w_m = w_0 = u_1$ ולפי למת העזר חסום בסכום הקשתות של קטע הזה. סהכ נקבל

$$c\left(H\right) = \sum_{j=1}^{n} c\left(u_j, u_{j+1}\right) \leq \sum_{j=1}^{n} \sum_{t=i_j+1}^{i_{j+1}} c\left(w_{t-1}, w_t\right) = c\left(W\right)$$

סה"כ קיבלנו

$$\mathrm{ALG} = c\left(H\right) \leq c\left(w\right) = 2c\left(T\right) \leq 2\mathrm{OPT}$$

לכן מדובר באלגוריתם 2־קירוב.

זמן הריצה של MST על גרף שלם הוא $\mathcal{O}\left(n^2\right)$ וכך גם זמן ריצת האלגוריתם.

אלגוריתם $\frac{3}{2}$ קירוב

בעץ הכפול שילמנו עוד $\mathrm{cost}\left(T\right)$ רק על מנת שהדרגות יהיו זוגיות. אממה, רק לחלק מהקודקודים יש דרגה אי זוגית ורק אותם צריך לתקן. לכן נתקן רק אותם, בזול ככל שאפשר: **זיווג מינימלי.** ונוכיח שהתיקון עולה לכל היותר $\frac{\mathrm{OPT}}{2}$.
יהי O אוסף הקודקודים בעלי דרגה אי זוגית בעץ T שהוא MST. נגדיר את האלגוריתם:

ALGORITHM():
1. מחשבים זיווג מושלם במשקל מינימלי M על O.
2. יוצרים מולטיגרף $T \cup M$.
3. מוצאים מעגל אוילר, ואז מדלגים על ביקורים חוזרים.

מה הרעיון? בעץ הכפול הכפלנו את כל העץ רק כדי שכל הדרגות יהיו זוגיות. חבל, כעת רק הקודקודים בO צריכים עוד קשת אחת למעשה על מנת להגדיל דרגתם. לכן, מוסיפים קשת אחת לכל אחד מהם ומחברים אותם בזוגות.
זיווג מושלם על O אומר שמחלקים את O לזוגות, עם קשת לכל זוג.
ואז, מאחדים את הזיווג M עם העץ T: זהו איחוד שמותיר כפילויות. אם קשת של M כבר בT ישנם שני עותקים.
נשים לב כי $|O|$ זוגי, שכן סכום הדרגות בכל גרף הוא $2|E|$, זוגי. סכום הדרגות הזוגיות זוגי. לכן גם סכום האי זוגיות זוגי. זה אפשרי רק אם יש מס' זוגי שלהן.

משפט 105. האלגוריתם מחזיר מעגל המילטוני.
הוכחה: כמו שאמרנו, $|O|$ זוגי לכן אכן קיים זיווג מושלם שכזה. כעת, ב$T \cup M$ יש מעגל אוילר שכן:
הוא קשיר, מכיל בהכרח את T וכן כל הדרגות בו זוגיות. קודקוד בעל דרגה זוגית קודם נשאר כזו וקודקוד בO מקבל מM קשת אחת בדיוק שמחברת אותו לקודקוד מדרגה אי זוגית אחרת שהיה בזיווג. לכן דרגת כל הקודקודים זוגית וישנו מעגל אוילר.
וכן, כעת H המילטוני: ההוכחה זהה לעץ הכפול, T הוא פורש ולכן כל קודקוד בו מופיע בW, וסה"כ אותה הוכחה מניבה כי H הוא המילטוני.

משפט 106. מתקיים $\mathrm{ALG} \leq \frac{3}{2}\mathrm{OPT}$
הוכחה: נשים לב כי:

$$c\left(H\right) \leq c\left(W\right) = c\left(T\right) + c\left(M\right) \leq \mathrm{OPT} + \frac{\mathrm{OPT}}{2} = \frac{3}{2}\mathrm{OPT}$$

כל מה שיש להוכיח הוא רק מדוע $c\left(M\right) \leq \frac{\mathrm{OPT}}{2}$.

יהי הסיור האופטימלי H^*, ונדלג על כל הקודקודים שלא בO. נקבל מעגל C שעובר רק בקודקודים של O באותו סדר. לפי אי שוויון המשולש $c(C) \leq \mathrm{OPT}$. בC יש $|O|$ קשתות וזהו מס' זוגי. לכן ניתן לצבוע את קשתותיו לסירוגין, בשני צבעים. אחת אדומה, אחת כחולה וכן הלאה. הקשתות האדומות יוצרות זיווג מושלם M_1 על O וכך גם הכחולות M_2. מתקיים $c(C) = c(M_1) + c(M_2) \leq \mathrm{OPT}$. מכאן, M הוא הזיווג המושלם המינימלי על O. בפרט,

$$c(M) = \min\{c(M_1), c(M_2)\} \leq \frac{1}{2}\mathrm{OPT}$$

נעיר כי C הוא ניסוי מחשבתי, האלגו' לא בונה אותו ואי אפשר לבנותו בפועל כי איננו יודעים את הסיור האופטימלי. אנחנו רק לוקחים את הסיור האופטימלי שעובר בכל הקודקודים בסדר מסוים. אנחנו כותבים את הסדר הנ"ל ומוחקים את הקודקודים שלא שייכים לO. נותרו סדרת קודקודים של O בסדר שבו H^* ביקר בהם. נוכל לחבר אותם לפי הסדר - תזכורת זה הגרף המלא, ולקבל מעגל. במעגל הזה, יש מס' זוגי של קשתות ולכן הוא ניתן לצביעה בשני צבעים. כל צבע הוא זיווג. ישנם סה"כ שני זיווגים אפשריים, ולכן הזיווג על O הוא המינימלי מבין שניהם. שניהם יחד בעלות של $c(M_1) + c(M_2) \leq \mathrm{OPT}$ ולכן בהכרח המינימום לא יכול לעבור על $\frac{\mathrm{OPT}}{2}$, כנדרש.

0>0